Probabilistic Robotics

Inspired by TTIC 31170 by Dr. Matthew Walter.

Mihir Rao

How to Read These Notes

A robot never receives a perfect statement of where it is. It receives wheel rotations, distances, images, and other clues. Each clue is incomplete. The central question of these notes is how to combine those clues without pretending that an uncertain quantity has become exact.

Think of a robot moving through a hallway with several similar doors. A door sensor may tell it that it is near a door without telling it which door. A wheel encoder may suggest that it moved one meter without ruling out a little slip. A useful estimate keeps several possibilities alive, predicts how they move, and changes their relative plausibility when new evidence arrives. The same reasoning eventually lets the robot estimate an unfamiliar map.

The material is organized by ideas rather than class meetings. Part I develops probability and graphical models. Part II turns those ideas into algorithms for hidden states: finite-state inference, the Bayes filter, Gaussian filters, and particle filters. Part III estimates the map together with the robot, then explains information form, marginalization, and graph optimization. Each part uses the tools developed before it.

Read an equation as a sequence of operations. Ask which quantities are known, which remain uncertain, what is multiplied, what is summed or integrated out, and what must be normalized. Numerical examples use small invented models so that these operations can be checked by hand. Their numbers illustrate the methods; they are not measurements from a robot experiment.

Notation	Meaning
X, x	A random variable and one candidate value.
x_t	State at time t; in SLAM, robot pose.
z_t	Measurement received at time t.
u_t	Input for the transition from $t-1$ to t.
$x_{0:t}$	The sequence $x_0, x_1, \ldots, x_t$.
μ, Σ	Mean vector and covariance matrix.
R, Q	Motion-noise and measurement-noise covariance.
Λ, η	Gaussian information matrix and vector.

A conditioning bar means "given." A transpose, written $\top$, turns a column into a row. A superscript -1 on a matrix denotes an inverse; implementations usually solve the corresponding linear system instead. Some local derivations reuse short letters for matrix blocks, and define those meanings where needed.

Contents

How to Read These Notes 2

I Probability and Models 5

1 Robot Beliefs and Probability 5
- 1.1 Robots must reason under uncertainty . . . 5
- 1.2 Probability as a language for uncertainty . . . 6
- 1.3 Random processes over time . . . 9
- 1.4 Means, covariance, and stationarity . . . 10

2 Graphical Models and Inference 14
- 2.1 Why graphical models? . . . 14
- 2.2 Bayesian networks and directed separation . . . 15
- 2.3 Markov random fields and undirected separation . . . 17
- 2.4 Converting a Bayesian network to an MRF . . . 19
- 2.5 Inference with evidence: from factors to messages . . . 20
- 2.6 Variable elimination in a graph with cycles . . . 23

II Estimating a Hidden State 25

3 Hidden States and Sequential Inference 25
- 3.1 The Markov assumption . . . 25
- 3.2 Hidden Markov models and forward–backward inference . . . 26
- 3.3 MAP sequences and the Viterbi algorithm . . . 30
- 3.4 Filtering, smoothing, and robot localization . . . 32
- 3.5 Learning an HMM from observations . . . 33

4 The Bayes Filter and Robot Localization 37
- 4.1 What belongs in the robot's state? . . . 37
- 4.2 Generative laws and belief distributions . . . 38
- 4.3 Deriving the Bayes filter . . . 40
- 4.4 A complete three-cell Bayes-filter example . . . 41
- 4.5 A hallway with a door sensor . . . 42

5 Least Squares and Gaussian Filtering 44
- 5.1 Least squares estimation . . . 44
- 5.2 Gaussian distributions . . . 46
- 5.3 The Kalman filter . . . 49
- 5.4 The extended Kalman filter . . . 53

6 Sampling and Particle Filters 57
- 6.1 The discrete Bayes filter . . . 57
- 6.2 Monte Carlo approximation . . . 59
- 6.3 Sampling from distributions . . . 59
- 6.4 Importance sampling . . . 60
- 6.5 Sequential importance sampling . . . 61
- 6.6 Choosing the proposal . . . 63
- 6.7 Resampling . . . 65

III Learning the Map **69**

7 Joint Localization and Mapping **69**
7.1 What the robot knows, and what it must estimate . . . 69
7.2 Online SLAM and full SLAM . . . 71
7.3 A Gaussian representation of pose and landmarks . . . 74
7.4 Deriving the EKF prediction . . . 75
7.5 Deriving the measurement update . . . 76
7.6 Growing the map without losing shared uncertainty . . . 78
7.7 A complete one-dimensional SLAM calculation . . . 81
7.8 What limits EKF SLAM, and why pose graphs help . . . 83

8 Information Filters and Marginalization **84**
8.1 Why change the Gaussian representation? . . . 84
8.2 Deriving Gaussian information form . . . 84
8.3 What the information matrix says about a graph . . . 86
8.4 Deriving an information-form observation update . . . 87
8.5 Conditioning and marginalization are different operations . . . 89
8.6 Deriving time prediction in the information filter . . . 91
8.7 Sparsity, approximation, and uncertainty . . . 93

9 Graph Based SLAM **95**
9.1 Keeping old poses: full SLAM as an optimization problem . . . 95
9.2 Turning nonlinear residuals into a linear least-squares step . . . 97
9.3 Worked loop closure: how a discrepancy is distributed . . . 99

Part I

Probability and Models

Chapter 1: Robot Beliefs and Probability

1.1 Robots must reason under uncertainty

Robots face uncertain state, noisy measurements, and unpredictable motion. Probability represents both current uncertainty and future outcomes.

The state contains the quantities needed to describe the system. For an underwater vehicle, these include position, orientation, linear velocity, and angular velocity. An inertial navigation system combines inertial measurements with a filter. A magnetometer helps estimate heading but can be biased; a Doppler velocity log measures velocity relative to the bottom only when sufficiently close to it. Cameras and sonar provide additional measurements.

For a mobile robot, the state may be position and heading. Wheel encoders estimate forward velocity, a laser scanner measures obstacle distance and bearing, and a camera provides images. Unmodeled dynamics and wheel slip introduce motion errors. Dead reckoning accumulates motion estimates from an initial pose, so these errors can accumulate into drift and misaligned maps.

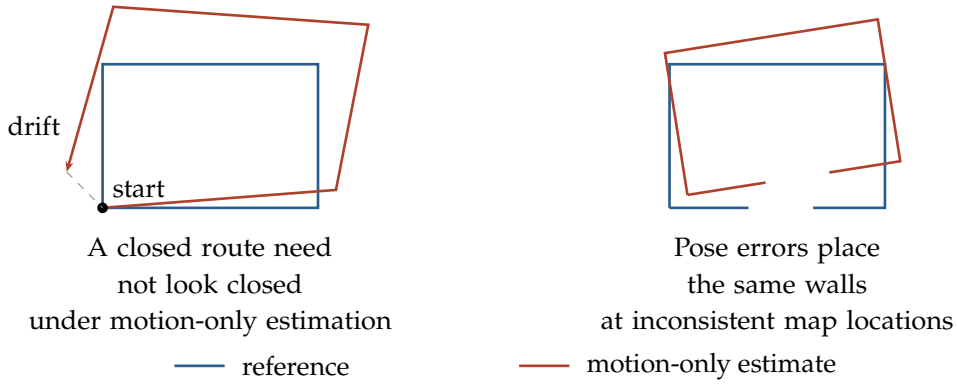


Figure 1: Two consequences of accumulated pose error. Left: motion estimates can fail to close a loop. Right: inconsistent poses can misalign repeated views of walls. The drawings are schematic.

1.1.1 An estimate is more than a dot on a map

Imagine pointing to a single hallway door and saying, "The robot is here." That statement hides whether the door is almost certain or merely the best of several weak guesses. A BELIEF DISTRIBUTION attaches a probability to each candidate state. It is like keeping several possible explanations on a desk, with a weight attached to each, rather than immediately throwing all but one away. The candidates are possible states of one robot, not several robots.

If two separated doors are equally plausible, the mean position lies between them. There may be no door there at all. This is one reason that a mean and a confidence number cannot describe every useful belief. Later, Gaussian filters will use that compact description when it is appropriate, while particle filters will keep more complicated shapes.

1.2 Probability as a language for uncertainty

Definition 1.1 [PROBABILITY SPACE]. A probability model needs three things: possible outcomes, events we can ask about, and probabilities for those events. A PROBABILITY SPACE is a triple $(\Omega, \mathcal{S}, P)$. The sample space Ω contains possible outcomes. The sigma-algebra $\mathcal{S}$ is the collection of allowed events, each a subset of Ω; it is closed under complements and countable unions. The probability measure P is the function that assigns a probability to each event.

Probabilities cannot be negative, all possible outcomes together have probability one, and disjoint events add without overlap. Formally:

$$P(A) \geq 0, \qquad P(\Omega) = 1, \qquad P\left(\bigcup_{i=1}^{\infty} A_i\right) = \sum_{i=1}^{\infty} P(A_i) \quad \text{for pairwise disjoint } A_i.$$

The empty event has $P(\varnothing) = 0$. For two events, count the overlap only once:

$$P(A \cup B) = P(A) + P(B) - P(A \cap B).$$

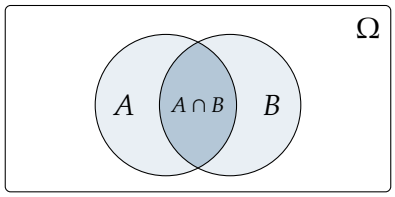


Figure 2: The intersection contributes to both $P(A)$ and $P(B)$, so inclusion–exclusion subtracts it once. Conditioning on B restricts attention to the right-hand event and renormalizes its probability to one.

Conditioning means updating our probabilities after learning that an event occurred. For $P(B) > 0$, CONDITIONAL PROBABILITY IS

$$P(A \mid B) = \frac{P(A \cap B)}{P(B)}.$$

Rearranging gives the product rule $P(A \cap B) = P(A \mid B)P(B)$. Applying this rule repeatedly yields the CHAIN RULE:

$$p(x_1, \ldots, x_n) = \prod_{i=1}^{n} p(x_i \mid x_1, \ldots, x_{i-1}), \tag{1.1}$$

where the first factor is $p(x_1)$. Any ordering gives a valid chain rule; a useful graphical model allows many variables to be removed from these conditioning sets.

Theorem 1.2 [BAYES' RULE]. *For events with positive conditioning probability,*

$$P(A \mid B) = \frac{P(B \mid A)P(A)}{P(B)}.$$

For a discrete unknown X and observed $Y = y$, the corresponding update is

$$p(x \mid y) = \frac{p(y \mid x)p(x)}{\sum_{x'} p(y \mid x')p(x')}. \tag{1.2}$$

The PRIOR $p(x)$ describes uncertainty before observing y. The LIKELIHOOD $p(y \mid x)$ measures how compatible that observation is with a candidate state. The POSTERIOR $p(x \mid y)$ combines them. The denominator is the probability of the evidence and makes the posterior sum to one. Continuous variables use densities and integrals instead of probability masses and sums.

A RANDOM VARIABLE $X : \Omega \to \mathcal{D}$ maps an outcome to a value in its domain $\mathcal{D}$. It is measurable: an event about its value corresponds to an allowed event in Ω. A realization x is one value it takes. For a discrete variable, $p(x) = P(X = x)$; continuous variables use densities and integrals. A RANDOM VECTOR collects several random variables and has a joint distribution over their values.

1.2.1 Reading probability notation as instructions

For a discrete position variable X, the expression $p(X = 2) = 0.3$ means that candidate position 2 receives probability 0.3. We often shorten it to $p(x)$ when x is the candidate being considered. A joint distribution $p(x, z)$ assigns a probability to a pair of values. A marginal $p(x)$ adds over possible readings: $p(x) = \sum_z p(x, z)$. A conditional $p(x \mid z)$ keeps the reading fixed and asks how uncertainty about position changes.

For a continuous position, $p(x)$ is a *density*. A single exact value usually has probability zero, and a density may be greater than one. Probability belongs to an interval: $P(a \le X \le b) = \int_a^b p(x)\,dx$. The area under the entire density is one. A uniform position on a hallway segment of length $1/2$

meter has density 2 per meter; the probability of the first $1/4$ meter is $2 \times (1/4) = 1/2$. A tall curve need not imply an impossible probability.

The symbol $\propto$ means equality up to a factor that is constant in the unknown. For example, $p(x \mid z) \propto p(z \mid x)p(x)$ instructs us to multiply likelihood by prior, then divide every entry by their common total. That last step is normalization. It preserves relative weights while making all probabilities add to one.

1.2.2 A door sensor: Bayes' rule with actual counts

Suppose the robot is near a door with prior probability $1/5$. A sensor reports "door" with probability $9/10$ near a door and $1/10$ elsewhere. It now reports "door." The likelihood is a model of sensor behavior *given position*; it is not yet a probability of position given the reading.

1. **Weight the door hypothesis.** Its unnormalized posterior weight is $(9/10)(1/5) = 9/50$.
2. **Weight the alternative.** Being away from a door has prior $4/5$, so its weight is $(1/10)(4/5) = 4/50$.
3. **Add the weights.** The evidence probability is $13/50$.
4. **Normalize.** The posterior probability of being near a door is $(9/50)/(13/50) = 9/13$, about 0.692.

A count interpretation gives the same answer. Out of 1000 comparable cases, about 200 are near doors: 180 produce positive reports. Of the 800 other cases, about 80 produce false positives. Among the 260 positive reports, 180 are genuine, giving $180/260 = 9/13$. The sensor's 90% detection rate and the posterior 69.2% answer different questions.

If a second, conditionally independent reading also reports "door," and the robot has not moved, multiply by the likelihood again:

$$P(\text{door} \mid \text{two reports}) = \frac{(9/10)^2(1/5)}{(9/10)^2(1/5) + (1/10)^2(4/5)} = \frac{81}{85}.$$

The independence assumption matters. Copying the first sensor packet and processing it twice does not create a second measurement. Neither does a persistent fault necessarily supply independent evidence on consecutive reads.

1.2.3 Mean, variance, and the information a mean loses

Suppose a robot is at position 0 with probability $1/4$ and position 2 with probability $3/4$. Its mean is the probability-weighted balance point:

$$\mu = \mathbb{E}[X] = 0\left(\frac{1}{4}\right) + 2\left(\frac{3}{4}\right) = \frac{3}{2}.$$

Variance is the average *squared* displacement from that mean:

$$\mathrm{Var}(X) = \frac{1}{4}\left(0 - \frac{3}{2}\right)^2 + \frac{3}{4}\left(2 - \frac{3}{2}\right)^2 = \frac{9}{16} + \frac{3}{16} = \frac{3}{4}.$$

The standard deviation is $\sqrt{3}/2$, in the same distance units as X; variance has squared-distance units. The mean is 1.5, even though this model assigns probability zero to being exactly there. Keeping the whole distribution retains the two hypotheses that the mean alone conceals.

For two variables, covariance records shared deviations. Suppose two landmark estimates contain the same uncertain position offset B plus independent measurement errors E_1, E_2:

$$M_1 = B + E_1, \quad M_2 = B + E_2, \quad \mathrm{Var}(B) = 4, \quad \mathrm{Var}(E_1) = \mathrm{Var}(E_2) = 1.$$

With all three errors independent and zero mean, each landmark has variance 5, but $\mathrm{Cov}(M_1, M_2) = 4$. Their difference cancels the common offset, so

$$\mathrm{Var}(M_2 - M_1) = 5 + 5 - 2(4) = 2.$$

Ignoring covariance would incorrectly give 10. This common-offset example will explain why SLAM must keep robot and landmark uncertainty together.

1.3 Random processes over time

Definition 1.3 [Stochastic process]. A stochastic process is a family of random variables indexed by time or another coordinate. Write $X[n, \omega]$ for discrete time and $X(t, \omega)$ for continuous time. Fixing ω gives a sample path; fixing time gives a random variable across outcomes.

Time and value space are separate choices:

	Discrete values	Continuous values
Discrete time	Discrete random sequence	Discrete random process
Continuous time	Continuous random sequence	Continuous random process

The index need not be time: an image has two spatial indices, and a video has two spatial indices and one temporal index.

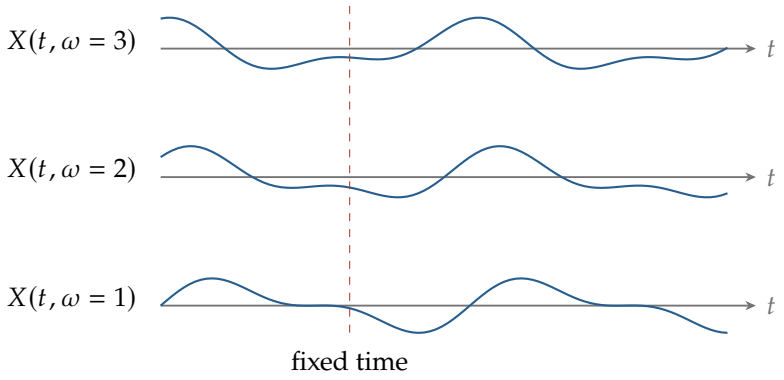


Figure 3: Schematic sample paths. Each row fixes an outcome; the dashed vertical line selects values of one random variable across outcomes. Curves illustrate the distinction and are not measured data.

1.3.1 Examples

For a robot in a discretized one-dimensional environment, $X_t \in \{1, \dots, n\}$ is its cell at time t. Wheel slip and model error make its trajectory uncertain. Write $X^T = (X_0, \dots, X_T)$ for the history through time T; this superscript denotes a history, not a power or transpose.

In language modeling, X_i is the word at position i in an utterance, for example "wreck a nice beach." Such sequences arise in speech recognition and machine translation. In binary communication, $X_t \in \{0, 1\}$ is a transmitted bit in a sequence sent through a noisy channel.

1.4 Means, covariance, and stationarity

For a real vector-valued process with finite second moments, define

$$\begin{aligned} \mu_x(t) &= \mathbb{E}[x(t)], \\ K_{xx}(t, s) &= \mathbb{E}[(x(t) - \mu_x(t))(x(s) - \mu_x(s))^\top], \\ R_{xx}(t, s) &= \mathbb{E}[x(t)x(s)^\top] = K_{xx}(t, s) + \mu_x(t)\mu_x(s)^\top. \end{aligned} \tag{1.3}$$

These are the mean, AUTOCOVARIANCE, and AUTOCORRELATION functions. The discrete-time definitions replace t, s by n, m and parentheses by brackets. Covariance measures joint variation after subtracting the means; autocorrelation retains the mean contribution.

Definition 1.4 [Wide-sense stationarity]. A process is wide-sense stationary if its mean is constant and its autocorrelation depends only on the lag $\tau = t - s$:

$$\mu_x(t) = \mu, \qquad R_{xx}(t, s) = R_{xx}(t - s).$$

For a real scalar process, $R_{xx}(\tau) = R_{xx}(-\tau)$. For a real vector process, the corresponding identity includes a transpose: $R_{xx}(\tau) = R_{xx}(-\tau)^\top$.

1.4.1 Random walk

Consider the discrete-time random walk

$$x[n] = x[n-1] + w[n], \qquad x[n] = x[0] + \sum_{k=1}^{n} w[k],$$

where $x[0]$ is deterministic and the increments $w[k]$ are independent and identically distributed with mean μ_w and variance σ^2. Equivalently,

$$\mathbb{E}[(w[n] - \mu_w)(w[m] - \mu_w)] = \sigma^2 \delta[n - m],$$

where $\delta[0] = 1$ and $\delta[k] = 0$ for $k \neq 0$ is the discrete unit impulse. Linearity of expectation and independence give

$$\mathbb{E}[x[n]] = x[0] + n\mu_w, \qquad \mathrm{Var}(x[n]) = n\sigma^2. \tag{1.4}$$

The first formula reduces to $n\mu_w$ when $x[0] = 0$. Even with zero drift, a nondegenerate random walk is not wide-sense stationary: its variance grows.

For $n > m$, only the increments after time m remain in the difference:

$$x[n] - x[m] = \sum_{k=m+1}^{n} w[k].$$

Consequently,

$$\mathbb{E}[x[n] - x[m]] = (n - m)\mu_w, \qquad \mathrm{Var}(x[n] - x[m]) = (n - m)\sigma^2.$$

Increments over disjoint intervals are independent, since they use disjoint sets of independent $w[k]$. Also,

$$\mathrm{Cov}(x[n] - x[m], x[k]) = 0 \qquad (n > m \geq k),$$

because $x[k]$ uses only increments at or before k. If the increments were assumed only uncorrelated, these covariance conclusions would hold, but independence would not follow in general.

1.4.2 A two-step random walk by enumeration

Let $x[0] = 0$, and let each independent step be $+1$ or -1, with probability $1/2$ each. After two steps the four equally probable step sequences are $(+1, +1), (+1, -1), (-1, +1), (-1, -1)$. Their endpoints are $2, 0, 0, -2$. Thus the endpoint probabilities are $1/4, 1/2, 1/4$ at $-2, 0, 2$ respectively. The mean is zero and the variance is

$$\frac{1}{4}(-2)^2 + \frac{1}{2}(0)^2 + \frac{1}{4}(2)^2 = 2.$$

A zero mean does not mean the robot has stayed still. Across repeated runs, positive and negative errors cancel in the mean while uncertainty grows. After n such steps, variance is n and standard deviation is $\sqrt{n}$. This is the basic reason unbiased odometry can still drift substantially.

1.4.3 Valid covariance functions and white noise

A scalar covariance function is symmetric and POSITIVE SEMIDEFINITE: for every finite collection of times $t_1, \ldots, t_r$ and real numbers $a_1, \ldots, a_r$,

$$\sum_{i,j=1}^{r} a_i K_{xx}(t_i, t_j) a_j \geq 0. \tag{1.5}$$

Conversely, a symmetric function with this property can serve as a covariance function. Positive semidefiniteness constrains the whole matrix of values, rather than requiring every entry to be nonnegative.

Proof. The key idea is that the left side is a variance. Set $z = \sum_i a_i x(t_i)$. Then

$$\operatorname{Var}(z) = \sum_{i,j} a_i \operatorname{Cov}(x(t_i), x(t_j)) a_j \geq 0.$$

For an integrable weighting function, the continuous counterpart is $z = \int a(t)x(t)\,dt$. Whenever these integrals and second moments exist,

$$\operatorname{Var}(z) = \iint a(t) K_{xx}(t, s) a(s)\,dt\,ds \geq 0.$$

This proves the necessary positive-semidefinite condition. □

DISCRETE-TIME WHITE NOISE has uncorrelated samples at distinct times; its means need not be constant. A common zero-mean, constant-variance case has

$$\mathbb{E}[x[n]] = 0, \qquad R_{xx}[n, m] = \sigma^2 \delta[n - m],$$

which is wide-sense stationary. Independent $\mathcal{N}(0, \sigma^2)$ samples give Gaussian white noise. Whiteness alone does not imply independence.

1.4.4 Uncorrelated does not mean independent

The distinction matters when interpreting a covariance matrix or a white-noise assumption. Let X be equally likely to be $-1, 0, 1$, and define $Y = X^2$. The possible pairs are $(-1, 1), (0, 0), (1, 1)$, each with probability $1/3$. Then

$$\mathbb{E}[X] = 0, \qquad \mathbb{E}[Y] = \frac{2}{3}, \qquad \mathbb{E}[XY] = \mathbb{E}[X^3] = 0,$$

so $\mathrm{Cov}(X, Y) = \mathbb{E}[XY] - \mathbb{E}[X]\mathbb{E}[Y] = 0$. Yet Y is completely determined by X. They are not independent: observing $Y = 0$ tells us $X = 0$ with certainty, while $P(X = 0) = 1/3$ before that observation.

Covariance detects linear shared variation. The positive and negative values of X cancel when multiplied by X^2, even though a nonlinear relationship remains. Thus a zero covariance entry alone does not generally permit us to factor a joint distribution into independent parts.

For a jointly Gaussian vector, zero cross-covariance does imply independence between the corresponding groups. Joint Gaussianity is stronger than saying that each coordinate has a Gaussian marginal distribution. The Gaussian filtering chapter states the joint and independence assumptions explicitly, so that dropping cross terms in a covariance calculation does not silently become an unjustified independence assumption.

Source. Matthew R. Walter, TTIC 31170, Lectures 1, 2.

Chapter 2: Graphical Models and Inference

A graph gives a compact map of which unknowns interact. This chapter explains how to read that map and how to use its local structure to avoid repeating enormous probability sums.

2.1 Why graphical models?

A joint probability table grows exponentially with the number of variables. A marginal distribution keeps the variables we want by summing out the rest. Without useful structure, this sum also requires exponentially many assignments.

A probabilistic graphical model combines a graph with local functions to describe a joint distribution. The graph says which variables directly participate in each local relationship; the functions give those relationships numerical values. The structure also implies conditional independence statements, which tell us when some information is unnecessary. This gives three related tasks:

1. **Representation:** express a distribution using local, interpretable components.
2. **Inference:** compute marginals, conditional probabilities, or other quantities of interest.
3. **Learning:** estimate the parameters, and sometimes the graph, from data.

Local structure makes the representation explicit and modular, and can make marginalization tractable.

Robot localization and mapping use a dynamic Bayesian network. In figure 4, X_t is pose, U_t is an action, and Z_t is a measurement. The arrows connect successive poses, actions to the next pose, and poses to their measurements.

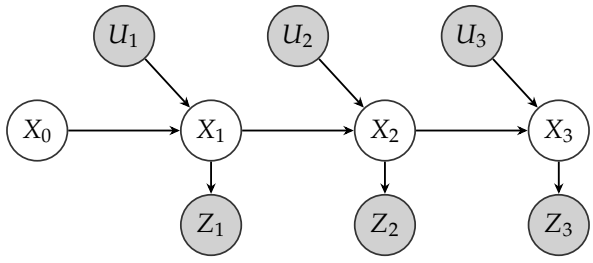


Figure 4: An unrolled localization model. Unshaded nodes are unknown poses; shaded nodes are supplied actions or observed measurements. Action U_t controls the transition from X_{t-1} to X_t.

Other applications have different local structure. Language grounding links phrases to candidate objects and actions through consistency factors. A factorial hidden Markov model uses several latent

chains to explain shared observations. Lattice models describe interactions between neighboring spins or pixels, while coding models connect transmitted bits through local constraints. In each case, the graph records which quantities participate in each local relationship.

2.2 Bayesian networks and directed separation

Definition 2.1 [Bayesian network]. A Bayesian network is a directed acyclic graph (DAG) whose nodes index random variables. Each node v has a normalized conditional distribution given its parents $\mathrm{Pa}(v)$. The joint distribution factorizes as

$$p(x_V) = \prod_{v \in V} p(x_v \mid x_{\mathrm{Pa}(v)}). \tag{2.1}$$

A root node has no parents and contributes $p(x_v)$. A parent is simply a node with an arrow into this node; acyclic means that following arrows cannot bring us back to where we started.

A mixture model is a simple example: a component index I points to an observation X, giving $p(i, x) = p(i)p(x \mid i)$. The graph's assumptions reduce the chain rule's conditioning sets to parent sets. The normalized local conditionals make the product a normalized joint distribution.

Definition 2.2 [Conditional independence]. X and Y are conditionally independent given Z, written $X \perp\!\!\!\perp Y \mid Z$, if

$$p(x, y \mid z) = p(x \mid z)p(y \mid z)$$

for conditioning values with positive probability. Equivalently, learning Y provides no additional information about X once Z is known.

The three patterns in figure 5 explain how conditioning changes information flow. In a chain $X \to Z \to Y$, observing Z blocks the path. In a fork $X \leftarrow Z \to Y$, observing the common parent also blocks the path. A collider $X \to Z \leftarrow Y$ behaves differently: the path is blocked when neither Z nor a descendant of Z is observed. Observing the collider, or one of its descendants, can activate the path.

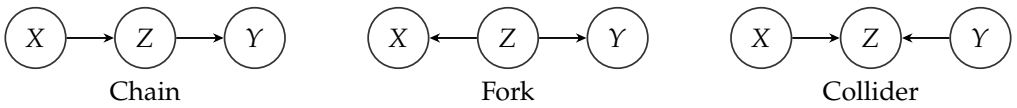


Figure 5: The three local path patterns. Chains and forks are blocked by conditioning on the middle node. A collider is blocked until it or one of its descendants is conditioned on. Arrow directions are essential to this distinction.

For the collider distribution $p(x, y, z) = p(x)p(y)p(z \mid x, y)$, summing over z shows that $p(x, y) = p(x)p(y)$. The root variables are marginally independent. Once z is fixed, however, $p(x, y \mid z)$ is

proportional to $p(x)p(y)p(z \mid x, y)$ and need not factor into separate functions of x and y. Observing a shared effect can therefore connect its possible causes.

2.2.1 Why observing an effect can connect independent causes

A binary alarm is triggered by either an obstacle O or a sensor fault F. For this deliberately simple example, O and F are independent, each has probability $1/10$, and the alarm A is their logical OR. Before the alarm, learning whether the sensor is faulty tells us nothing about the obstacle. The graph is the collider $O \to A \leftarrow F$.

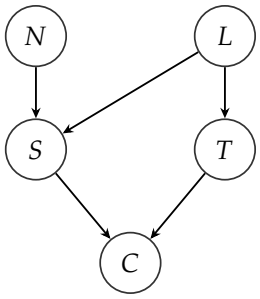
The alarm probability is $1 - (9/10)^2 = 19/100$. An obstacle always triggers it, so $P(O = 1 \mid A = 1) = (1/10)/(19/100) = 10/19$. Now additionally learn that there is no fault. Since the alarm is on, the obstacle must be present: $P(O = 1 \mid A = 1, F = 0) = 1$. If instead a fault is confirmed, the alarm is already explained and $P(O = 1 \mid A = 1, F = 1) = 1/10$. Once the shared effect is observed, information about one cause changes the probability of the other. This is called EXPLAINING AWAY; it does not mean that the causes physically influence each other.

Definition 2.3 [D-SEPARATION**].** Consider a path in a DAG, ignoring arrow direction when traversing it, and a set of conditioned nodes C. The path is blocked if it contains either

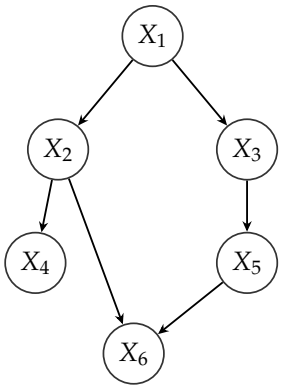
1. a noncollider that belongs to C, or
2. a collider such that neither it nor any of its descendants belongs to C.

Disjoint node sets A and B are D-SEPARATED by C if every path between them is blocked. A distribution that factorizes over the DAG then satisfies $X_A \perp\!\!\!\perp X_B \mid X_C$.

All paths must be blocked to guarantee independence. An active path permits dependence but does not force it. Graphs that imply the same conditional independences are I-EQUIVALENT; chains and forks give an example, since both imply $X \perp\!\!\!\perp Y \mid Z$.



Five-node example



Six-node example

Figure 6: Two Bayesian networks. The five-node graph factors as $p(n)p(l)p(s \mid n, l)p(t \mid l)p(c \mid s, t)$. In the six-node graph, X_6 has parents X_2 and X_5, while every other nonroot node has one parent.

For the six-node graph,

$$p(x_{1:6}) = p(x_1)p(x_2 \mid x_1)p(x_3 \mid x_1)p(x_4 \mid x_2)p(x_5 \mid x_3)p(x_6 \mid x_2, x_5).$$

A full joint table grows exponentially with the number of variables. Bayesian-network tables instead grow exponentially with the number of parents per node. With a fixed bound on parents and a fixed number of states, storage grows linearly with the number of nodes.

2.3 Markov random fields and undirected separation

Definition 2.4 [Markov random field**].** An undirected graphical model, or Markov random field (MRF), represents a distribution through nonnegative potential functions on cliques:

$$p(x_V) = \frac{1}{Z} \prod_{C \in \mathcal{C}} \psi_C(x_C), \qquad Z = \sum_{x_V} \prod_{C \in \mathcal{C}} \psi_C(x_C). \tag{2.2}$$

A clique is a set of mutually adjacent nodes. The partition function Z is the sum of the unnormalized weights. Dividing by it makes probabilities sum to one. It must be finite and positive. Continuous models replace the sum by an integral.

Unlike a Bayesian-network conditional table, an individual potential need not sum to one and need not be a probability distribution. It assigns a compatibility weight to each local configuration: a larger value makes that combination more favored when the other factors are held fixed. Multiplying these weights and normalizing produces the joint distribution.

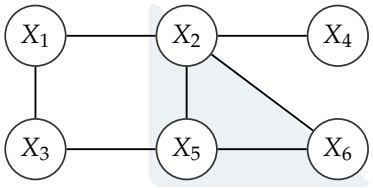


Figure 7: The six-node MRF used below for variable elimination. Four pairwise cliques and the three-node clique $\{2, 5, 6\}$ support the displayed factorization. The shaded triangle highlights the three-variable potential.

For figure 7, use the factorization

$$p(x_{1:6}) = \frac{1}{Z}\psi_{12}(x_1, x_2)\psi_{13}(x_1, x_3)\psi_{24}(x_2, x_4)\psi_{35}(x_3, x_5)\psi_{256}(x_2, x_5, x_6). \tag{2.3}$$

The triangle contributes the single potential ψ_{256}, which depends on all three variables together.

In an undirected graph, separation has its ordinary graph meaning. A set C SEPARATES A and B if every path from A to B passes through C. For a distribution factorized as in equation (2.2), this implies $X_A \perp\!\!\!\perp X_B \mid X_C$.

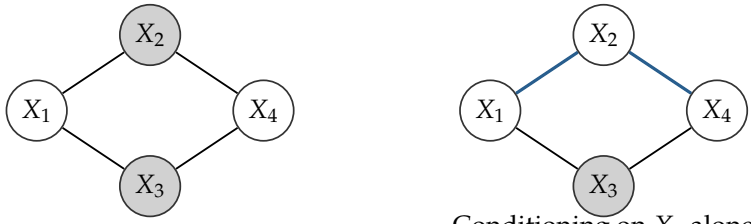


Figure 8: Undirected separation in a four-cycle. Shading denotes conditioning. Both middle nodes must be conditioned on to block both paths between X_1 and X_4.

2.3.1 Examples of potentials

For discrete variables, a potential can be a table of nonnegative weights. A binary four-cycle has the following example potentials:

$$\psi_{AB} = \psi_{DC} = \begin{pmatrix} 10 & 2 \\ 2 & 10 \end{pmatrix}, \qquad \psi_{AC} = \psi_{BD} = \begin{pmatrix} 2 & 10 \\ 10 & 2 \end{pmatrix}.$$

Rows index the first named variable's values $0, 1$, and columns index the second's. The first pair of tables favors agreement; the second favors disagreement. None is individually normalized. Their

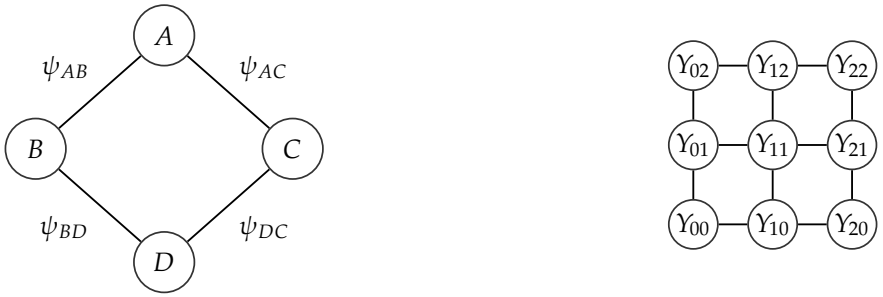
product defines the joint model

$$p(a,b,c,d) \propto \psi_{AB}(a,b)\psi_{AC}(a,c)\psi_{BD}(b,d)\psi_{DC}(d,c).$$

An Ising model uses spins $Y_s \in \{-1,+1\}$ and pair potentials

$$\psi_{st}(y_s,y_t) = \exp(w_{st}y_s y_t).$$

A positive w_{st} favors agreeing spins because agreement contributes $e^{w_{st}}$ rather than $e^{-w_{st}}$. Negative coupling favors disagreement. The factors can be placed on a grid of neighboring sites.



Tabular four-cycle Ising grid

Figure 9: The tabular model multiplies the four edge potentials. The Ising grid uses pairwise potentials between neighboring spins.

A Gaussian Markov random field uses quadratic single-variable and pairwise potentials. For a symmetric positive-definite matrix Λ and a vector η,

$$p(x) \propto \prod_i \exp\left[-\tfrac{1}{2}\left(\Lambda_{ii}x_i^2 - 2\eta_i x_i\right)\right] \prod_{i<j} \exp(-\Lambda_{ij}x_i x_j).$$

Each undirected pair appears once. The single-variable factors supply quadratic and linear terms; the pairwise factors supply cross terms.

2.4 Converting a Bayesian network to an MRF

A Bayesian-network factor $p(x_v \mid x_{\mathrm{Pa}(v)})$ involves the child and all its parents. To make its scope a clique in an undirected graph, connect every pair of parents of a common child, then remove all arrowheads. This operation is moralization. The parents occur together in the child's conditional table, so they must form a clique with the child. The resulting graph supports the same joint distribution.

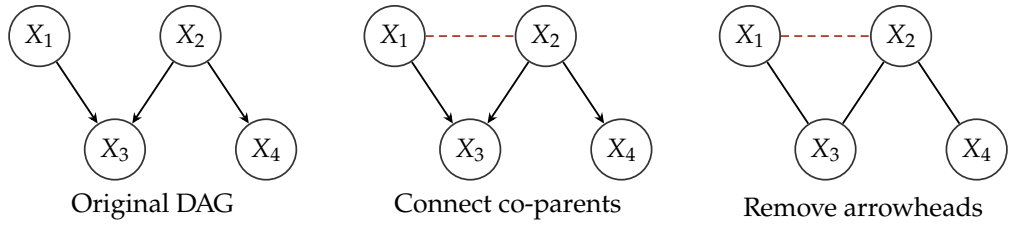


Figure 10: Moralization adds the dashed edge between X_1 and X_2, the parents of X_3, and removes arrowheads. The child and its parents then form the clique required by the conditional factor.

For the example in figure 10,

$$p(x_{1:4}) = p(x_1)p(x_2)p(x_3 \mid x_1, x_2)p(x_4 \mid x_2).$$

Choose $\psi_{123} = p(x_1)p(x_2)p(x_3 \mid x_1, x_2)$ and $\psi_{24} = p(x_4 \mid x_2)$. Their product is the original normalized joint, so $Z = 1$.

2.5 Inference with evidence: from factors to messages

Consider three independent binary events D_1, D_2, D_3 and two observed findings F_1, F_2. The first finding depends on D_1, D_2; the second depends on D_2, D_3. Each finding is a common effect, so observing it can connect its possible causes. Its joint distribution is

$$p(d_1, d_2, d_3, f_1, f_2) = p(d_1)p(d_2)p(d_3)p(f_1 \mid d_1, d_2)p(f_2 \mid d_2, d_3). \tag{2.4}$$

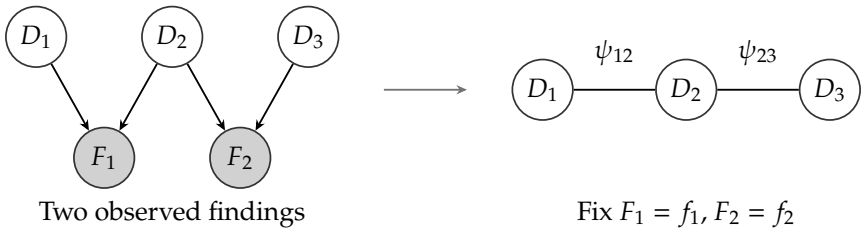


Figure 11: Conditioning on the two findings leaves a chain of event variables. The finding values are absorbed into the pairwise potentials rather than retained as unknown nodes.

Write the fixed evidence as $e = (f_1, f_2)$ and define

$$\begin{aligned}\psi_{12}(d_1, d_2) &= p(d_1)p(d_2)p(f_1 \mid d_1, d_2),\\ \psi_{23}(d_2, d_3) &= p(d_3)p(f_2 \mid d_2, d_3).\end{aligned} \tag{2.5}$$

Then $p(d_1, d_2, d_3, e) = \psi_{12}\psi_{23}$ and

$$p(d_1, d_2, d_3 \mid e) = \frac{\psi_{12}(d_1, d_2)\psi_{23}(d_2, d_3)}{Z_e}, \quad Z_e = \sum_{d_1,d_2,d_3} \psi_{12}\psi_{23} = p(e).$$

The prior $p(d_2)$ appears only once, so it is not double-counted. Conditioning requires $Z_e > 0$.

2.5.1 Computing one marginal

To find $p(d_3 \mid e)$, first compute the unnormalized marginal $p(d_3, e)$:

$$\begin{aligned} p(d_3, e) &= \sum_{d_1,d_2} \psi_{12}(d_1, d_2)\psi_{23}(d_2, d_3) \\ &= \sum_{d_2} \psi_{23}(d_2, d_3) \underbrace{\sum_{d_1} \psi_{12}(d_1, d_2)}_{m_{1\to 2}(d_2)}. \end{aligned} \tag{2.6}$$

The inner sum depends on d_2 but not on d_3. Compute it once and reuse it. Define

$$m_{1\to 2}(d_2) = \sum_{d_1} \psi_{12}(d_1, d_2), \qquad m_{2\to 3}(d_3) = \sum_{d_2} m_{1\to 2}(d_2)\psi_{23}(d_2, d_3).$$

These functions are MESSAGES: each summarizes the contribution of the variables already summed out. The desired posterior is

$$p(d_3 \mid e) = \frac{m_{2\to 3}(d_3)}{\sum_{d_3'} m_{2\to 3}(d_3')}.$$

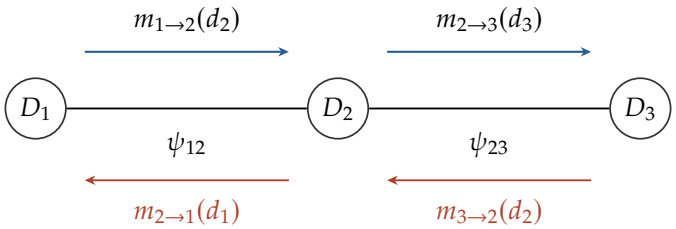


Figure 12: Forward messages in blue and reverse messages in red. Each arrow is labeled by the variable that survives its summation. The two passes provide all three node marginals without repeating the full joint sum.

2.5.2 A message-passing calculation with small tables

Use binary variables D_1, D_2, D_3, and suppose the two pairwise factors are

$$\psi_{12} = \begin{bmatrix} 3 & 1 \\ 1 & 3 \end{bmatrix}, \qquad \psi_{23} = \begin{bmatrix} 2 & 1 \\ 1 & 4 \end{bmatrix}.$$

Rows correspond to the first variable named and columns to the second, both in order $0, 1$. These are weights, not normalized probability tables. For example, the assignment $(0, 0, 1)$ has weight $3 \times 1 = 3$.

1. **Remove D_1.** Add each column of ψ_{12}: $m_{1\to2}(0) = 3 + 1 = 4$ and $m_{1\to2}(1) = 1 + 3 = 4$.
2. **Remove D_2.** For candidate $D_3 = 0$, use the first column of ψ_{23}: $m_{2\to3}(0) = 4(2) + 4(1) = 12$. For $D_3 = 1$, the second column gives $m_{2\to3}(1) = 4(1) + 4(4) = 20$.
3. **Normalize once.** The total weight is $Z = 12 + 20 = 32$, so $P(D_3 = 0) = 3/8$ and $P(D_3 = 1) = 5/8$.

A message is a small lookup table that preserves everything the eliminated part can contribute to the remaining variables. It is like replacing a long receipt by subtotals for the categories still needed. A single grand total would lose the distinction between the two possible values of D_2.

For the reverse pass, summing the rows of ψ_{23} gives $(3, 5)$. Hence $P(D_1 = 0)$ has unnormalized weight $3(3) + 1(5) = 14$, while $P(D_1 = 1)$ has weight $1(3) + 3(5) = 18$. Normalize by the same 32 to obtain $(7/16, 9/16)$. The middle marginal has weights $(4 \times 3, 4 \times 5) = (12, 20)$. These three calculations agree with enumerating all eight assignments, but reuse intermediate sums rather than starting over for each question.

2.5.3 Reusing work for all marginals

Compute the reverse messages

$$m_{3\to2}(d_2) = \sum_{d_3} \psi_{23}(d_2, d_3), \qquad m_{2\to1}(d_1) = \sum_{d_2} \psi_{12}(d_1, d_2) m_{3\to2}(d_2).$$

Then the unnormalized node marginals are

$$\begin{aligned} p(d_1, e) &= m_{2\to1}(d_1), \\ p(d_2, e) &= m_{1\to2}(d_2) m_{3\to2}(d_2), \\ p(d_3, e) &= m_{2\to3}(d_3). \end{aligned}$$

Each sums to the same $Z_e = p(e)$, so normalizing any of these vectors gives its posterior marginal.

2.6 Variable elimination in a graph with cycles

VARIABLE ELIMINATION multiplies the factors containing the next variable, sums that variable out, and replaces those factors by the resulting function. Other factors remain outside the sum. Distributivity makes this rearrangement exact.

Return to the six-node MRF in equation (2.3). To obtain $p(x_1)$, eliminate X_6, X_5, X_4, X_3, X_2 in that order. Suppress function arguments only when they are apparent from the subscripts.

A message subscript identifies the eliminated variable; its arguments index the remaining variables. Thus $m_6(x_2, x_5)$ is a table, not a scalar.

Algorithm 2.1: Eliminate variables in the six-node MRF

Input: Potentials $\psi_{12}, \psi_{13}, \psi_{24}, \psi_{35}, \psi_{256}$ in equation (2.3)
Output: Marginal $p(x_1)$

$m_6(x_2, x_5) \leftarrow \sum_{x_6} \psi_{256}(x_2, x_5, x_6)$

$m_5(x_2, x_3) \leftarrow \sum_{x_5} \psi_{35}(x_3, x_5) m_6(x_2, x_5)$

$m_4(x_2) \leftarrow \sum_{x_4} \psi_{24}(x_2, x_4)$

$m_3(x_1, x_2) \leftarrow \sum_{x_3} \psi_{13}(x_1, x_3) m_5(x_2, x_3)$

$m_2(x_1) \leftarrow \sum_{x_2} \psi_{12}(x_1, x_2) m_4(x_2) m_3(x_1, x_2)$

$Z \leftarrow \sum_{x_1} m_2(x_1)$

return $p(x_1) = m_2(x_1)/Z$

Each assignment computes a message for every combination of its remaining arguments. Only factors involving the variable being eliminated go inside that sum; factors independent of it can be used in a later step. Only ψ_{256} contains x_6, so the first step replaces it by m_6. The next step combines m_6 with ψ_{35} and removes x_5. Its result $m_5(x_2, x_3)$ couples X_2 and X_3, although the original graph had no edge between them. The remaining rows apply the same rule.

m_2 is the unnormalized marginal; its sum is Z. The nested sums avoid enumerating the full joint table.

2.6.1 Fill-in edges and the induced graph

Eliminating a node generally produces a factor on its remaining neighbors. Make those neighbors into a clique before removing the node. Any newly introduced edge is a FILL-IN EDGE. The graph formed by the original edges and all fill-in edges is the INDUCED GRAPH for that elimination order.

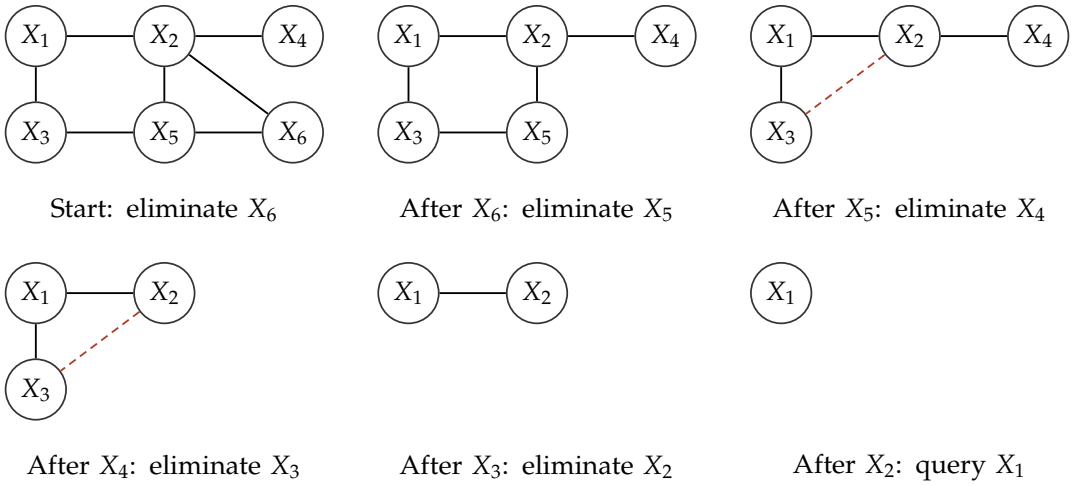


Figure 13: Eliminate $X_6, \ldots, X_2$, keeping nodes at fixed positions. Eliminating X_6 adds no edge because X_2 and X_5 are adjacent. Eliminating X_5 adds the dashed edge X_2–X_3; later steps add none.

2.6.2 Complexity depends on the elimination order

For r states per variable, direct summation over the six-variable joint costs $O(r^6)$. The displayed order uses products involving at most three variables, giving $O(r^3)$ cost for this fixed graph.

Definition 2.5 [Elimination width and treewidth]. An elimination order's width is one less than the largest elimination clique size, counting the variable being removed. The treewidth is the minimum of this width over all elimination orders. A largest clique of size three therefore gives width two and tables whose work can scale as $O(r^{2+1})$.

The ordering goal is to minimize the largest intermediate clique, because summation cost grows exponentially with its size. Finding an order that achieves treewidth is NP-hard. The ordering problem also has parallels with solving linear equations.

Source. Matthew R. Walter, TTIC 31170, Lecture 1.

Part II

Estimating a Hidden State

Chapter 3: Hidden States and Sequential Inference

When observations arrive over time, the hidden state links them into a sequence. We first infer states under a known model, then use uncertainty about those states to learn the model itself.

3.1 The Markov assumption

Without a simplifying assumption, the chain rule gives

$$P(x_{0:T}) = P(x_0) \prod_{t=1}^{T} P(x_t \mid x_{0:t-1}).$$

Each conditional can depend on the entire past.

Definition 3.1 [First-order Markov property]. A process is first-order Markov if, given its present state, its next state is conditionally independent of earlier states:

$$P(X_{t+1} \mid X_t, X_{t-1}, \ldots, X_0) = P(X_{t+1} \mid X_t).$$

Consequently,

$$P(x_{0:T}) = P(x_0) \prod_{t=1}^{T} P(x_t \mid x_{t-1}). \tag{3.1}$$

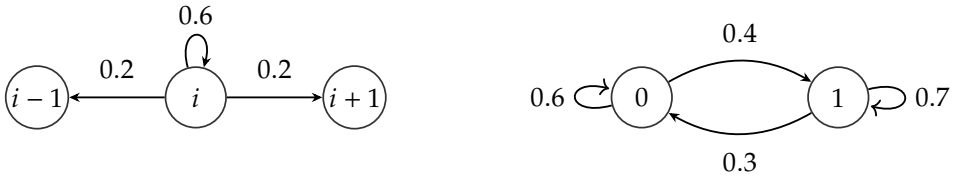
The initial distribution is a separate factor; there is no transition from an unspecified X_{-1}.

For the one-dimensional robot, the transition model at an interior cell is

$$P(X_t = j \mid X_{t-1} = i) = \begin{cases} 0.2, & j = i + 1, \\ 0.6, & j = i, \\ 0.2, & j = i - 1, \\ 0, & \text{otherwise.} \end{cases}$$

The boundary cells require their own valid transition probabilities. For the binary sequence, the transition diagram specifies

$$a_{00} = 0.6, \quad a_{10} = 0.4, \quad a_{01} = 0.3, \quad a_{11} = 0.7, \qquad a_{ji} = P(X_t = j \mid X_{t-1} = i).$$



One step from an interior cell Binary-state Markov chain

Figure 14: The two transition models. Outgoing probabilities from each state sum to one; they describe state evolution rather than sensor errors.

3.2 Hidden Markov models and forward–backward inference

The state often cannot be observed directly. A sensor instead generates a noisy observation, such as $Z_t = X_t + \text{noise}$.

Definition 3.2 [Hidden Markov model]. A hidden Markov model (HMM) has a Markov state sequence X_t and observations Z_t that are conditionally independent of other states and observations given X_t. Its joint distribution is

$$P(x_{0:T}, z_{0:T}) = P(x_0)P(z_0 \mid x_0) \prod_{t=1}^{T} P(x_t \mid x_{t-1})P(z_t \mid x_t). \tag{3.2}$$

The two local models are the transition and emission distributions.

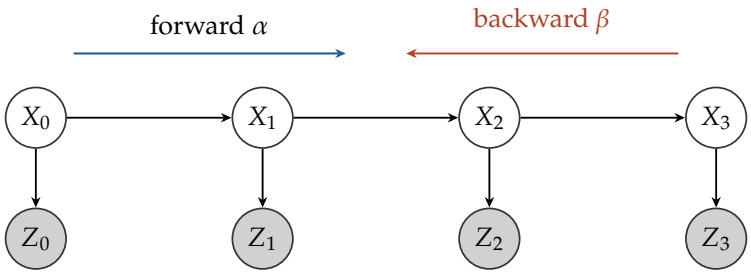


Figure 15: An HMM with observed nodes shaded. Each state emits its observation; forward and backward messages summarize the two sides of the chain.

Fix the observed sequence $z_{0:T}$. To infer one state, use

$$P(X_t = i \mid z_{0:T}) = \frac{P(X_t = i, z_{0:T})}{P(z_{0:T})}.$$

Given X_t, future observations are independent of the earlier observations. This splits the numerator into two reusable parts:

$$\begin{aligned} \alpha_t(i) &= P(z_{0:t}, X_t = i), \\ \beta_t(i) &= P(z_{t+1:T} \mid X_t = i), \\ P(X_t = i, z_{0:T}) &= \alpha_t(i)\beta_t(i). \end{aligned} \tag{3.3}$$

These are the FORWARD and BACKWARD PROBABILITIES. The forward message includes the observation at time t; the backward message starts at $t + 1$, so that observation is not counted twice.

3.2.1 Forward recursion

Sum over the previous state and apply the HMM independences:

$$\begin{aligned} \alpha_t(i) &= \sum_j P(z_{0:t}, X_{t-1} = j, X_t = i) \\ &= P(z_t \mid X_t = i) \sum_j P(X_t = i \mid X_{t-1} = j) P(z_{0:t-1}, X_{t-1} = j) \\ &= b_i(z_t) \sum_j a_{ij} \alpha_{t-1}(j), \end{aligned}$$

where $b_i(z) = P(Z_t = z \mid X_t = i)$ and $a_{ij} = P(X_t = i \mid X_{t-1} = j)$. Start with

$$\alpha_0(i) = b_i(z_0)\pi_i, \qquad \pi_i = P(X_0 = i).$$

Each update first collects probability arriving from all previous states, then multiplies by how well the new state explains the observation.

For the binary-state example, the first update is explicitly

$$\begin{aligned} \alpha_1(0) &= b_0(z_1)[a_{00}\alpha_0(0) + a_{01}\alpha_0(1)], \\ \alpha_1(1) &= b_1(z_1)[a_{10}\alpha_0(0) + a_{11}\alpha_0(1)], \end{aligned}$$

with $\alpha_0(0) = b_0(z_0)\pi_0$ and $\alpha_0(1) = b_1(z_0)\pi_1$. Both possible predecessors contribute to each sum.

3.2.2 Two positions and two sensor reports

Consider a robot in positions L and R. Initially each has probability $1/2$. Between readings it stays in the same position with probability $4/5$ and switches with probability $1/5$. The sensor reports a door, denoted D, with probability $9/10$ at L and $1/5$ at R. Its other report N has probabilities $1/10$ and $4/5$. Observe D at time 0, then N at time 1.

The first joint forward weights are $\alpha_0(L) = (1/2)(9/10) = 9/20$ and $\alpha_0(R) = (1/2)(1/5) = 1/10$. Their sum is $11/20$, so the filtered probability of L at time 0 is $9/11$. For the next step, first add the possible predecessor contributions, then multiply by the new likelihood:

$$\alpha_1(L) = \frac{1}{10}\left[\frac{4}{5}\frac{9}{20} + \frac{1}{5}\frac{1}{10}\right] = \frac{19}{500},$$
$$\alpha_1(R) = \frac{4}{5}\left[\frac{1}{5}\frac{9}{20} + \frac{4}{5}\frac{1}{10}\right] = \frac{68}{500}.$$

The observed sequence has probability $87/500$. The current filtered belief is therefore $(19/87, 68/87)$. The second reading makes the right position more plausible, but it does not erase the contribution of the first reading.

3.2.3 Backward recursion and normalization

For the future observations, sum over the next state:

$$\begin{aligned}\beta_t(i) &= \sum_j P(z_{t+1:T}, X_{t+1} = j \mid X_t = i) \\ &= \sum_j P(X_{t+1} = j \mid X_t = i) P(z_{t+1} \mid X_{t+1} = j) P(z_{t+2:T} \mid X_{t+1} = j) \\ &= \sum_j a_{ji} b_j(z_{t+1}) \beta_{t+1}(j).\end{aligned}$$

Initialize $\beta_T(i) = 1$: there are no remaining observations after T. Work backward from $T - 1$ to 0.

The observation-sequence likelihood is the same at every cut of the chain:

$$L = P(z_{0:T}) = \sum_i \alpha_t(i)\beta_t(i) = \sum_i \alpha_T(i). \tag{3.4}$$

The first equality follows by summing $P(X_t = i, z_{0:T})$ over i; the final one uses $\beta_T(i) = 1$. When $L > 0$,

$$P(X_t = i \mid z_{0:T}) = \frac{\alpha_t(i)\beta_t(i)}{L}. \tag{3.5}$$

3.2.4 Using the second report to reconsider the first position

In the two-position example, the backward likelihood at time 0 is the probability of the future report N, conditional on the starting position:

$$\beta_0(L) = \frac{4}{5}\frac{1}{10} + \frac{1}{5}\frac{4}{5} = \frac{6}{25},$$
$$\beta_0(R) = \frac{1}{5}\frac{1}{10} + \frac{4}{5}\frac{4}{5} = \frac{33}{50}.$$

Multiply each by its forward weight. This gives $\alpha_0(L)\beta_0(L) = 54/500$ and $\alpha_0(R)\beta_0(R) = 33/500$. Their sum is again $87/500$. The smoothed time-0 belief is $(18/29, 11/29)$, rather than the filtered $(9/11, 2/11)$. A later report has changed our opinion about an earlier state. It has not changed that earlier physical state.

Filtering is like giving a location estimate during a journey; smoothing is like revising the journey afterward using the complete travel log. A robot that must act immediately needs filtering. A map-building system can also use later evidence to improve past poses.

For long sequences, raw forward probabilities can underflow to zero in a computer even when the mathematical likelihood is positive. Scale each forward vector by its sum, record those scale factors, and use matching scaling for the backward pass. The log sequence likelihood is the sum of the logs of the forward scale factors. Consistent scaling preserves the posterior; dividing raw and scaled messages indiscriminately does not.

Algorithm 3.1: Forward–backward inference

Input: Observations $z_{0:T}$, initial probabilities π_i, transitions a_{ji}, emissions b_i
Output: All state posteriors $\gamma_t(i)$ and likelihood L

for each state i **do**
  $\alpha_0(i) \leftarrow \pi_i b_i(z_0)$
for $t \leftarrow 1$ **to** T **do**
  for each state i **do**
    $\alpha_t(i) \leftarrow b_i(z_t) \sum_j a_{ij}\alpha_{t-1}(j)$
$L \leftarrow \sum_i \alpha_T(i)$
for each state i **do**
  $\beta_T(i) \leftarrow 1$
for $t \leftarrow T-1$ **down to** 0 **do**
  for each state i **do**
    $\beta_t(i) \leftarrow \sum_j a_{ji} b_j(z_{t+1})\beta_{t+1}(j)$
for $t \leftarrow 0$ **to** T **do**
  for each state i **do**
    $\gamma_t(i) \leftarrow \alpha_t(i)\beta_t(i)/L$

For K states and dense transitions, each time step costs $O(K^2)$; both passes together take $O((T+1)K^2)$ time. One running message vector needs $O(K)$ working memory, independent of sequence length. Computing all smoothed marginals with stored forward messages needs $O((T+1)K)$ storage; returning all of them also requires that much output space.

3.3 MAP sequences and the Viterbi algorithm

The most likely state at each separate time is

$$\hat{x}_t = \arg\max_i P(X_t = i \mid z_{0:T}).$$

Putting these marginal choices together need not give the most likely whole trajectory. A state marginal adds up the probabilities of many trajectories; the joint maximum chooses one compatible trajectory:

$$x^*_{0:T} = \arg\max_{x_{0:T}} P(x_{0:T} \mid z_{0:T}) = \arg\max_{x_{0:T}} P(x_{0:T}, z_{0:T}).$$

The denominator $P(z_{0:T})$ is constant with respect to the trajectory. Enumerating all K^{T+1} trajectories is unnecessary.

3.3.1 A joint best path need not use every marginal best state

Suppose a two-time posterior assigns probabilities

	$X_1 = L$	$X_1 = R$
$X_0 = L$	0.35	0.25
$X_0 = R$	0.01	0.39

The first-time marginal prefers L, since $0.35 + 0.25 = 0.60$. The second-time marginal prefers R, since $0.25 + 0.39 = 0.64$. Choosing the marginal winner at each time gives (L, R), whose joint probability is 0.25. The most probable whole path is (R, R), with probability 0.39. Marginal inference pools support from different paths; path inference compares paths individually.

For the earlier door/no-door HMM, the four path-and-observation probabilities are

path	LL	LR	RL	RR
joint weight	18/500	36/500	1/500	32/500

respectively. Viterbi must select LR. This small enumeration is a useful independent check of the max-product recursion.

Definition 3.3 [Viterbi score]. Let $\delta_t(i)$ be the probability of the best path ending in state i at time t, together with the observations so far:

$$\delta_t(i) = \max_{x_{0:t-1}} P(x_{0:t-1}, X_t = i, z_{0:t}).$$

The Markov property makes the last state a sufficient summary of a path for extending it. For a fixed predecessor j, only its best prefix can improve the best path to i: all such prefixes receive the

same transition and emission multiplier. Thus

$$\delta_0(i) = \pi_i b_i(z_0), \qquad \delta_t(i) = b_i(z_t) \max_j \{a_{ij}\delta_{t-1}(j)\}. \tag{3.6}$$

Compared with the forward recursion, the sum becomes a maximum. Record the maximizing predecessor as well:

$$\mathrm{Pre}_t(i) = \arg\max_j \{a_{ij}\delta_{t-1}(j)\}.$$

Once the best terminal state is known, these pointers recover the whole path.

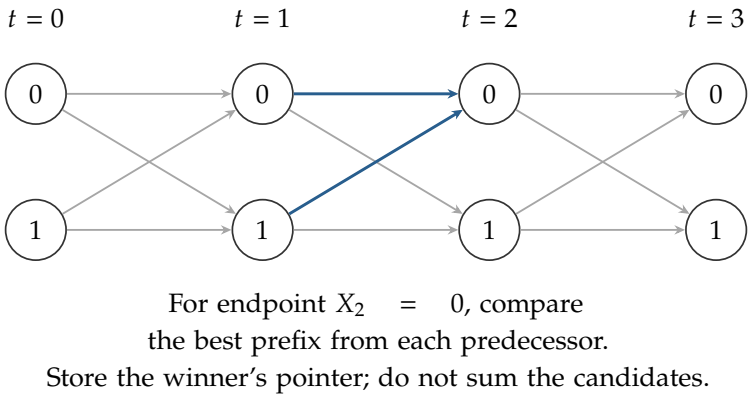


Figure 16: The binary-state trellis. Each column represents one state variable. Blue edges show the candidates for one endpoint; the diagram supplies no numerical emission probabilities and does not select a winner.

Algorithm 3.2: Viterbi decoding

Input: Observations $z_{0:T}$, probabilities π_i, a_{ji}, b_i
Output: A most likely state sequence $x^*_{0:T}$

for each state i **do**
  $\delta_0(i) \leftarrow \pi_i b_i(z_0)$
  $\mathrm{Pre}_0(i) \leftarrow \varnothing$
for $t \leftarrow 1$ **to** T **do**
  for each state i **do**
    $\mathrm{Pre}_t(i) \leftarrow \arg\max_j \{a_{ij}\delta_{t-1}(j)\}$
    $\delta_t(i) \leftarrow b_i(z_t) \max_j \{a_{ij}\delta_{t-1}(j)\}$
$x^*_T \leftarrow \arg\max_i \delta_T(i)$
for $t \leftarrow T-1$ **down to** 0 **do**
  $x^*_t \leftarrow \mathrm{Pre}_{t+1}(x^*_{t+1})$
return $x^*_{0:T}$

Use a consistent choice when maxima tie. As with forward–backward inference, the running time is $O((T+1)K^2)$ for dense transitions. Scores need $O(K)$ working memory, but the backtracking pointers need $O((T+1)K)$ storage. These algorithms assume the observed sequence has positive probability.

3.4 Filtering, smoothing, and robot localization

Filtering uses observations only through the time being estimated:

$$P(X_t = i \mid z_{0:t}) = \frac{\alpha_t(i)}{\sum_j \alpha_t(j)}. \tag{3.7}$$

Smoothing also uses later observations. For a fixed history through $T \geq t$,

$$P(X_t = i \mid z_{0:T}) = \frac{\alpha_t(i)\beta_t(i)}{\sum_j \alpha_t(j)\beta_t(j)} = \frac{\alpha_t(i)\beta_t(i)}{\sum_j \alpha_T(j)}. \tag{3.8}$$

The denominators are observation likelihoods, assumed positive. At $t = T$, $\beta_T(i) = 1$, so filtering and smoothing agree. For $t < T$, future observations can change the belief about the state at time t.

Example: A robot moves in a grid using up, down, left, or right actions. Its heading is known. With small probability, the requested motion does not occur. Four sensors report whether there is a wall in front, behind, to the left, and to the right; the illustrated sensor model allows at most one incorrect report.

Initially many cells are plausible. After each requested move, prediction transfers probability to possible destinations, including the possibility of staying put. A sensor update favors cells whose surrounding walls agree with the reports. In the illustrated sequence, several locations remain plausible through the early moves; by time 5, the belief concentrates near the upper junction. A motion or sensor error is a possibility within the model, rather than a reason to discard all earlier information.



Initial ambiguity After repeated updates

Figure 17: The grid geometry with qualitative belief shading. White interior rectangles are obstacles. Darker traversable cells indicate greater belief; the red dots show the illustrated true robot locations. Shading summarizes the initial and final situations and is not a numerical probability reconstruction.

3.5 Learning an HMM from observations

Inference assumes the transition and emission models are known. Learning instead uses observed sequences to estimate these models while the states remain hidden.

Definition 3.4 [Finite HMM parameters]. For finite state space $\mathcal{X}$ and observation alphabet $\mathcal{Z}$, write $\lambda = \{\mathsf{T}, \mathsf{M}, \pi\}$, where

$$\begin{aligned} \mathsf{T}_{ji} &= P(X_{t+1} = j \mid X_t = i; \lambda), \\ \mathsf{M}_{mi} &= P(Z_t = m \mid X_t = i; \lambda), \\ \pi_i &= P(X_0 = i; \lambda). \end{aligned}$$

All entries are nonnegative. Each column of T and M sums to one, and $\sum_i \pi_i = 1$. The source state is the column index of T. These entries agree with the notation a_{ji} and $b_i(m)$.

For observations $z_{0:T}$, maximum-likelihood learning seeks

$$\lambda^* = \arg\max_{\lambda} P(z_{0:T}; \lambda), \qquad P(z_{0:T}; \lambda) = \sum_{x_{0:T}} P(z_{0:T} \mid x_{0:T}; \lambda) P(x_{0:T}; \lambda). \tag{3.9}$$

There are exponentially many state sequences in this written sum. The complete-data joint factorizes locally, but summing out the shared hidden states couples the observations, so one cannot fit an independent model at each time. Forward inference evaluates the likelihood efficiently for a fixed model; it does not solve the global parameter-optimization problem.

The Baum–Welch algorithm uses expectation–maximization (EM): infer expected hidden-state counts under the current model, then fit a new model to those counts. A step seeks λ' with

$$P(z_{0:T}; \lambda') \geq P(z_{0:T}; \lambda).$$

Repeated updates seek a local solution rather than guaranteeing the global maximum. Effective learning needs enough data, often many long observation histories.

3.5.1 Expected state and transition counts

Under the current parameters, define the smoothed state probabilities

$$\gamma_t(i) = P(X_t = i \mid z_{0:T}; \lambda) = \frac{\alpha_t(i)\beta_t(i)}{L}, \qquad L = P(z_{0:T}; \lambda). \tag{3.10}$$

Let $\mathbf{1}\{X_t = i\}$ be one if $X_t = i$ and zero otherwise. Its conditional expectation is $\gamma_t(i)$. Adding indicators and using linearity of expectation gives

$$\mathbb{E}[\text{number of visits to } i \mid z_{0:T}] = \sum_{t=0}^{T} \gamma_t(i),$$
$$\mathbb{E}[\text{number of departures from } i \mid z_{0:T}] = \sum_{t=0}^{T-1} \gamma_t(i). \tag{3.11}$$

The last state contributes a visit, but has no outgoing transition in this observed sequence. This explains the different upper limits.

For adjacent states, define

$$\xi_t(i, j) = P(X_t = i, X_{t+1} = j \mid z_{0:T}; \lambda), \qquad 0 \le t < T. \tag{3.12}$$

Its numerator separates into a prefix, one transition, the next emission, and a suffix:

$$\xi_t(i, j) = \frac{\alpha_t(i)\mathsf{T}_{ji}\mathsf{M}_{z_{t+1},j}\beta_{t+1}(j)}{L}. \tag{3.13}$$

Summing the numerator over i, j gives L, so this is also normalized by its own double sum. In particular,

$$\sum_j \xi_t(i, j) = \gamma_t(i), \qquad \sum_i \xi_t(i, j) = \gamma_{t+1}(j).$$

Thus the expected number of transitions from i to j is $\sum_{t=0}^{T-1} \xi_t(i, j)$.

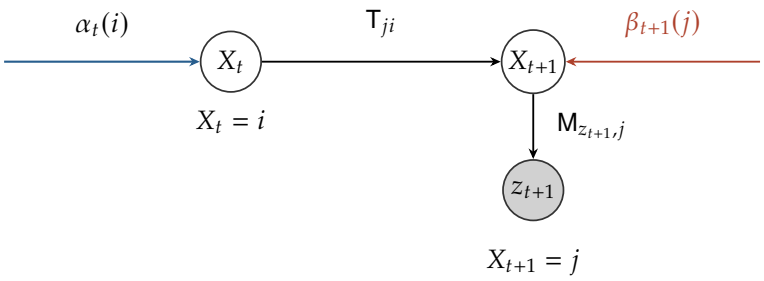


Figure 18: The four factors in the adjacent-state posterior. The next observation belongs to the emission term, and the backward message starts after it.

3.5.2 The Baum–Welch update

Fit the initial distribution, transition probabilities, and emission probabilities using the expected counts:

$$\begin{aligned}\pi'_i &= \gamma_0(i),\\ \mathsf{T}'_{ji} &= \frac{\sum_{t=0}^{T-1} \xi_t(i,j)}{\sum_{t=0}^{T-1} \gamma_t(i)},\\ \mathsf{M}'_{mi} &= \frac{\sum_{t=0}^{T} \gamma_t(i)\mathbf{1}\{z_t = m\}}{\sum_{t=0}^{T} \gamma_t(i)}.\end{aligned} \tag{3.14}$$

The transition ratio is "expected i-to-j transitions divided by expected departures from i." The emission ratio is "expected visits to i when symbol m was observed divided by all expected visits to i." Summing each ratio over its destination or symbol gives one, using the count identities above. A zero denominator means this sequence supplies no expected count for that column; retaining the current valid column avoids an undefined update.

3.5.3 One learning update without hiding the counts

Reuse the two-position example and its observations D, N. Dividing the four path weights by $87/500$ gives the adjacent-state posterior

$$\xi_0 = \frac{1}{87}\begin{bmatrix}18 & 36\\ 1 & 32\end{bmatrix},$$

where rows are the old state L, R and columns are the new state L, R. Its row sums are $(54/87, 33/87)$ and its column sums are $(19/87, 68/87)$. Those are exactly the two smoothed state distributions.

The new probability of staying at L is expected $L \to L$ transitions divided by expected departures from L: $(18/87)/(54/87) = 1/3$. The new probability of $L \to R$ is $2/3$. Departures from R give $P'(L \mid R) = 1/33$ and $P'(R \mid R) = 32/33$. In destination-row, source-column notation,

$$\mathsf{T}' = \begin{bmatrix}1/3 & 1/33\\ 2/3 & 32/33\end{bmatrix}.$$

For the door emission at L, expected door-associated visits are $54/87$ and all expected visits are $(54 + 19)/87$. Thus $P'(D \mid L) = 54/73$. Similarly $P'(D \mid R) = 33/101$, with complementary probabilities for N. Finally, $\pi' = (18/29, 11/29)$.

These are fractional counts because the states are uncertain. We do not pick one guessed path and pretend its states were observed. All expected counts come from the old model, and only then do

we replace its parameters. Two observations are far too little data for a trustworthy robot model; this example illustrates one update, not a claim of reliable parameter recovery.

Algorithm 3.3: One Baum–Welch update for a single sequence

Input: Current model $\lambda = \{\mathsf{T}, \mathsf{M}, \pi\}$ and observations $z_{0:T}$
Output: Updated model λ'

Run forward–backward inference under λ to obtain α, β, L
Compute $\gamma_t(i)$ and $\xi_t(i, j)$ using equations (3.10) and (3.13)
for each state i **do**
    $\pi'_i \leftarrow \gamma_0(i)$
    if $\sum_{t=0}^{T-1} \gamma_t(i) > 0$ **then**
        for each state j **do**
            Update T'_{ji} using equation (3.14)
    else
        Keep $\mathsf{T}'_{\cdot i} \leftarrow \mathsf{T}_{\cdot i}$
    if $\sum_{t=0}^{T} \gamma_t(i) > 0$ **then**
        for each symbol m **do**
            Update M'_{mi} using equation (3.14)
    else
        Keep $\mathsf{M}'_{\cdot i} \leftarrow \mathsf{M}_{\cdot i}$
return $\lambda' = \{\mathsf{T}', \mathsf{M}', \pi'\}$

With exact inference and maximization, EM does not decrease the observation likelihood. At a regular interior fixed point, the likelihood is stationary; this does not establish a global maximum. The posterior counts are computed using the old model throughout one update, then the new model becomes the current model for the next iteration.

Source. Matthew R. Walter, TTIC 31170, Lectures 2, 3.

Chapter 4: The Bayes Filter and Robot Localization

The finite-state calculations suggest a more general pattern: predict where the system could have moved, then reweight those possibilities using the new measurement. This chapter derives that pattern for controlled robot motion.

4.1 What belongs in the robot's state?

A state is a parameterized description of properties of the world that matter for its future. It is complete if it retains the past information needed for the transition and measurement laws. Bits can be states in communication, words in language models, and robot pose together with relevant environment properties in robotics.

4.1.1 Pose and environment representations

A ground robot's pose is typically (x, y, θ): two Cartesian coordinates and a heading. An aerial or underwater robot typically uses three position coordinates plus roll, pitch, and yaw. A manipulator configuration specifies each joint's angle for a revolute joint or distance for a prismatic joint. Body-frame linear and angular velocities may also belong in the state.

The environment description includes object locations and properties. Walls and trees may be static; people and cars may move. Properties can include appearance, color, or whether a door is open. A map can use continuous parameters, such as wall line segments or landmark poses, or discrete cells, such as an occupancy grid with a probability of occupancy for each cell.

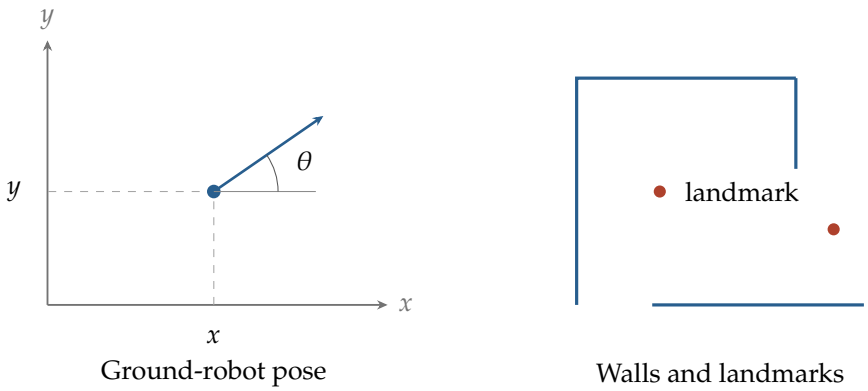


Figure 19: Schematic state representations. Pose gives position and orientation; a feature map gives parameters of surrounding objects. These drawings illustrate representations rather than a particular measured map.

4.1.2 Controls and measurements

A robot interacts with the world in two ways. Control actions change the world state, for example by moving the robot. Sensor measurements provide information about it, using a camera, lidar, or tactile sensor. Control data include velocity and odometry; environment measurements include camera images and range data. Bearing-based measurements can report the direction of identifiable landmarks, such as visual fiducials.

Use lowercase notation for the continuous-state setting and write histories as

$$x^t = (x_0, \dots, x_t), \quad u^t = (u_0, \dots, u_t), \quad z^t = (z_0, \dots, z_t).$$

The input u_t describes the control during $(t-1, t]$, while z_t is the measurement at time t.

4.1.3 A state must remember what prediction needs

Two robots can be at the same position while moving in opposite directions. If position alone is the state, a transition model cannot distinguish them. Including velocity lets a constant-velocity model predict different next positions. Likewise, an unknown sensor bias that persists over time may need to be part of the state; calling each biased report independent fresh noise would lose its time dependence.

A useful analogy is a paused game with a saved state. Resuming should require the saved state and the next input, rather than the complete record of every button pressed. The Markov assumption makes the probabilistic state play that role. It is a statement about the chosen model, not a universal property of position coordinates.

4.2 Generative laws and belief distributions

The state, control, and measurements vary over time. If the state is complete, the history-dependent models simplify to

$$\begin{aligned} p(x_t \mid x^{t-1}, z^{t-1}, u^t) &= p(x_t \mid x_{t-1}, u_t), \\ p(z_t \mid x_t, z^{t-1}, u^t) &= p(z_t \mid x_t). \end{aligned} \tag{4.1}$$

The first is the state transition model; the second is the measurement model. Completeness is a modeling requirement: omitting information needed to predict motion or measurements can invalidate these conditional-independence assumptions.

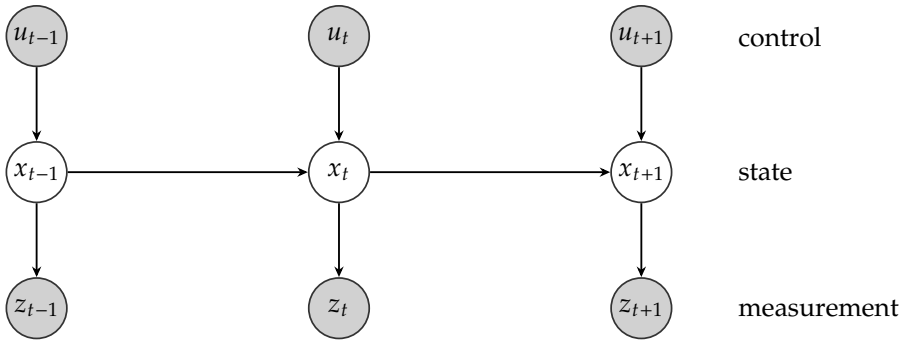


Figure 20: The controlled dynamic Bayesian network. Input u_t affects the transition into x_t; the observation z_t depends on x_t. Known inputs and observed measurements are shaded.

Definition 4.1 [Belief and predicted belief]. The belief is the posterior distribution over the state given all measurements and controls so far:

$$\mathrm{bel}(x_t) = p(x_t \mid z^t, u^t).$$

Before incorporating the current measurement, the predicted belief is

$$\overline{\mathrm{bel}}(x_t) = p(x_t \mid z^{t-1}, u^t).$$

The measurement update converts the predicted belief into the belief.

The prediction includes the latest control and omits the current measurement z_t, while keeping all earlier measurements. The relevant inference tasks are distinguished by the state's time relative to the data history:

Task	State time	Distribution
Filtering	Current	$p(x_t \mid z^t, u^t)$
Prediction	Future, $k > 0$	$p(x_{t+k} \mid z^t, u^t)$
Smoothing	Past, $k < t$	$p(x_k \mid z^t, u^t)$

Future prediction also requires a specified model or plan for future controls.

4.3 Deriving the Bayes filter

Given a prior, a transition model, a measurement model, and a stream of controls and measurements, the BAYES FILTER recursively computes the current belief. Treat controls as supplied inputs: conditional on the known history, choosing u_t provides no extra observation of x_{t-1}.

4.3.1 Prediction

Marginalize over the unknown previous state:

$$\begin{aligned}\overline{\text{bel}}(x_t) &= \int p(x_t \mid x_{t-1}, z^{t-1}, u^t) p(x_{t-1} \mid z^{t-1}, u^t)\, dx_{t-1} \\ &= \int p(x_t \mid x_{t-1}, u_t)\, \text{bel}(x_{t-1})\, dx_{t-1}.\end{aligned}$$

The transition independence simplifies the first factor. The input convention allows the second factor to be the previous belief. The integral averages possible next-state distributions over uncertainty in the previous state.

4.3.2 Measurement update

Bayes' rule and the measurement independence give

$$\begin{aligned}\text{bel}(x_t) &= \frac{p(z_t \mid x_t, z^{t-1}, u^t) p(x_t \mid z^{t-1}, u^t)}{p(z_t \mid z^{t-1}, u^t)} \\ &= \eta\, p(z_t \mid x_t) \overline{\text{bel}}(x_t),\end{aligned}$$

where the normalizing constant is independent of x_t:

$$\eta^{-1} = \int p(z_t \mid x_t) \overline{\text{bel}}(x_t)\, dx_t.$$

This integral must be finite and positive. Combining the two steps gives

$$\text{bel}(x_t) = \eta\, p(z_t \mid x_t) \int p(x_t \mid x_{t-1}, u_t)\, \text{bel}(x_{t-1})\, dx_{t-1}. \tag{4.2}$$

For discrete states, replace the integral by a sum. The finite-state HMM filter is the same recursion with no explicit control input.

Algorithm 4.1: Bayes filter for a finite state space

Input: Previous belief bel_{t-1}, input u_t, observation z_t, transition and measurement models
Output: Current belief bel_t

for each state x **do**
  $\overline{\mathrm{bel}}_t(x) \leftarrow \sum_{x'} p(x \mid x', u_t)\,\mathrm{bel}_{t-1}(x')$
  $q_t(x) \leftarrow p(z_t \mid x)\overline{\mathrm{bel}}_t(x)$
$c_t \leftarrow \sum_x q_t(x)$
for each state x **do**
  $\mathrm{bel}_t(x) \leftarrow q_t(x)/c_t$
return bel_t

Initialize from $p(x_0)$; if z_0 is available, first normalize $p(z_0 \mid x_0)p(x_0)$. In the discrete procedure, $c_t > 0$ is required and normalization happens after computing weights for all states. A continuous implementation represents and integrates functions instead of enumerating individual real-valued states.

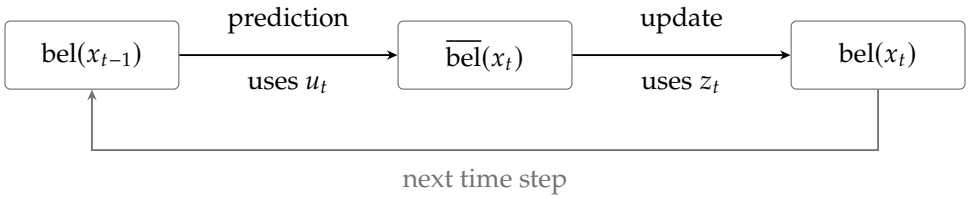


Figure 21: The prediction–update cycle. Prediction pushes the belief through the transition model; the update reweights by the measurement likelihood and normalizes.

4.4 A complete three-cell Bayes-filter example

Let the robot occupy cells A, B, C. Its previous belief is $(1/2, 1/2, 0)$. For a particular control, use the transition matrix

$$T = \begin{bmatrix} 4/5 & 1/10 & 0 \\ 1/5 & 4/5 & 1/5 \\ 0 & 1/10 & 4/5 \end{bmatrix}.$$

Column i describes where the robot may go *from* cell i; each column sums to one. There are no transitions outside this three-cell world. Suppose the received sensor report has likelihood $(1/5, 4/5, 1/5)$ at A, B, C.

1. **Spread the old belief through motion.** For cell A, collect $(4/5)(1/2) + (1/10)(1/2) + 0(0) = 9/20$. Similarly the predicted probabilities of B, C are $1/2$ and $1/20$.
2. **Score the predicted states against the report.** Multiply entry by entry: $(9/100, 2/5, 1/100)$. These are joint weights, whose sum $1/2$ is the probability of this report under the prediction.
3. **Normalize.** Divide by $1/2$ to obtain the updated belief $(9/50, 4/5, 1/50)$.

	A	B	C
Old belief	$1/2$	$1/2$	0
Prediction	$9/20$	$1/2$	$1/20$
Report likelihood	$1/5$	$4/5$	$1/5$
Unnormalized update	$9/100$	$2/5$	$1/100$
New belief	$9/50$	$4/5$	$1/50$

Motion is an accounting operation: send each old cell's probability to its possible destinations, then add the incoming contributions. Measurement is a reweighting operation: states that better explain the report retain a larger fraction. The two operations answer different questions and cannot generally be swapped.

If the same control and an independent report of the same type occur at the next time, the new prediction is $(28/125, 17/25, 12/125)$. Reweighting gives $(28/625, 68/125, 12/625)$, whose sum is $76/125$. Normalization gives $(7/95, 17/19, 3/95)$. The belief from the previous update is the complete input to this step; we do not process the old report a second time.

4.4.1 When the normalizer is zero

Suppose a model assigns zero likelihood to the received report at every state with positive predicted probability. Every updated weight is then zero, and division cannot produce a posterior. The observation was impossible under the model. A program should detect this case and report or handle the mismatch, for example using an explicitly specified outlier model. Quietly normalizing a zero vector or declaring an arbitrary cell certain is not Bayes' rule. For tiny positive weights, log-domain arithmetic or rescaling can avoid a numerical zero without changing the model.

4.5 A hallway with a door sensor

Consider a one-dimensional hallway and a robot with a binary door sensor. The known feature map m gives the door locations; the sensor model is $p(z \mid x, m)$. The map is fixed, so its conditioning can be suppressed in the filter notation.

Initially the belief is uniform because the robot does not know its position. Detecting a door favors three possible door locations, giving three equally plausible hypotheses. There is still positive probability away from doors because the sensor is imperfect. Moving along the hallway shifts the peaks and broadens them to account for motion uncertainty. Another door detection then strengthens the hypothesis consistent with both motion and the door map, while weakening the others. A later motion again shifts and broadens the belief.

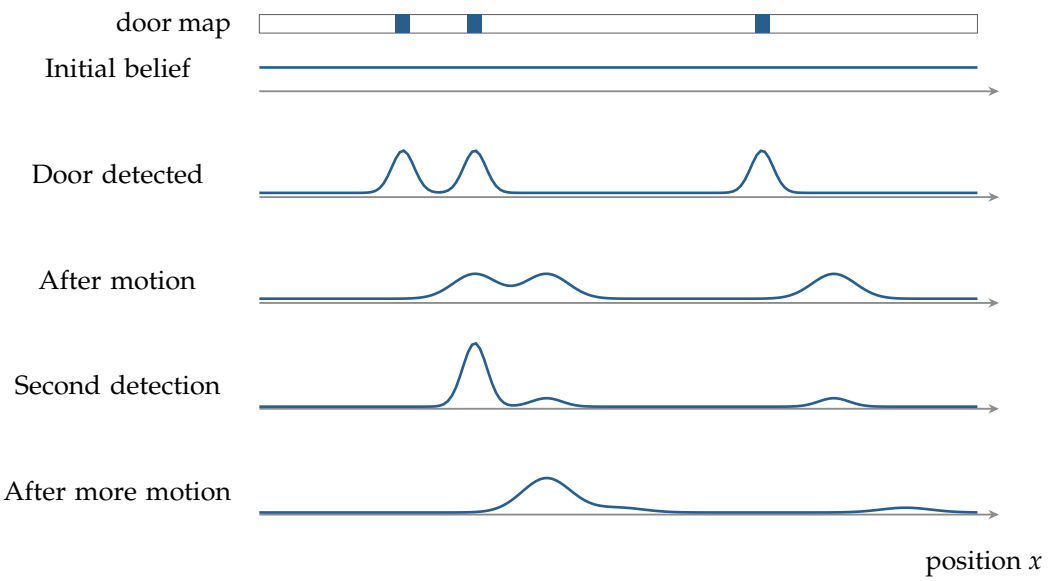


Figure 22: Schematic progression of the hallway example. Rows share a position axis; curve heights are illustrative, with no numerical probability scale. Door observations favor map-compatible positions, and uncertain motion shifts and spreads the belief.

In this example, motion spreads the belief and the observations make it more concentrated. Those are behaviors of the illustrated models and data, rather than universal guarantees that every prediction increases uncertainty or every observation reduces it.

Kalman filters, particle filters, and HMM filters implement this recursion. Belief updates also underlie partially observable Markov decision processes (POMDPs).

Source. Matthew R. Walter, TTIC 31170, Lecture 3.

Chapter 5: Least Squares and Gaussian Filtering

Least squares asks which proposed state best fits imperfect measurements. Gaussian probability explains how to weight those disagreements by uncertainty, and leads to a recursive algorithm that updates a mean and covariance.

5.1 Least squares estimation

Suppose $x \in \mathbb{R}^n$ is an unknown constant vector and $z \in \mathbb{R}^m$ contains linear, noisy measurements:

$$z_i = \sum_{j=1}^{n} H_{ij} x_j + v_i, \qquad z = Hx + v, \quad H \in \mathbb{R}^{m \times n}.$$

No probability model for the noise is needed to define least squares. Choose the estimate whose predicted measurements are closest to the observed ones.

Definition 5.1 [Least squares]. The residual is $\epsilon_z = z - H\hat{x}$. The least squares estimate minimizes

$$J(\hat{x}) = \epsilon_z^\top \epsilon_z = \|z - H\hat{x}\|_2^2.$$

Expand the quadratic and differentiate with respect to the column vector $\hat{x}$:

$$J(\hat{x}) = z^\top z - 2\hat{x}^\top H^\top z + \hat{x}^\top H^\top H \hat{x},$$
$$\nabla_{\hat{x}} J = -2H^\top z + 2H^\top H \hat{x}.$$

Setting the gradient to zero gives the normal equations

$$H^\top H \hat{x} = H^\top z. \tag{5.1}$$

If H has full column rank, $H^\top H$ is positive definite and the minimizer is unique:

$$\hat{x} = (H^\top H)^{-1} H^\top z = H^\dagger z. \tag{5.2}$$

Here $H^\dagger = (H^\top H)^{-1} H^\top$ is the pseudoinverse for this full-column-rank case. There must be at least as many measurements as unknowns, but $m \geq n$ alone is insufficient: the columns of H must be linearly independent.

The estimate is linear in z for this fixed linear measurement model. If H is square and invertible, $\hat{x} = H^{-1}z$. For a tall full-column-rank matrix,

$$H^{\dagger}H = (H^{\top}H)^{-1}H^{\top}H = I_n,$$

so $H^{\dagger}$ is a left inverse. If the noise samples are independent $\mathcal{N}(0, \sigma^2)$, their joint likelihood is proportional to $\exp[-\|z - Hx\|^2/(2\sigma^2)]$, making least squares also the maximum-likelihood estimate under that additional model.

5.1.1 Fitting a position and velocity by hand

Suppose a robot moves approximately at constant velocity for three seconds. A simple model of position is $p(t) = a + bt$, where a is the starting position and b is velocity. Three noisy position measurements at times $0, 1, 2$ are $1, 2, 2$. A straight line cannot pass through all three observations, so choose a, b to minimize their total squared discrepancy. Here

$$x = \begin{bmatrix} a \\ b \end{bmatrix}, \quad H = \begin{bmatrix} 1 & 0 \\ 1 & 1 \\ 1 & 2 \end{bmatrix}, \quad z = \begin{bmatrix} 1 \\ 2 \\ 2 \end{bmatrix}.$$

Each row of H turns the candidate parameters into one predicted measurement. For example, the last row produces $a + 2b$. Direct multiplication gives

$$H^{\top}H = \begin{bmatrix} 3 & 3 \\ 3 & 5 \end{bmatrix}, \qquad H^{\top}z = \begin{bmatrix} 5 \\ 6 \end{bmatrix}.$$

The normal equations say $3a + 3b = 5$ and $3a + 5b = 6$. Subtracting gives $2b = 1$, so $b = 1/2$ and $a = 7/6$. The fitted positions are $7/6, 5/3, 13/6$, and the residuals $z - H\hat{x}$ are $-1/6, 1/3, -1/6$. Their squared sum is $1/6$. No observation is fitted exactly; all contribute to the compromise.

The residuals add to zero and their time-weighted sum is also zero: $-1/6 + 1/3 - 1/6 = 0$ and $0(-1/6) + 1(1/3) + 2(-1/6) = 0$. Those two statements are the two rows of $H^{\top}\epsilon_z = 0$. They say that neither changing the intercept a little nor changing the slope a little can improve the fit to first order. If all three measurements were taken at time 0, the velocity column would be zero and velocity could not be inferred. More rows alone do not guarantee that all unknowns are observable.

5.1.2 Orthogonality and projection

Let $\mathcal{R}(H)$ denote the column space of H. Every possible predicted measurement Hy lies in this space. The least squares solution chooses the closest such point to z:

$$H\hat{x} = \mathcal{P}_{\mathcal{R}(H)}z, \qquad \mathcal{P}_{\mathcal{R}(H)} = H(H^{\top}H)^{-1}H^{\top}.$$

An exact solution $H\hat{x} = z$ exists if and only if $z \in \mathcal{R}(H)$.

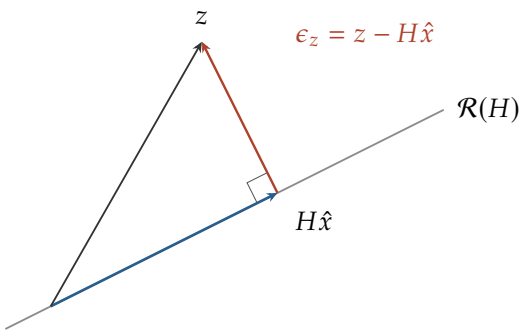


Figure 23: The fitted measurements lie in the column space. The residual is perpendicular to that space, so moving within it cannot reduce the squared error.

Proposition 5.2 [Orthogonality principle]. *The least squares residual satisfies $H^\top \epsilon_z = 0$, and hence $\epsilon_z^\top H y = 0$ for every $y \in \mathbb{R}^n$.*

Proof. The normal equations give $H^\top \epsilon_z = H^\top z - H^\top H\hat{x} = 0$. Therefore $\epsilon_z^\top H y = (H^\top \epsilon_z)^\top y = 0$. Equivalently, substituting the projection explicitly gives

$$z^\top [I - H(H^\top H)^{-1} H^\top] H y = z^\top [H - H I_n] y = 0.$$

□

Example: For repeated measurements of a scalar, $z_i = x + v_i$ for $i = 1, \dots, m$, the matrix H is the $m \times 1$ column of ones. Thus

$$H^\top H = m, \qquad H^\top z = \sum_{i=1}^{m} z_i, \qquad \hat{x} = \frac{1}{m} \sum_{i=1}^{m} z_i.$$

Least squares returns the sample mean.

5.2 Gaussian distributions

5.2.1 Univariate and multivariate densities

A scalar Gaussian with mean μ and variance σ^2 has density

$$\mathcal{N}(x; \mu, \sigma^2) = \frac{1}{\sqrt{2\pi}\,\sigma} \exp\left\{ -\frac{(x-\mu)^2}{2\sigma^2} \right\}.$$

For $x \in \mathbb{R}^d$ with mean μ and symmetric positive-definite covariance Σ,

$$\mathcal{N}(x; \mu, \Sigma) = \frac{1}{(2\pi)^{d/2} |\Sigma|^{1/2}} \exp\left\{ -\tfrac{1}{2} (x-\mu)^\top \Sigma^{-1} (x-\mu) \right\}.$$

The level sets are the ellipsoids $(x-\mu)^\top \Sigma^{-1}(x-\mu) = r^2$. Their axes point along the eigenvectors of Σ, with semi-axis lengths $r\sqrt{\lambda_i}$ for eigenvalues λ_i. Scaling $\Sigma = sI$ makes the contours circles of radius $r\sqrt{s}$: a smaller s gives a narrower, taller peak. For $\Sigma = \left[\begin{smallmatrix} 1 & \rho \\ \rho & 1 \end{smallmatrix}\right]$ the eigenvalues are $1 \pm \rho$ along the diagonal directions, so a larger correlation ρ stretches the contours along $x_1 = x_2$ and squeezes them across it.

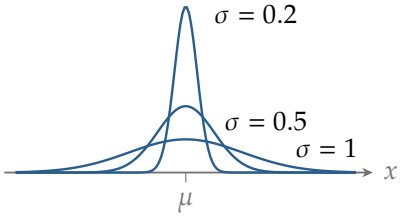


Figure 24: Univariate Gaussian densities with a common mean. All have total area one, so a smaller standard deviation gives a taller and narrower curve.

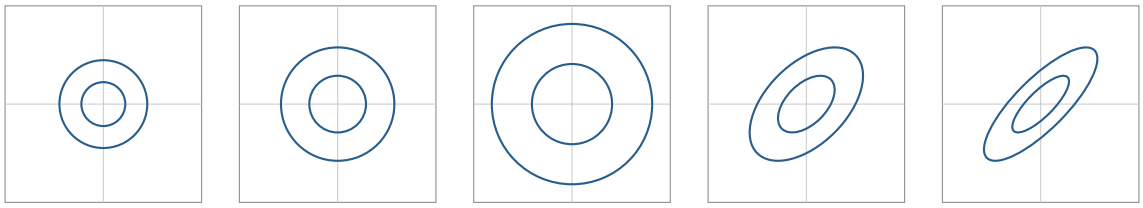


Figure 25: Level sets at Mahalanobis radii 0.75 and 1.5 for zero-mean two-dimensional Gaussians, drawn on a common $[-2.6, 2.6]^2$ window. The first three panels scale an isotropic covariance; the last two introduce positive correlation.

5.2.2 Properties used by the Kalman filter

A Gaussian is unimodal, and its mode, median, and mean coincide. It is completely described by its mean and covariance; an equivalent description by an information vector and information matrix will appear later. Two further properties make Gaussian filtering closed-form.

Proposition 5.3 [Closure under affine maps]. *If $x \sim \mathcal{N}(\mu, \Sigma)$ and $y = Ax + b$, then $y \sim \mathcal{N}(A\mu + b,\ A\Sigma A^\top)$. In one dimension, $y = ax + b$ gives $y \sim \mathcal{N}(a\mu + b,\ a^2\sigma^2)$.*

Proof. An affine map of a Gaussian vector is Gaussian, so it suffices to find the moments. By linearity, $\mathbb{E}[y] = A\mathbb{E}[x] + b = A\mu + b$, and

$$\operatorname{Cov}(y) = \mathbb{E}[(Ax - A\mu)(Ax - A\mu)^\top] = A\Sigma A^\top.$$

If $A\Sigma A^\top$ is singular, y lies in a lower-dimensional affine set and has no density, but the distribution is still the Gaussian with these moments. □

Proposition 5.4 [Product of Gaussian densities]. *For densities in the same variable x,*

$$\mathcal{N}(x;\mu_1,\Sigma_1)\,\mathcal{N}(x;\mu_2,\Sigma_2) \propto \mathcal{N}(x;\mu,\Sigma),$$

where

$$\Sigma = (\Sigma_1^{-1}+\Sigma_2^{-1})^{-1}, \qquad \mu = \Sigma(\Sigma_1^{-1}\mu_1+\Sigma_2^{-1}\mu_2) = \Sigma_2(\Sigma_1+\Sigma_2)^{-1}\mu_1 + \Sigma_1(\Sigma_1+\Sigma_2)^{-1}\mu_2.$$

In one dimension,

$$\mu = \frac{\sigma_2^2}{\sigma_1^2+\sigma_2^2}\mu_1 + \frac{\sigma_1^2}{\sigma_1^2+\sigma_2^2}\mu_2, \qquad \sigma^2 = \frac{1}{\sigma_1^{-2}+\sigma_2^{-2}}.$$

Proof. The exponent of the product is $-\frac{1}{2}$ times

$$(x-\mu_1)^\top\Sigma_1^{-1}(x-\mu_1) + (x-\mu_2)^\top\Sigma_2^{-1}(x-\mu_2) = x^\top\Sigma^{-1}x - 2x^\top(\Sigma_1^{-1}\mu_1+\Sigma_2^{-1}\mu_2) + c,$$

with c independent of x. Completing the square gives $(x-\mu)^\top\Sigma^{-1}(x-\mu)+c'$ with Σ and μ as stated, so the product is a constant multiple of $\mathcal{N}(x;\mu,\Sigma)$. For the second form of the mean, use $\Sigma = \Sigma_1(\Sigma_1+\Sigma_2)^{-1}\Sigma_2 = \Sigma_2(\Sigma_1+\Sigma_2)^{-1}\Sigma_1$, which follows from $\Sigma_1^{-1}+\Sigma_2^{-1} = \Sigma_1^{-1}(\Sigma_1+\Sigma_2)\Sigma_2^{-1}$. Then $\Sigma\Sigma_1^{-1} = \Sigma_2(\Sigma_1+\Sigma_2)^{-1}$ and $\Sigma\Sigma_2^{-1} = \Sigma_1(\Sigma_1+\Sigma_2)^{-1}$. □

The product's precision is the sum of the two precisions, and its mean is a precision-weighted average: the more certain factor pulls harder. The product of densities is not the density of the product of two random variables; it is the operation that appears in the Bayes update. Together, the two properties say that beliefs remain Gaussian if one starts with a Gaussian and applies only linear operations.

5.2.3 Combining two uncertain position estimates

Let the prior position be $X \sim \mathcal{N}(2,4)$. An independent sensor has model $Z = X + W$, with $W \sim \mathcal{N}(0,1)$, and reports $z = 4$. As a function of candidate position x, its likelihood is proportional to $\exp[-(4-x)^2/2]$. Multiply it by the prior:

$$p(x \mid z=4) \propto \exp\left[-\frac{1}{2}\left(\frac{(x-2)^2}{4} + (x-4)^2\right)\right].$$

Expand the terms inside the parentheses:

$$\frac{(x-2)^2}{4} + (x-4)^2 = \frac{5}{4}x^2 - 9x + 17 = \frac{5}{4}\left(x - \frac{18}{5}\right)^2 + \text{constant}.$$

Comparing with a Gaussian exponent gives posterior mean $18/5 = 3.6$ and variance $4/5$. The prior precision is $1/4$ and the sensor precision is 1; the sensor therefore has four times as much weight in the mean. This is like asking two advisers who disagree, while also knowing how uncertain each is. The posterior is closer to the more precise adviser.

Changing the sensor variance changes the compromise. If its variance were 4 instead of 1, both precisions would be $1/4$, the mean would be 3, and the posterior variance would be 2. If the sensor variance tends to infinity, its precision tends to zero and the posterior approaches the prior. If it tends to zero, the measurement dominates. These limits help check that the formulas have their weights in the correct places.

Both calculations assume independent new sensor noise. Two identical copies of one reading are not two independent advisers. Their correlation must be modeled; otherwise the reported posterior variance becomes too small.

5.3 The Kalman filter

5.3.1 Linear Gaussian models

Assumption 5.5 (Linear Gaussian system)**.** The process and measurement models are linear in the state and noise:

$$\begin{aligned} x_t &= A_t x_{t-1} + B_t u_t + v_t, & v_t &\sim \mathcal{N}(0, R_t), \\ z_t &= C_t x_t + w_t, & w_t &\sim \mathcal{N}(0, Q_t). \end{aligned}$$

The initial state and all noise variables are jointly Gaussian. Noise is independent across times, between the two noise sequences, and of the initial state. In particular, the noise is zero-mean and white, with $\mathbb{E}[v_t v_s^\top] = R_t \delta_{t-s}$ and $\mathbb{E}[w_t w_s^\top] = Q_t \delta_{t-s}$; the two noise sequences are uncorrelated with each other and with x_0. The initial belief is $\mathcal{N}(\mu_0, \Sigma_0)$.

Here $x_t \in \mathbb{R}^n$, $u_t \in \mathbb{R}^l$, and $z_t \in \mathbb{R}^k$. The $n \times n$ matrix A_t describes how the state evolves from $t-1$ to t without control; the $n \times l$ matrix B_t describes how the control u_t changes the state over the same interval; the $k \times n$ matrix C_t maps the state to an observation. Gaussian noise is assumed here for convenience; the zero-mean, white, uncorrelated structure is what the recursion's moment equations use. Under the assumption,

$$p(x_t \mid x_{t-1}, u_t) = \mathcal{N}(x_t; A_t x_{t-1} + B_t u_t,\ R_t), \qquad p(z_t \mid x_t) = \mathcal{N}(z_t; C_t x_t,\ Q_t).$$

The Kalman filter is the Bayes filter for this model. It represents $\mathrm{bel}(x_t) = \mathcal{N}(\mu_t, \Sigma_t)$ and $\overline{\mathrm{bel}}(x_t) = \mathcal{N}(\bar{\mu}_t, \bar{\Sigma}_t)$.

5.3.2 Prediction

Suppose $\text{bel}(x_{t-1}) = \mathcal{N}(\mu_{t-1}, \Sigma_{t-1})$. The prediction integral is

$$\begin{aligned}\overline{\text{bel}}(x_t) &= \int \underbrace{\mathcal{N}(x_t; A_t x_{t-1} + B_t u_t, R_t)}_{p(x_t \mid x_{t-1}, u_t)} \underbrace{\mathcal{N}(x_{t-1}; \mu_{t-1}, \Sigma_{t-1})}_{\text{bel}(x_{t-1})} \, dx_{t-1} \\ &= \eta \int \exp\Big\{-\tfrac{1}{2}(x_t - A_t x_{t-1} - B_t u_t)^\top R_t^{-1}(x_t - A_t x_{t-1} - B_t u_t)\Big\} \\ &\qquad \cdot \exp\Big\{-\tfrac{1}{2}(x_{t-1} - \mu_{t-1})^\top \Sigma_{t-1}^{-1}(x_{t-1} - \mu_{t-1})\Big\} \, dx_{t-1}.\end{aligned}$$

The integrand is the joint density of (x_{t-1}, x_t), and its exponent is quadratic in both. Rather than integrating directly, read the integral as the distribution of $x_t = A_t x_{t-1} + B_t u_t + v_t$ when $x_{t-1} \sim \mathcal{N}(\mu_{t-1}, \Sigma_{t-1})$ and v_t is independent of it. The pair (x_{t-1}, v_t) is jointly Gaussian and x_t is affine in it, so x_t is Gaussian with

$$\bar{\mu}_t = A_t \mu_{t-1} + B_t u_t, \qquad \bar{\Sigma}_t = A_t \Sigma_{t-1} A_t^\top + R_t. \tag{5.3}$$

The cross terms vanish because v_t has zero mean and is uncorrelated with x_{t-1}. This is the A PRIORI ESTIMATE: the motion model moves the mean, maps the covariance through A_t, and adds process noise.

5.3.3 Measurement update

The update multiplies the measurement likelihood by the predicted belief:

$$\begin{aligned}\text{bel}(x_t) &= \eta \underbrace{\mathcal{N}(z_t; C_t x_t, Q_t)}_{p(z_t \mid x_t)} \underbrace{\mathcal{N}(x_t; \bar{\mu}_t, \bar{\Sigma}_t)}_{\overline{\text{bel}}(x_t)} \\ &= \eta \exp\Big\{-\tfrac{1}{2}(z_t - C_t x_t)^\top Q_t^{-1}(z_t - C_t x_t)\Big\} \cdot \exp\Big\{-\tfrac{1}{2}(x_t - \bar{\mu}_t)^\top \bar{\Sigma}_t^{-1}(x_t - \bar{\mu}_t)\Big\}.\end{aligned}$$

Theorem 5.6 [KALMAN UPDATE]. *Assume Q_t and $\bar{\Sigma}_t$ are positive definite. Then* $\text{bel}(x_t) = \mathcal{N}(\mu_t, \Sigma_t)$*, where*

$$K_t = \bar{\Sigma}_t C_t^\top (C_t \bar{\Sigma}_t C_t^\top + Q_t)^{-1}, \qquad \mu_t = \bar{\mu}_t + K_t(z_t - C_t \bar{\mu}_t), \qquad \Sigma_t = (I - K_t C_t)\bar{\Sigma}_t. \tag{5.4}$$

Proof. Expanding the two quadratics, the exponent is $-\frac{1}{2}$ times $x_t^\top \Lambda x_t - 2 x_t^\top b_t + c$, with c independent of x_t and

$$\Lambda = C_t^\top Q_t^{-1} C_t + \bar{\Sigma}_t^{-1}, \qquad b_t = C_t^\top Q_t^{-1} z_t + \bar{\Sigma}_t^{-1} \bar{\mu}_t.$$

Since Λ is positive definite, completing the square as in Proposition 5.4 gives a Gaussian with

$$\Sigma_t = \Lambda^{-1}, \qquad \mu_t = \Sigma_t b_t = \Sigma_t (C_t^\top Q_t^{-1} z_t + \bar{\Sigma}_t^{-1} \bar{\mu}_t).$$

Substitute $\bar{\Sigma}_t^{-1} = \Lambda - C_t^\top Q_t^{-1} C_t$ in the mean:

$$\mu_t = \Sigma_t C_t^\top Q_t^{-1} z_t + \bar{\mu}_t - \Sigma_t C_t^\top Q_t^{-1} C_t \bar{\mu}_t = \bar{\mu}_t + \Sigma_t C_t^\top Q_t^{-1} (z_t - C_t \bar{\mu}_t).$$

It remains to identify $\Sigma_t C_t^\top Q_t^{-1}$ with K_t and Λ^{-1} with $(I - K_t C_t)\bar{\Sigma}_t$. Write $S_t = C_t \bar{\Sigma}_t C_t^\top + Q_t$, which is positive definite. Then

$$\Lambda \bar{\Sigma}_t C_t^\top = C_t^\top Q_t^{-1} C_t \bar{\Sigma}_t C_t^\top + C_t^\top = C_t^\top Q_t^{-1} S_t,$$

so $\Lambda K_t = C_t^\top Q_t^{-1}$, that is, $K_t = \Sigma_t C_t^\top Q_t^{-1}$. Using this identity,

$$\Lambda (I - K_t C_t) \bar{\Sigma}_t = \Lambda \bar{\Sigma}_t - C_t^\top Q_t^{-1} C_t \bar{\Sigma}_t = \bar{\Sigma}_t^{-1} \bar{\Sigma}_t = I,$$

so $(I - K_t C_t)\bar{\Sigma}_t = \Lambda^{-1} = \Sigma_t$. □

This is the A POSTERIORI ESTIMATE. The difference $z_t - C_t \bar{\mu}_t$ between the actual and predicted measurement is the INNOVATION, and K_t is the KALMAN GAIN. The gain weighs the predicted covariance in measurement space, $C_t \bar{\Sigma}_t C_t^\top$, against the measurement noise Q_t: a confident prediction or a noisy sensor gives a small correction, while an uncertain prediction or a precise sensor moves the estimate further toward the measurement. The form of equation (5.4) inverts only the $k \times k$ matrix S_t.

5.3.4 Two complete scalar Kalman steps

A robot starts with mean $\mu_0 = 0$ and variance $P_0 = 4$. Each control requests one unit of forward motion, with independent motion-noise variance $R = 1$. A position sensor has variance $Q = 1$. Thus $A = B = C = 1$. The first report is $z_1 = 2$.

1. **Predict.** The mean moves to $\bar{\mu}_1 = 0 + 1 = 1$ and the variance becomes $\bar{P}_1 = 4 + 1 = 5$. The motion is uncertain, so moving the mean is not the whole prediction.
2. **Compare the report with the prediction.** The innovation is $r_1 = 2 - 1 = 1$, with variance $S_1 = 5 + 1 = 6$.
3. **Compute the correction fraction.** The gain is $K_1 = 5/6$. The uncertain prediction yields most of the way toward the more precise reading.
4. **Update.** The new mean is $\mu_1 = 1 + (5/6)(1) = 11/6$, and the new variance is $P_1 = (1 - 5/6)5 = 5/6$.

Now apply the next control and receive $z_2 = 5/2$. Prediction gives $\bar{\mu}_2 = 17/6$ and $\bar{P}_2 = 11/6$. Hence

$$r_2 = \frac{5}{2} - \frac{17}{6} = -\frac{1}{3}, \quad S_2 = \frac{17}{6}, \quad K_2 = \frac{11}{17}, \quad \mu_2 = \frac{17}{6} - \frac{11}{51} = \frac{89}{34}, \quad P_2 = \frac{11}{17}.$$

The second correction moves backward because its innovation is negative. The filter uses the previous mean *and* variance. A rule that simply averages the old estimate with every new reading would not give these changing gains.

5.3.5 What each matrix multiplication does

For a vector state, $C\bar{\Sigma}C^\top$ converts uncertainty in state coordinates into uncertainty in sensor coordinates. Adding Q includes sensor noise, producing innovation covariance S. The product $\bar{\Sigma}C^\top$ describes which state errors tend to accompany a measurement error. Multiplying by S^{-1} converts that relationship into a correction per unit innovation. Finally Kr turns the actual discrepancy back into a state correction.

The dimensions check the interpretation: C is $k \times n$, S is $k \times k$, K is $n \times k$, and Kr is an n-entry state vector. The gain can have negative entries or entries greater than one in general coordinate systems; only the simplest scalar direct-position model makes it a correction fraction between zero and one.

For covariance updates, the numerically useful Joseph form is

$$P^+ = (I - KC)\bar{P}(I - KC)^\top + KQK^\top.$$

It follows by writing the corrected error as $(I - KC)e - Kw$ and using the independence of predicted state error e and sensor noise w. This equals $(I - KC)\bar{P}$ in exact arithmetic for the Kalman gain, but exposes symmetry and positive semidefiniteness more clearly in a numerical implementation.

Algorithm 5.1: Kalman filter, one time step

Input: Previous belief $(\mu_{t-1}, \Sigma_{t-1})$, control u_t, measurement z_t
Output: Current belief (μ_t, Σ_t)
// Prediction
$\bar{\mu}_t \leftarrow A_t\mu_{t-1} + B_t u_t$
$\bar{\Sigma}_t \leftarrow A_t\Sigma_{t-1}A_t^\top + R_t$
// Update
$K_t \leftarrow \bar{\Sigma}_t C_t^\top (C_t\bar{\Sigma}_t C_t^\top + Q_t)^{-1}$
$\mu_t \leftarrow \bar{\mu}_t + K_t(z_t - C_t\bar{\mu}_t)$
$\Sigma_t \leftarrow (I - K_t C_t)\bar{\Sigma}_t$
return (μ_t, Σ_t)

5.3.6 Properties

The ESTIMATION ERROR $\tilde{x}_t = x_t - \mu_t$ is itself a random variable. Suppose the estimate should minimize a weighted squared error at each step:

$$\mu_t = \arg\min_{\hat{x}} \mathbb{E}\left[(x_t - \hat{x})^\top S\,(x_t - \hat{x}) \,\middle|\, z^t, u^t\right]$$

for a positive-definite weight S.

- If v_t and w_t are zero-mean, uncorrelated, white, and Gaussian, the Kalman filter provides the optimal solution: its mean is the exact conditional mean, which minimizes this criterion.
- If the noise is zero-mean, uncorrelated, and white but not Gaussian, the Kalman filter provides the optimal *linear* solution, the best estimate among those that are linear in the measurements.
- If the noise is correlated or colored, a modified Kalman filter is needed.

For dense matrices, straightforward arithmetic costs $O(n^3 + n^2k + nk^2 + k^3)$ per step, including covariance prediction and the measurement solve. Memory is $O(n^2)$ for the covariance. Structure in the motion matrix and fixed-size measurements can reduce this cost, so a complexity claim must state what structure is being used. Nonlinear motion and sensing, rather than time alone, motivate the extension below.

The basic filter has several extensions: the extended and unscented Kalman filters for nonlinear systems; sparse information filters for many weakly correlated states; expectation–maximization to estimate the parameters A_t, B_t, C_t, R_t, Q_t; and smoothing, which computes $p(x_t \mid u^T, z^T)$ for $T > t$. The Kalman filter can also be derived as the recursive least-squares solution to a growing set of linear equations.

Remark 5.7. The filter is named after Rudolf E. Kálmán; with Richard Bucy's continuous-time contribution it is also called the Kalman–Bucy filter. It was used for navigation in the Apollo program.

5.4 The extended Kalman filter

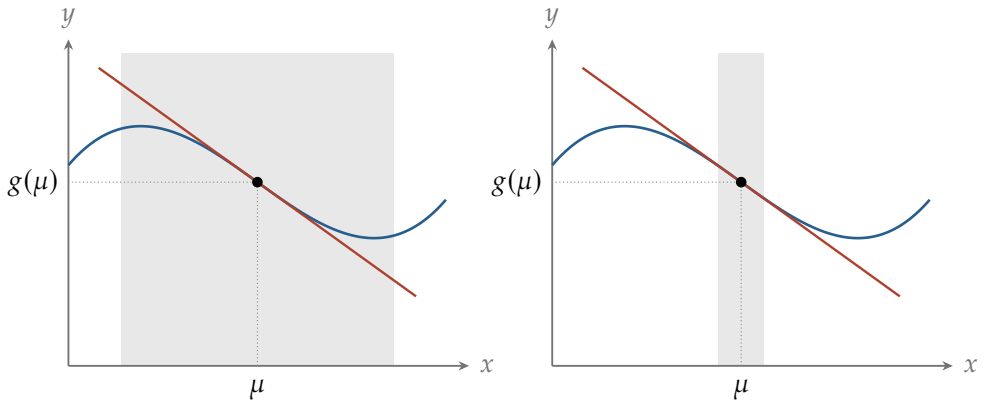
5.4.1 Nonlinear models

Most systems involve nonlinear process and measurement models:

$$\begin{aligned} x_t &= f_t(x_{t-1}, u_t) + v_t, & v_t &\sim \mathcal{N}(0, R_t), \\ z_t &= h_t(x_t) + w_t, & w_t &\sim \mathcal{N}(0, Q_t), \end{aligned}$$

in place of $A_t x_{t-1} + B_t u_t$ and $C_t x_t$. For an affine map $y = ax + b$ of $x \sim \mathcal{N}(\mu, \sigma^2)$, the output is exactly $\mathcal{N}(a\mu + b, a^2\sigma^2)$. For a nonlinear $y = g(x)$, the output density $p(y)$ is generally not Gaussian; it can be skewed or multimodal, and its mean $\mathbb{E}[g(x)]$ differs from $g(\mu)$.

The *moment-matched* Gaussian shares $p(y)$'s mean and covariance and minimizes $\mathrm{KL}(p \,\|\, q)$ among Gaussian q. Linearization instead uses $g(\mu)$ and the slope at μ. It is accurate when most input mass lies where the tangent is a good approximation; high input variance can violate this condition.



High input variance Low input variance

Figure 26: The same nonlinear function g (blue) and its tangent at the input mean (red). The shaded band is $\mu \pm 2\sigma$. With high variance, g departs far from its tangent inside the band, so the propagated distribution is distorted; with low variance, the tangent is accurate across the band. The curve is illustrative.

5.4.2 Linearization by Taylor expansion

The EXTENDED KALMAN FILTER (EKF) replaces each nonlinear model by its first-order Taylor expansion about the current estimate. For x_{t-1} close to μ_{t-1},

$$f_t(x_{t-1}, u_t) \approx f_t(\mu_{t-1}, u_t) + \left.\frac{\partial f_t(x_{t-1}, u_t)}{\partial x_{t-1}}\right|_{\mu_{t-1}} (x_{t-1} - \mu_{t-1}) = f_t(\mu_{t-1}, u_t) + F_t(x_{t-1} - \mu_{t-1}),$$

and for x_t close to the predicted mean $\bar{\mu}_t$,

$$h_t(x_t) \approx h_t(\bar{\mu}_t) + \left.\frac{\partial h_t(x_t)}{\partial x_t}\right|_{\bar{\mu}_t} (x_t - \bar{\mu}_t) = h_t(\bar{\mu}_t) + H_t(x_t - \bar{\mu}_t).$$

The Jacobians F_t $(n \times n)$ and H_t $(k \times n)$ play the roles of A_t and C_t.

With the linearized process model, x_t is affine in (x_{t-1}, v_t). The affine-closure argument behind equation (5.3) then gives mean $f_t(\mu_{t-1}, u_t)$ and covariance $F_t \Sigma_{t-1} F_t^\top + R_t$. With the linearized measurement model,

$$z_t - h_t(\bar{\mu}_t) + H_t \bar{\mu}_t \approx H_t x_t + w_t,$$

which is a linear measurement of x_t with matrix H_t. Applying theorem 5.6 to this shifted measurement gives the innovation $\big(z_t - h_t(\bar{\mu}_t) + H_t\bar{\mu}_t\big) - H_t\bar{\mu}_t = z_t - h_t(\bar{\mu}_t)$.

Algorithm 5.2: Extended Kalman filter, one time step

Input: Previous belief $(\mu_{t-1}, \Sigma_{t-1})$, control u_t, measurement z_t
Output: Current belief (μ_t, Σ_t)
// Prediction
$F_t \leftarrow \partial f_t(x_{t-1}, u_t)/\partial x_{t-1}$ evaluated at μ_{t-1}
$\bar{\mu}_t \leftarrow f_t(\mu_{t-1}, u_t)$
$\bar{\Sigma}_t \leftarrow F_t\Sigma_{t-1}F_t^\top + R_t$
// Update
$H_t \leftarrow \partial h_t(x_t)/\partial x_t$ evaluated at $\bar{\mu}_t$
$K_t \leftarrow \bar{\Sigma}_t H_t^\top (H_t\bar{\Sigma}_t H_t^\top + Q_t)^{-1}$
$\mu_t \leftarrow \bar{\mu}_t + K_t\big(z_t - h_t(\bar{\mu}_t)\big)$
$\Sigma_t \leftarrow (I - K_tH_t)\bar{\Sigma}_t$
return (μ_t, Σ_t)

The means use the nonlinear functions themselves; only the covariances and gain use the Jacobians.

5.4.3 An EKF update for a squared-distance sensor

Consider a scalar state with predicted mean 2 and variance $1/4$. A sensor measures $z = x^2 + w$, with independent noise variance 1, and reports 5. The prediction of the reading is $h(2) = 4$, and the derivative is $H = 2x|_{x=2} = 4$. The tangent approximation is $h(x) \approx 4 + 4(x - 2)$.

$$S = 4\left(\frac{1}{4}\right)4 + 1 = 5, \qquad K = \frac{(1/4)4}{5} = \frac{1}{5}, \qquad \mu^+ = 2 + \frac{1}{5}(5 - 4) = \frac{11}{5}.$$

The approximate posterior variance is $(1 - (1/5)4)(1/4) = 1/20$. The measurement pulls the estimate upward because its squared value was underpredicted. These are exact Kalman calculations for the tangent model, not exact posterior moments for the original squared model.

There is already a small approximation before the update: the true predicted mean of X^2 is $\mathrm{Var}(X) + (\mathbb{E}[X])^2 = 1/4 + 4 = 17/4$, while the EKF predicts $h(\mathbb{E}[X]) = 4$. Nonlinear functions do not generally commute with averaging.

For a more serious failure, linearize the same sensor at mean 0. Its Jacobian is then 0, so the EKF gain is zero. Yet a reading near 4 can strongly favor positions near $+2$ or -2. The measurement is informative, but the tangent at zero cannot express its information, and one Gaussian cannot describe both separated hypotheses. Linearization is like replacing a curved road by its tangent: useful nearby, unreliable over a wide region.

5.4.4 Properties and extensions

The EKF is not optimal: its Gaussian is a linearization, not the true posterior or even its moment-matched Gaussian. It has the same dense linear-algebra costs as the Kalman filter, plus evaluating the nonlinear functions and their Jacobians, and uses $O(n^2)$ covariance memory. It can *diverge* if the nonlinearities are large or the uncertainty is high, since the expansion point may then be far from much of the probability mass. In practice, it often works well even when its assumptions are violated.

For highly nonlinear models, the UNSCENTED KALMAN FILTER applies the nonlinear function to several deterministically chosen sample points (the unscented transform) to better estimate the output mean and covariance. Jacobians can also be computed by numerical linearization. A further question is OBSERVABILITY, the dual of controllability: whether the measurements contain enough information to determine the state that the (E)KF estimates.

Source. Matthew R. Walter, TTIC 31170, Lectures 2, 4.

Chapter 6: Sampling and Particle Filters

The Kalman filter needs linear or linearized motion and measurement models, can diverge under significant nonlinearity, and cannot represent multimodal beliefs. Nonparametric filters drop the Gaussian form. The discrete Bayes filter keeps a probability for each cell of a partition of the state space; the particle filter represents the belief by weighted random samples.

6.1 The discrete Bayes filter

Partition the state space $\mathcal{X}$ into disjoint regions $x_{k,t}$:

$$x_{i,t} \cap x_{j,t} = \varnothing \quad \text{for all } i \neq j, \qquad \bigcup_k x_{k,t} = \mathcal{X}.$$

The discrete Bayes filter, or histogram filter, keeps a probability $P_{k,t}$ for each region and spreads it uniformly across the region, so that inside region k

$$p(x_t) = \frac{P_{k,t}}{|x_{k,t}|}.$$

The belief is thus a histogram approximation of the continuous density. Passing a random variable through a nonlinear function $y = g(x)$ redistributes the mass of its source bins across destination bins. With a suitable discretization, a histogram can approximate non-Gaussian output densities.

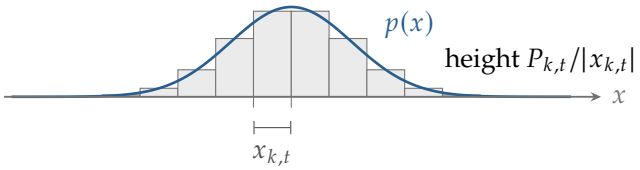


Figure 27: A histogram belief. Each region holds probability $P_{k,t}$, spread evenly over the region, so the bar height is $P_{k,t}/|x_{k,t}|$. The curve is illustrative.

In the hallway example of figure 22, the histogram filter starts from a uniform belief, multiplies by the door likelihood $p(z \mid x)$ to get three peaks, shifts and blurs them when the robot moves, and after a second door detection concentrates most of the mass on a single cell.

Replacing the integral in equation (4.2) by a sum over regions gives the algorithm. It is the forward step of the HMM forward–backward algorithm, with the transition probabilities now depending on u_t.

Algorithm 6.1: Discrete Bayes filter

Input: Previous probabilities $\{P_{k,t-1}\}$, control u_t, measurement z_t
Output: Current probabilities $\{P_{k,t}\}$

$\eta \leftarrow 0$
for each region k **do**
  $\bar{P}_{k,t} \leftarrow \sum_i p(X_t = x_k \mid X_{t-1} = x_i, u_t)\, P_{i,t-1}$
for each region k **do**
  $P_{k,t} \leftarrow p(z_t \mid X_t = x_k)\, \bar{P}_{k,t}$
  $\eta \leftarrow \eta + P_{k,t}$
for each region k **do**
  $P_{k,t} \leftarrow \eta^{-1} P_{k,t}$
return $\{P_{k,t}\}$

Here η accumulates the unnormalized total, so the final loop divides by it. The drawbacks are size and cost. The number of cells n is exponential in the number of state dimensions: a planar pose (x, y, θ) already needs a three-dimensional grid, and accuracy often requires fine resolution. The prediction sums over all n previous cells for each of the n cells, so an update costs $O(n^2)$, and memory for a dense transition table is also $O(n^2)$.

6.1.1 Probability mass is not the same as histogram height

Suppose one cell has width 1 meter and another width 2 meters, each with probability $1/2$. Their constant density heights must be $1/2$ and $1/4$ per meter respectively, because height times width gives probability mass. Setting both heights to $1/2$ would assign total mass $3/2$.

For a discretized continuous state, a cell transition probability is an integral over possible destinations, averaged over the assumed distribution inside the source cell. Evaluating a continuous transition *density* at one pair of cell centers does not automatically give a normalized transition *probability*. An implementation must integrate, approximate and normalize, or define a discrete model directly. Likewise, a nonlinear map can spread one source bin across several destination bins; it need not send a whole bin to one other bin.

A grid with 20 bins per coordinate needs $20^2 = 400$ cells for a two-coordinate position, $20^3 = 8000$ for position and heading, and $20^6 = 64{,}000{,}000$ for six coordinates. This is the practical cost of a representation that covers every combination. Sparse local transitions can reduce update time below the dense $O(n^2)$ bound, but do not eliminate the number of cells.

6.2 Monte Carlo approximation

The particle filter keeps a set of samples instead. It can use the actual motion and measurement models, can compute properties of the belief such as its moments, and concentrates computation where the probability is, giving an adaptive resolution. It is also called the Monte Carlo filter, survival of the fittest, or condensation. In global localization the samples start spread over the whole free space of the map and, as measurements arrive, collapse onto a small cluster at the robot's pose.

6.2.1 Empirical distributions

The Dirac delta $\delta(x)$ is a generalized function that vanishes for $x \neq 0$ and satisfies $\int_{-\infty}^{\infty} \delta(x)\,dx = 1$. The Dirac delta distribution $\delta_\mu(x) = \delta(x - \mu)$ puts all mass at μ. Given samples $x^{(i)} \sim p(x)$, the empirical distribution is

$$\hat{p}(x) = \frac{1}{n} \sum_{i=1}^{n} \delta\big(x - x^{(i)}\big).$$

The more samples fall into an interval, the higher the probability assigned to it.

6.2.2 Monte Carlo estimates

For any function f, such as cell occupancy or distance to an obstacle, substituting $\hat{p}$ for p gives

$$\mathbb{E}[f(x)] = \int f(x)p(x)\,dx \approx \int f(x)\,\frac{1}{n}\sum_{i=1}^{n} \delta\big(x - x^{(i)}\big)\,dx = \frac{1}{n}\sum_{i=1}^{n} f\big(x^{(i)}\big) =: \hat{\mathbb{E}}[f(x)].$$

Each term has mean $\mathbb{E}[f(x)]$, so the estimate is unbiased. For independent samples its variance is $\mathrm{Var}(f(x))/n$, inversely proportional to n, and by the strong law of large numbers it converges almost surely to $\mathbb{E}[f(x)]$ as $n \to \infty$.

6.3 Sampling from distributions

Sampling from a Gaussian or a uniform distribution is easy. The general strategy is to reduce to sampling $s \sim \mathrm{Uniform}[0, 1]$.

- *Discrete distributions.* If $X \in \{X_1, \ldots, X_m\}$ with $P(X_i) = \theta_i$, partition $[0, 1]$ into consecutive intervals $[0, \theta_1), [\theta_1, \theta_1 + \theta_2), \ldots$ of lengths θ_i and return X_i when s lands in the ith interval.
- *Inverse transform.* For continuous $x \sim p(x)$ with cumulative distribution $F_x(y) = p(x \leq y) = \int_{-\infty}^{y} p(x)\,dx$, return $F_x^{-1}(s)$. This has the right law because $p(F_x^{-1}(s) \leq y) = p(s \leq F_x(y)) = F_x(y)$. For the exponential density $p(x) = \lambda e^{-\lambda x}$, $F_x(y) = 1 - e^{-\lambda y}$ and $F_x^{-1}(s) = -\lambda^{-1}\ln(1 - s)$. The catch is that F_x often cannot be computed or inverted.

- *Gaussians (polar Box–Muller).* Draw $s_1, s_2 \sim \text{Uniform}[-1, 1]$ independently and reject the pair unless $0 < s_1^2 + s_2^2 < 1$, so that (s_1, s_2) is uniform on the disk with density $1/\pi$. With $r^2 = s_1^2 + s_2^2$,

$$x_1 = s_1 \sqrt{\frac{-2 \ln r^2}{r^2}}, \qquad x_2 = s_2 \sqrt{\frac{-2 \ln r^2}{r^2}}$$

  are independent $\mathcal{N}(0, 1)$ samples.

- *Rejection sampling.* Suppose $p(x) < 1$ for all x on a bounded domain. Sample x uniformly, sample $c \sim \text{Uniform}[0, 1]$, and keep x if $p(x) > c$. A point is kept with probability $p(x)$, so accepted points have density proportional to p. Points where p is small are mostly rejected, which wastes samples.

6.4 Importance sampling

Suppose $p(x)$ is hard to sample from but easy to evaluate at any point. Take any density q with $q(x) > 0$ wherever $p(x) > 0$. Then

$$\mathbb{E}_{X \sim p}[f(X)] = \int f(x) \frac{p(x)}{q(x)} q(x)\, dx = \mathbb{E}_{X \sim q}\left[\frac{p(X)}{q(X)} f(X)\right] \approx \frac{1}{m} \sum_{i=1}^{m} \frac{p(x^{(i)})}{q(x^{(i)})} f\big(x^{(i)}\big), \qquad x^{(i)} \sim q(x).$$

Equivalently, the weighted empirical distribution

$$\hat{p}(x) = \frac{1}{n} \sum_{i=1}^{n} \frac{p(x^{(i)})}{q(x^{(i)})}\, \delta_{x^{(i)}}(x)$$

approximates p. The TARGET p is hard to sample; the PROPOSAL q is easy to sample; the IMPORTANCE WEIGHT $w^{(i)} = p(x^{(i)})/q(x^{(i)})$ accounts for the difference between them. In a filter, the target is a posterior. A likelihood-only incremental weight appears for the particular motion-prior proposal derived below; it is not the general importance ratio.

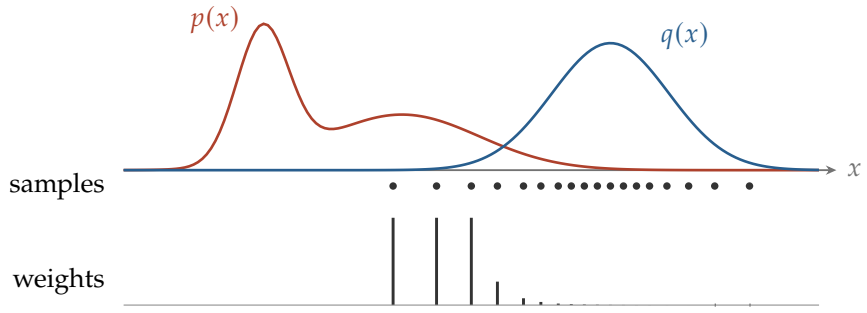


Figure 28: Importance sampling. Samples (dots) are drawn from the proposal q (blue), and each is weighted by p/q (bar heights, capped for display). Samples where the target p (red) is large relative to q get large weights. Here q covers p's left mode poorly, so few samples represent it. Curves are illustrative.

6.4.1 Resampling

Importance sampling with resampling converts weighted samples into unweighted ones: sample $x^{(1)}, \ldots, x^{(n)}$ from an easy density such as a uniform, attach the weights $w^{(i)} = p(x^{(i)})/q(x^{(i)})$, and draw n new particles with replacement from $\{x^{(i)}\}$ with probabilities proportional to $\{w^{(i)}\}$. The result is approximately distributed according to the target: high-weight particles are duplicated and low-weight ones disappear.

6.4.2 Importance weights as a correction for uneven sampling

Suppose the target assigns probabilities $p(A) = 1/5$ and $p(B) = 4/5$, but an easy proposal chooses each state with probability $1/2$. The importance ratios are $p(A)/q(A) = 2/5$ and $p(B)/q(B) = 8/5$. A B sample gets four times the weight of an A sample because the proposal underrepresents B relative to the target.

For one illustrative draw A, A, B, B, the raw weights are $(2/5, 2/5, 8/5, 8/5)$. They sum to 4, giving normalized weights $(1/10, 1/10, 2/5, 2/5)$. The estimated probability of B is the sum of its particles' weights, $2/5 + 2/5 = 4/5$. Other random sample counts can give a different finite-sample estimate.

There are two related estimators. If the normalized target p is known, $N^{-1} \sum_i [p(x^{(i)})/q(x^{(i)})] f(x^{(i)})$ is unbiased under the usual integrability assumptions. If only unnormalized weights are available, divide by their sample sum and use $\sum_i \tilde{w}_i f(x^{(i)})$. This self-normalized importance estimator is generally biased for finite N, but is consistent under suitable support and integrability conditions. The realized empirical weights need not sum to one before this normalization.

The proposal must cover every region that matters to the target. If $q(B) = 0$, no particle can be drawn at B, and no finite weight correction can recover its missing probability. Think of surveying only one neighborhood and then trying to reweight the survey into a citywide census: reweighting helps with unequal sampling, but cannot reconstruct a neighborhood that was never eligible to be sampled.

6.5 Sequential importance sampling

6.5.1 Weights over time

The goal is an importance sampler that approximates $p(x_{1:t})$ sequentially. Suppose $\{x_1^{(i)}\}$ was drawn from $q_1(x_1)$ with weights $w_1^{(i)} = p(x_1^{(i)})/q_1(x_1^{(i)})$. At $t = 2$, extend each sample by $x_2^{(i)} \sim q(x_2 \mid x_1^{(i)})$, so that q factors over time, $q_2(x_{1:2}) = q(x_2 \mid x_1) q(x_1)$. The weight is then

$$w_2 = \frac{p(x_{1:2})}{q(x_2 \mid x_1) q(x_1)} = \frac{p(x_1)}{q(x_1)} \cdot \frac{p(x_{1:2})}{p(x_1) q(x_2 \mid x_1)} = w_1(x_1) \frac{p(x_{1:2})}{p(x_1) q(x_2 \mid x_1)}.$$

In general, use a history proposal that factors as

$$q(x_{1:t}) = q(x_1) \prod_{k=2}^{t} q(x_k \mid x_{1:k-1}).$$

After drawing a new state from the last conditional, the weight obeys

$$w_t = w_{t-1}(x_{1:t-1}) \frac{p(x_{1:t})}{p(x_{1:t-1})\, q(x_t \mid x_{1:t-1})}.$$

Old samples are never revised: each step appends a new state and multiplies in one factor.

6.5.2 Application to Bayesian filtering

Maintain a belief over the state history $x_{0:t} = (x_0, \ldots, x_t)$ rather than only the current state. Bayes' rule and the Markov assumptions give

$$\mathrm{bel}(x_{0:t}) = p(x_{0:t} \mid u_{1:t}, z_{1:t}) = p(x_{0:t-1} \mid u_{1:t-1}, z_{1:t-1}) \frac{p(z_t \mid x_t)\, p(x_t \mid x_{t-1}, u_t)}{p(z_t \mid z_{1:t-1}, u_{1:t})}.$$

For nonlinear or non-Gaussian models this has no closed form, and it is often too complex to sample from directly. Even if it could be sampled, the cost would grow at least linearly in t. The aim is fixed cost per step, without modifying previously sampled trajectories $\{x^{(i)}_{0:t-1}\}_{i=1}^{n}$. The controls are dropped from the notation below.

Choose a proposal that factors over time:

$$q(x_{0:t} \mid z_{1:t}) = q(x_{0:t-1} \mid z_{1:t-1})\, q(x_t \mid x_{0:t-1}, z_{1:t}) = q(x_0) \prod_{k=1}^{t} q(x_k \mid x_{0:k-1}, z_{1:k}).$$

This is a restriction: the proposal for earlier states cannot use later measurements. The importance weight is

$$w_t(x_{0:t}) = \frac{p(x_{0:t} \mid z_{1:t})}{q(x_{0:t} \mid z_{1:t})} = \frac{1}{p(z_{1:t})} \frac{p(x_{0:t}, z_{1:t})}{q(x_{0:t} \mid z_{1:t})}.$$

The factor $p(z_{1:t})$ is the same for every sample and is removed by normalization, so it suffices to track the UNNORMALIZED WEIGHT $\bar{w}_t(x_{0:t}) = p(x_{0:t}, z_{1:t})/q(x_{0:t} \mid z_{1:t})$.

Proposition 6.1 [WEIGHT RECURSION].

$$\bar{w}_t(x_{0:t}) = \bar{w}_{t-1}(x_{0:t-1}) \frac{p(z_t \mid x_t)\, p(x_t \mid x_{t-1})}{q(x_t \mid x_{0:t-1}, z_{1:t})}.$$

Proof. By the Markov and measurement assumptions, $p(x_{0:t}, z_{1:t}) = p(z_t \mid x_t)\, p(x_t \mid x_{t-1})\, p(x_{0:t-1}, z_{1:t-1})$. Divide by the factored proposal and group $p(x_{0:t-1}, z_{1:t-1})/q(x_{0:t-1} \mid z_{1:t-1}) = \bar{w}_{t-1}(x_{0:t-1})$. □

Algorithm 6.2: Sequential importance sampling

Input: Initialized states and normalized weights at time 0

for $t = 1, 2, \ldots$ **do**
  for each $i \in \{1, \ldots, n\}$ **do**
    Sample $x_t^{(i)} \sim q\big(x_t \mid x_{0:t-1}^{(i)}, z_{1:t}\big)$ and set $x_{0:t}^{(i)} = \big(x_{0:t-1}^{(i)}, x_t^{(i)}\big)$
    $\bar{w}_t^{(i)} \leftarrow \bar{w}_{t-1}^{(i)} \dfrac{p(z_t \mid x_t^{(i)})\, p(x_t^{(i)} \mid x_{t-1}^{(i)})}{q(x_t^{(i)} \mid x_{0:t-1}^{(i)}, z_{1:t})}$
  for each $i \in \{1, \ldots, n\}$ **do**
    $w_t^{(i)} \leftarrow \bar{w}_t^{(i)} / \sum_{j=1}^{n} \bar{w}_t^{(j)}$

Each step moves every particle by the proposal and reweights it by how well it explains the new measurement. The particle-filter objective is exactly this: given samples $\mathcal{P}_{t-1} = \{x_{0:t-1}^{(1)}, \ldots, x_{0:t-1}^{(n)}\}$ representing $\mathrm{bel}(x_{0:t-1}) = p(x_{0:t-1} \mid z_{1:t-1}, u_{1:t-1})$, find samples $\mathcal{P}_t = \{x_{0:t}^{(1)}, \ldots, x_{0:t}^{(n)}\}$ representing $\mathrm{bel}(x_{0:t})$.

6.6 Choosing the proposal

Repeated importance updates can concentrate weight onto very few particles. In difficult models, one particle may eventually carry nearly all the weight, leaving a poor approximation of the posterior. There are two remedies. The first is to choose a proposal that minimizes the variance of the weights conditioned on $x_{0:t-1}^{(i)}$ and $z_{1:t}$; the second, resampling, is treated in the next subsection.

6.6.1 The optimal proposal

The proposal

$$q\big(x_t \mid x_{0:t-1}^{(i)}, z_{1:t}, u_t\big) = p\big(x_t \mid x_{t-1} = x_{t-1}^{(i)}, z_t, u_t\big)$$

achieves zero conditional variance. By Proposition 6.1 and $p(z_t \mid x_t)\, p(x_t \mid x_{t-1}, u_t) = p(z_t, x_t \mid x_{t-1}, u_t) = p(x_t \mid x_{t-1}, z_t, u_t)\, p(z_t \mid x_{t-1}, u_t)$,

$$\bar{w}_t^{(i)} = \bar{w}_{t-1}^{(i)} \frac{p(z_t \mid x_t^{(i)})\, p(x_t^{(i)} \mid x_{t-1}^{(i)}, u_t)}{p(x_t^{(i)} \mid x_{t-1}^{(i)}, z_t, u_t)} = \bar{w}_{t-1}^{(i)}\, p\big(z_t \mid x_{t-1}^{(i)}, u_t\big) = \bar{w}_{t-1}^{(i)} \int p(z_t \mid x_t)\, p\big(x_t \mid x_{t-1}^{(i)}, u_t\big)\, dx_t.$$

The new weight does not depend on the sampled $x_t^{(i)}$, so given the past it is deterministic. Two challenges remain: it is often difficult to sample from $p(x_t \mid x_{t-1}, z_t, u_t)$, and the integral for the weight is expensive.

Example: Take nonlinear Gaussian motion with a linear Gaussian measurement,

$$x_t = f_t(x_{t-1}, u_t) + v_t, \quad v_t \sim \mathcal{N}(0, R_t), \qquad z_t = Hx_t + w_t, \quad w_t \sim \mathcal{N}(0, Q_t).$$

Given $x_{t-1}^{(i)}$, the prior on x_t is $\mathcal{N}(f_t(x_{t-1}^{(i)}, u_t), R_t)$, and completing the square as in the proof of theorem 5.6 gives $p(x_t \mid x_{t-1}^{(i)}, u_t, z_t) = \mathcal{N}(m_t, \Sigma)$ with

$$\Sigma = \left(R_t^{-1} + H^\top Q_t^{-1} H\right)^{-1}, \qquad m_t = \Sigma\left(R_t^{-1} f_t(x_{t-1}^{(i)}, u_t) + H^\top Q_t^{-1} z_t\right).$$

Since $z_t = Hx_t + w_t$ is affine in Gaussian variables, $z_t \mid x_{t-1}, u_t \sim \mathcal{N}(Hf_t(x_{t-1}, u_t),\ HR_tH^\top + Q_t)$, so

$$p(z_t \mid x_{t-1}, u_t) \propto \exp\!\left(-\tfrac{1}{2}\left(z_t - Hf_t(x_{t-1}, u_t)\right)^\top \left(Q_t + HR_tH^\top\right)^{-1}\left(z_t - Hf_t(x_{t-1}, u_t)\right)\right).$$

Both the optimal proposal and its weight are available in closed form here.

6.6.2 The Markov prior as proposal

The common alternative proposes from the motion model, $q(x_t \mid x_{0:t-1}^{(i)}, z_{1:t}, u_t) = p(x_t \mid x_{t-1} = x_{t-1}^{(i)}, u_t)$. The transition density cancels in Proposition 6.1, leaving

$$\bar{w}_t^{(i)} = \bar{w}_{t-1}^{(i)}\, p\left(z_t \mid x_t^{(i)}\right).$$

This is the STANDARD PARTICLE FILTER. In robotics the proposal is typically a Gaussian motion model on translation and rotation; repeatedly sampling it along a path produces a sample cloud that grows and bends with the accumulated heading uncertainty. Sampling and weighting are easy, but the proposal ignores z_t. When the measurement likelihood is much sharper than the motion model, most particles fall where $p(z_t \mid x_t)$ is tiny, giving many low-weight samples.

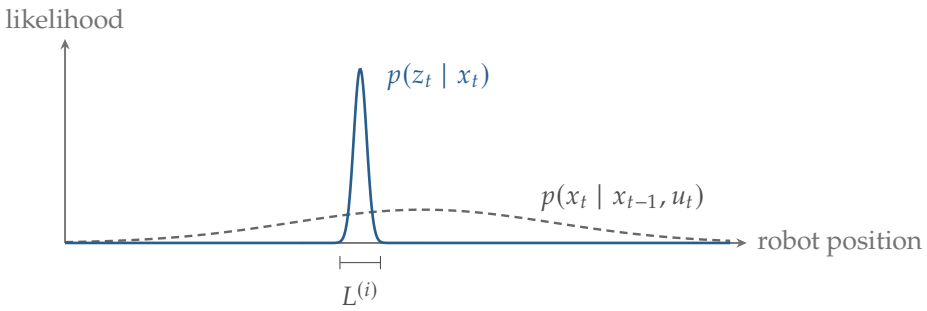


Figure 29: A peaked measurement likelihood (blue) and a broad motion model (dashed). Only particles proposed inside the narrow region $L^{(i)}$ receive appreciable weight. Curves are illustrative.

6.7 Resampling

The second remedy eliminates particles with small normalized weight and concentrates on those with large weight. The idea is to resample only when the particles are ineffective. With normalized weights $\tilde{w}_t^{(i)}$, the EFFECTIVE SAMPLE SIZE

$$\hat{N}_{\text{eff}} = \frac{1}{\sum_{i=1}^{N} \left(\tilde{w}_t^{(i)}\right)^2}$$

is inversely related to the variance of the weights. If all $\tilde{w}_t^{(i)} = 1/N$, then $\hat{N}_{\text{eff}} = N$; if all but one weight is zero, then $\hat{N}_{\text{eff}} = 1$.

Algorithm 6.3: Sequential importance resampling (SIR)

Input: Initialized states and normalized weights at time 0

for $k = 1, 2, \ldots$ **do**
  for each $i \in \{1, \ldots, n\}$ **do**
    Sample $x_k^{(i)} \sim q\left(x_k \mid x_{0:k-1}^{(i)}, z_{1:k}\right)$ and set $x_{0:k}^{(i)} = \left(x_{0:k-1}^{(i)}, x_k^{(i)}\right)$
    $w_k^{(i)} \leftarrow w_{k-1}^{(i)} \dfrac{p(z_k \mid x_k^{(i)})\, p(x_k^{(i)} \mid x_{k-1}^{(i)})}{q(x_k^{(i)} \mid x_{0:k-1}^{(i)}, z_{1:k})}$
  for each $i \in \{1, \ldots, n\}$ **do**
    $\tilde{w}_k^{(i)} \leftarrow w_k^{(i)} / \sum_{j=1}^{n} w_k^{(j)}$
  if $\hat{N}_{\text{eff}} \le N_{\text{thresh}}$ **then**
    Draw n particles with replacement with probabilities $\tilde{w}_k^{(i)}$ and reset all weights to $1/n$

6.7.1 A full particle update and an explicit resampling draw

Take four equally weighted scalar particles. After sampling their motions, the proposed positions are $1, 2, 3, 4$. Suppose the new report has likelihoods $1/10, 1/10, 2/5, 2/5$ at these positions. The motion-prior proposal cancels from the weight ratio, so raw weights are the old $1/4$ times these likelihoods. Their sum is $1/4$; normalizing leaves

$$\tilde{w} = (1/10, 1/10, 2/5, 2/5).$$

The estimated position mean is $1(1/10) + 2(1/10) + 3(2/5) + 4(2/5) = 31/10$. The effective sample size is

$$\hat{N}_{\text{eff}} = \frac{1}{(1/10)^2 + (1/10)^2 + (2/5)^2 + (2/5)^2} = \frac{50}{17} \approx 2.94.$$

For an illustrative threshold of 3, this triggers resampling. Build the cumulative intervals

$$[0, 1/10), \quad [1/10, 1/5), \quad [1/5, 3/5), \quad [3/5, 1].$$

Four example independent-uniform draw outcomes $0.05, 0.35, 0.65, 0.95$ select positions $1, 3, 4, 4$. Give each selected particle new weight $1/4$. The new empirical mean is 3, which differs slightly from 3.1 because resampling introduces Monte Carlo variation. Averaged over the resampling randomness, each old particle receives $N\tilde{w}_i$ copies and the weighted mean is preserved in expectation.

Do not keep the old unequal weights after duplicating particles according to those weights: doing so would favor high-weight particles twice. Resetting weights is part of the resampling operation. If no resampling occurs, retain the unequal normalized weights for the next importance update.

6.7.2 Initialization, zero weights, and storage

Choose an initial proposal q_0 that covers the prior. Sample initial states and assign weights proportional to $p(x_0)/q_0(x_0)$; if $q_0 = p(x_0)$, equal weights suffice. If an initial observation is available, multiply by its likelihood once. Only then begin the sequential recursion at time 1. There is no state x_{-1} or prior weight w_{-1} to access.

If all weights are zero, normalization is undefined. First distinguish numerical underflow from a model or particle-support failure. Log weights help with underflow. They cannot invent particles in a region that none of the proposals reached. Deliberately injecting random particles can aid recovery, but its proposal and weighting must be specified if the original posterior is to remain the target.

For filtering with a fixed-dimensional Markov model, storing current particles and weights needs $O(N)$ memory. Storing every ancestral trajectory through time t needs $O(Nt)$ memory. The phrase "fixed cost per step" assumes fixed-dimensional states, evaluable models, and a fixed particle count; it does not make the entire path history free.

6.7.3 Robot localization

In the hallway example with the Markov-prior proposal, the particles start uniformly spread. A door detection reweights them by $w_t^{(i)} = w_{t-1}^{(i)}\, p(z_t \mid x_t = x_t^{(i)})$, so the particles near the three doors gain weight. Resampling duplicates those particles; motion then shifts each one by a sample of the motion model, spreading the clusters. A second door detection makes only the cluster consistent with both observations heavy, and after resampling most particles sit there.

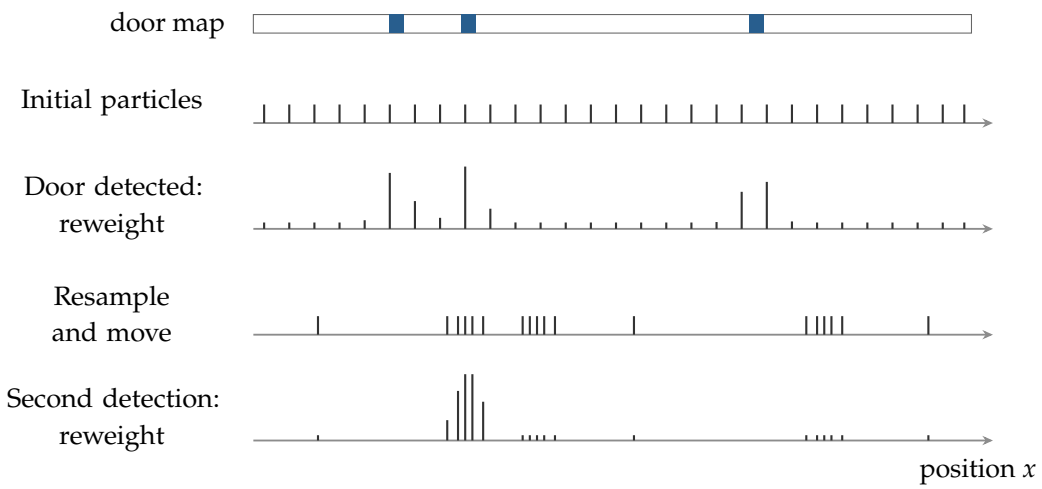


Figure 30: Schematic particle filter in the hallway. Bar positions are particles and bar heights are their weights. Particles near doors gain weight; after resampling and motion, only the cluster that moved from the first door to the second is supported by the second detection. Particle positions are illustrative.

The same procedure localizes a robot in a building map: the initial particles fill the free space, after some motion and laser measurements they form a few clusters at plausible poses, and eventually a single tight cluster remains at the true pose.

6.7.4 Issues with resampling

Consider a SIR filter for a system with deterministic motion and no sensors. All particles keep equal weight, yet resampling still removes some particles at random. As time increases, all particles become identical with high probability, which falsely suggests that the filter has converged to a single deterministic estimate. In general, resampling loses diversity. It also needs a global normalization and a sampling rule based on all weights, although those computations can be parallelized.

Particle depletion means that no particles are near the true state. It can occur when the likelihood is concentrated in a region with very little predictive state probability, so the proposed current particles miss that region. More fundamentally, the randomness of resampling means that, with nonzero probability, all particles near the true state are eliminated; repeated unnecessary resampling can make this risk substantial. Two remedies are common.

- *Roughening:* add a small amount of noise, such as Gaussian noise, to each particle after resampling.
- *Random particles:* add a small number of randomly generated particles when resampling. This reduces the likelihood of depletion, helps when the initial particles include no samples near the true state, and is simple. If the injection uses no corresponding proposal correction, the

algorithm targets a modified distribution; increasing particle count alone does not remove that change.

6.7.5 A plausible hypothesis can disappear by chance

Suppose four weighted particles include one near the true position, with total normalized weight 1/4 there. In multinomial resampling, each of four independent draws misses that particle with probability 3/4. The probability that all four draws miss it is

$$P(\text{lose the hypothesis}) = \left(\frac{3}{4}\right)^4 = \frac{81}{256},$$

about 31.6%. The sensor model did not declare that position impossible; finite random sampling removed its last representative.

More generally, if a region has total particle weight w, the probability of no offspring in that region after N independent multinomial draws is $(1 - w)^N$. This calculation describes that resampling rule; other resampling schemes have different dependence between draws. More particles can help, but blindly resampling at every step also creates repeated opportunities to lose diversity. The effective-sample-size test is a way to avoid needless resampling when the current weights are already reasonably balanced.

Resampling can only copy existing hypotheses. If every particle is already in the wrong corridor, making more copies of them will not discover the correct corridor. Recovery requires a proposal capable of reaching new regions, together with appropriate weighting.

Source. Matthew R. Walter, TTIC 31170, Lecture 5.

Part III

Learning the Map

Chapter 7: Joint Localization and Mapping

7.1 What the robot knows, and what it must estimate

Suppose a robot drives through a building that it has never visited. Its wheels provide an estimate of how far it has moved. A range sensor reports distances to nearby objects. Neither source is exact. Wheels can slip, a heading estimate can drift, and the sensor readings contain noise. The task is to use these imperfect measurements to estimate both the robot's location and a map of the building.

The robot's pose describes its position and orientation. For a robot moving on a plane, a common pose vector is $x_t = (p_{x,t}, p_{y,t}, \theta_t)^\top$: two coordinates and a heading at time t. A landmark is a feature that can be recognized and measured again, such as a corner or a pole. A feature map stores the locations of these landmarks in a common coordinate system.

Three closely related tasks should be distinguished.

- *Localization:* the map is given, and the robot estimates its pose. A measured distance can be compared with distances predicted from the known map.
- *Mapping:* the robot's path is given, and the robot estimates the map. A local sensor reading can be placed in global coordinates using the known pose.
- *Simultaneous localization and mapping (SLAM):* both the path and map are unknown. The robot must estimate them using the same observations.

The difficulty is their dependence. To place a corner in a map, the robot needs to know where it was when it saw the corner. To use that corner to locate itself later, it needs to know where the corner is. A probabilistic solution keeps their uncertainty together rather than treating one estimate as exact while estimating the other.

7.1.1 Why motion estimates alone are insufficient

Odometry estimates motion from the robot's movement, for example from wheel rotations. Integrating these motions produces a path estimate. Small errors accumulate because every new position is computed from an already uncertain position. Heading error is especially troublesome:

driving forward with a slightly wrong heading shifts all later observations sideways. Walls seen at different times may then be placed at inconsistent locations in the map.

A LOOP CLOSURE occurs when the robot recognizes a previously visited place. This supplies a new constraint between the current part of the path and an earlier part. If the motion estimate says the robot has not quite returned to its starting point but the observations identify the same place, the two sources disagree. SLAM uses their uncertainties to decide how to revise the path and map. It does not simply declare one source correct.

7.1.2 Notation and the meaning of a joint belief

We will use the following notation throughout this chapter.

Symbol	Meaning
x_t	Robot pose at time t.
$m = (m_1, \ldots, m_n)$	Static map containing n landmarks. Each m_j is a location vector.
u_t	Control or motion input for the transition from $t-1$ to t.
z_t	Sensor observation at time t; it may contain several feature measurements.
$u_{1:t}, z_{1:t}$	All controls and observations through time t.
$x_{0:t}$	The complete sequence of poses from the initial time through t.

The map is assumed static: its unknown value is the same at every time. This does not mean that its estimate stays fixed. New data can change what the robot believes about a stationary landmark.

A JOINT BELIEF assigns probability to combinations of pose and map. For example, a robot might be at position 0 with a landmark at 3, or at position 1 with the landmark at 4. Both combinations predict a relative distance of 3. A sensor that measures only that distance may distinguish these combinations from other combinations without distinguishing them from each other.

This example shows why keeping only separate pose and landmark distributions loses information. Knowing that the robot is likely near either 0 or 1 and that the landmark is near either 3 or 4 does not record which pairs go together. The joint distribution records that relationship.

7.1.3 A map drawn from an uncertain starting point

Imagine measuring room furniture with a tape measure while the location of your measuring station is uncertain. Distances from the station to two chairs can be precise even when both chairs' global coordinates are uncertain. Moving the assumed station shifts both chair coordinates together. The shared shift is not two independent chair errors.

In one dimension, suppose the robot position X has variance 4. It measures two new landmark offsets with independent noise variance 1 each. Writing the landmark locations as $M_A = X + z_A - E_A$

and $M_B = X + z_B - E_B$, their joint covariance, in order (X, M_A, M_B), is

$$\Sigma = \begin{bmatrix} 4 & 4 & 4 \\ 4 & 5 & 4 \\ 4 & 4 & 5 \end{bmatrix}.$$

Every off-diagonal 4 comes from the same uncertain robot position. The landmarks' relative separation has variance $5 + 5 - 2(4) = 2$, and each landmark's distance from the robot has variance $4 + 5 - 2(4) = 1$. The global uncertainty is large; the relative geometry is already much more precise.

A diagonal approximation would give separation variance 10, losing useful knowledge. A different mistake is to declare the robot coordinate exact when initializing the landmarks. That would understate their global uncertainty. The goal is to keep the right shared uncertainty, neither to erase it nor to pretend every variable has unrelated error.

This also explains a coordinate ambiguity. If every observation measures only relative geometry, shifting the robot and the entire map together can leave all observations unchanged. In a plane, a common rotation can also be unobservable. An initial-pose prior or another external reference anchors these freedoms; repeated relative readings alone cannot create an absolute reference.

7.1.4 Choosing what the map represents

A map need not be a list of landmarks. An OCCUPANCY GRID divides space into cells and records whether each is occupied or free, with uncertainty. A TOPOLOGICAL MAP records places and connections, such as rooms joined by doorways. A FEATURE MAP records geometric objects and their locations. The representation determines what a measurement refers to and what the robot can later ask of its map. We use feature maps in the EKF calculation below.

Different SLAM methods also use different ways to represent uncertainty and constraints. Scan matching aligns overlapping sensor scans; EKF and information filters maintain Gaussian approximations; particle-based approaches include FastSLAM; pose-graph methods optimize constraints across a trajectory. The derivations here concern Gaussian feature-based SLAM and the graph formulation that follows from it.

7.2 Online SLAM and full SLAM

ONLINE SLAM estimates the current pose and the map:

$$b_t(x_t, m) = p(x_t, m \mid z_{1:t}, u_{1:t}).$$

Here b_t is a name for the belief, not an additional random variable. FULL SLAM keeps the whole trajectory:

$$p(x_{0:t}, m \mid z_{1:t}, u_{1:t}).$$

A later observation can then revise a past pose as well as the present pose. Online SLAM removes old poses by integration, a process called MARGINALIZATION. Its belief is the marginal of the full posterior:

$$b_t(x_t, m) = \int p(x_{0:t}, m \mid z_{1:t}, u_{1:t})\, dx_0 \cdots dx_{t-1}.$$

Removing a variable from the stored state does not remove the information it contributed. As we will see in the next chapter, its effect remains as dependencies among the variables that are retained.

7.2.1 The assumptions behind the filter

The recursive calculation uses two model assumptions.

1. *Motion is Markov.* Given the previous pose and the current control, the new pose does not need the earlier history. We use $p(x_t \mid x_{t-1}, u_t)$. In this model, motion does not depend directly on the unknown map.
2. *An observation is local in time.* Given the current pose and map, the current observation does not need earlier poses, controls, or observations. We use $p(z_t \mid x_t, m)$.

The observation model must include the feature that was observed. For now, assume its identity is known. Deciding that identity is the data-association problem discussed below.

The graphical model makes the assumptions visible. Pose-to-pose arrows describe motion. An observation has the current pose and the map as parents because both determine what the sensor should see. Shaded nodes are known observations.

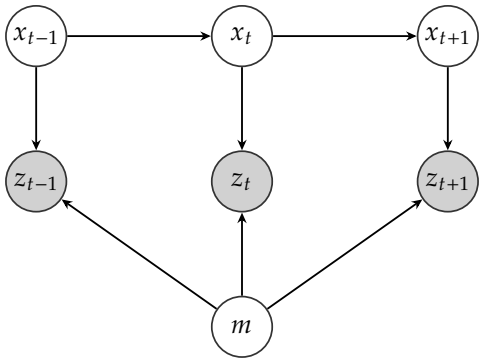


Figure 31: The static map is shared across time. Controls are omitted from the drawing; u_t is an input to the transition into x_t.

7.2.2 Deriving the prediction

Assume that $b_{t-1}(x_{t-1}, m)$ has already incorporated the previous observations. Before using z_t, define the predicted belief

$$\bar{b}_t(x_t, m) = p(x_t, m \mid z_{1:t-1}, u_{1:t}).$$

The bar means that the latest motion has been incorporated but the latest observation has not. Introduce the previous pose and integrate it out:

$$\begin{aligned}
\bar{b}_t(x_t, m) &= \int p(x_t, x_{t-1}, m \mid z_{1:t-1}, u_{1:t})\, dx_{t-1} \\
&= \int p(x_t \mid x_{t-1}, m, z_{1:t-1}, u_{1:t})\, p(x_{t-1}, m \mid z_{1:t-1}, u_{1:t})\, dx_{t-1} \\
&= \int p(x_t \mid x_{t-1}, u_t)\, b_{t-1}(x_{t-1}, m)\, dx_{t-1}.
\end{aligned}$$

The second line is the probability product rule. The last line uses the motion assumption and treats controls as given inputs. The integral averages over uncertainty in the old pose. For each possible old pose, the motion model supplies a distribution of new poses. We add these possibilities with the weights supplied by the old belief.

We integrate over the old pose, not over the map. The map remains part of the new state, and its correlations with the pose remain relevant.

7.2.3 Deriving the measurement correction

Bayes' rule gives the corrected belief:

$$\begin{aligned} b_t(x_t, m) &= \frac{p(z_t \mid x_t, m, z_{1:t-1}, u_{1:t})\, p(x_t, m \mid z_{1:t-1}, u_{1:t})}{p(z_t \mid z_{1:t-1}, u_{1:t})} \\ &= \frac{p(z_t \mid x_t, m)\, \bar{b}_t(x_t, m)}{Z_t}, \end{aligned}$$

where the normalizing constant is

$$Z_t = \int\int p(z_t \mid x_t, m)\, \bar{b}_t(x_t, m)\, dx_t\, dm.$$

The likelihood $p(z_t \mid x_t, m)$ asks how plausible the actual observation would be for a proposed pose-map pair. Multiplying by it increases the relative weight of pairs that fit the reading. Dividing by Z_t makes the posterior integrate to one. These equations define the Bayesian calculation; an implementation must still choose how to represent its distribution.

7.3 A Gaussian representation of pose and landmarks

EKF SLAM uses a feature map and approximates the joint belief by a multivariate Gaussian. Let the pose have d coordinates and each landmark have p coordinates. Stack the unknowns in the column vector

$$\xi_t = \begin{bmatrix} x_t \\ m_1 \\ \vdots \\ m_n \end{bmatrix} \in \mathbb{R}^D, \qquad D = d + np.$$

The approximation is $\xi_t \sim \mathcal{N}(\mu_t, \Sigma_t)$. The mean vector μ_t contains the pose estimate followed by the landmark estimates. The $D \times D$ covariance matrix Σ_t describes uncertainty in all these entries together.

For readability, group all landmarks into a single map block:

$$\mu_t = \begin{bmatrix} \hat{x}_t \\ \hat{m}_t \end{bmatrix}, \qquad \Sigma_t = \begin{bmatrix} P_t & C_t \\ C_t^\top & M_t \end{bmatrix}.$$

Here $P_t = \mathrm{Cov}(x_t, x_t)$ is pose covariance, $M_t = \mathrm{Cov}(m, m)$ is map covariance, and $C_t = \mathrm{Cov}(x_t, m)$ is pose-map cross-covariance. Covariance is an expectation of error products: $\mathrm{Cov}(a, b) = \mathbb{E}[(a - \mathbb{E}[a])(b - \mathbb{E}[b])^\top]$.

A diagonal variance measures uncertainty in one coordinate. A cross-covariance describes how two estimation errors vary together. In one dimension, positive cross-covariance means that an

error shifting the pose to the right tends to come with an error shifting the landmark to the right. This is natural for a relative-distance sensor: shifting both by the same amount does not change the predicted reading.

7.4 Deriving the EKF prediction

Use the motion model

$$x_t = f_t(x_{t-1}, u_t) + w_t, \qquad w_t \sim \mathcal{N}(0, R_t).$$

The function f_t predicts pose from previous pose and control; R_t is the $d \times d$ motion-noise covariance. Assume the new noise is independent of the previous state. Because the map is static, the augmented motion function is

$$\phi_t(\xi_{t-1}, u_t) = \begin{bmatrix} f_t(x_{t-1}, u_t) \\ m \end{bmatrix}.$$

The model can be nonlinear, so a Gaussian input need not give a Gaussian output. The extended Kalman filter uses a first-order Taylor approximation around the previous mean.

7.4.1 Mean and error propagation

Let F_t denote the pose Jacobian, the matrix of derivatives of f_t with respect to pose, evaluated at $\hat{x}_{t-1}$. With the control held fixed,

$$f_t(x_{t-1}, u_t) \approx f_t(\hat{x}_{t-1}, u_t) + F_t(x_{t-1} - \hat{x}_{t-1}).$$

Since the pose error and noise have zero mean, the predicted mean is

$$\bar{\mu}_t = \begin{bmatrix} f_t(\hat{x}_{t-1}, u_t) \\ \hat{m}_{t-1} \end{bmatrix}.$$

The landmark means stay unchanged. The full Jacobian and full noise covariance are

$$\mathcal{F}_t = \begin{bmatrix} F_t & 0 \\ 0 & I \end{bmatrix}, \qquad \mathcal{R}_t = \begin{bmatrix} R_t & 0 \\ 0 & 0 \end{bmatrix},$$

where I acts on the map coordinates. If $e_{t-1} = \xi_{t-1} - \mu_{t-1}$, the predicted error is approximately $\mathcal{F}_t e_{t-1} + (w_t, 0)^\top$. Taking its covariance gives

$$\bar{\Sigma}_t = \mathcal{F}_t \Sigma_{t-1} \mathcal{F}_t^\top + \mathcal{R}_t.$$

The mixed noise-state terms vanish by independence. Multiplying the blocks shows exactly what changes:

$$\boxed{\bar{\Sigma}_t = \begin{bmatrix} F_t P_{t-1} F_t^\top + R_t & F_t C_{t-1} \\ C_{t-1}^\top F_t^\top & M_{t-1} \end{bmatrix}.}$$

The map covariance is unchanged during prediction. The pose covariance changes, and its cross-covariance with the map is transformed by F_t. Treating the map as static therefore does not justify setting pose-map correlations to zero.

7.5 Deriving the measurement update

Suppose an observation of known landmarks is described by

$$z_t = h_t(\xi_t) + v_t, \qquad v_t \sim \mathcal{N}(0, Q_t).$$

The measurement has k entries, Q_t is $k \times k$, and its new noise is independent of the predicted state. The function h_t converts a proposed pose and map into a predicted sensor reading.

Evaluate its $k \times D$ Jacobian H_t at $\bar{\mu}_t$. Linearization gives

$$h_t(\xi_t) \approx h_t(\bar{\mu}_t) + H_t(\xi_t - \bar{\mu}_t).$$

Define the INNOVATION $r_t = z_t - h_t(\bar{\mu}_t)$. Substitution yields

$$r_t \approx H_t e_t + v_t, \qquad e_t = \xi_t - \bar{\mu}_t.$$

The innovation is the part of the reading not predicted by the current estimate. Its covariance and its cross-covariance with state error are

$$\begin{aligned} S_t &= \mathbb{E}[(H_t e_t + v_t)(H_t e_t + v_t)^\top] = H_t \bar{\Sigma}_t H_t^\top + Q_t, \\ \mathrm{Cov}(e_t, r_t) &= \mathbb{E}[e_t (H_t e_t + v_t)^\top] = \bar{\Sigma}_t H_t^\top. \end{aligned}$$

Again, independence makes the mixed terms zero. S_t measures how uncertain the predicted observation is, including both state uncertainty and sensor noise.

7.5.1 A range-and-bearing measurement spelled out

Let the robot pose be (p_x, p_y, θ) and a landmark be (m_x, m_y). Define $d_x = m_x - p_x$, $d_y = m_y - p_y$, and $r = \sqrt{d_x^2 + d_y^2}$. For $r > 0$, the ideal range and relative bearing are

$$h(\xi) = \begin{bmatrix} r \\ \mathrm{atan2}(d_y, d_x) - \theta \end{bmatrix}.$$

Range is distance from the robot. The function atan2 gives the global direction of the displacement, and subtracting the robot heading expresses that direction in the robot's frame. For the five state coordinates $(p_x, p_y, \theta, m_x, m_y)$, the local Jacobian is

$$H = \begin{bmatrix} -d_x/r & -d_y/r & 0 & d_x/r & d_y/r \\ d_y/r^2 & -d_x/r^2 & -1 & -d_y/r^2 & d_x/r^2 \end{bmatrix}.$$

For example, $\partial r/\partial p_x = -d_x/r$: moving the robot toward a landmark on its right decreases measured range. The negative derivative is therefore geometrically sensible. The heading derivative of bearing is -1: turning left makes a fixed landmark appear farther to the robot's right.

At pose $(1, 2, 0)$ with landmark $(4, 6)$, the displacement is $(3, 4)$ and range is 5. The Jacobian becomes

$$H = \begin{bmatrix} -3/5 & -4/5 & 0 & 3/5 & 4/5 \\ 4/25 & -3/25 & -1 & -4/25 & 3/25 \end{bmatrix}.$$

Notice that moving robot and landmark together cancels in both rows. That is the local version of the common-shift ambiguity.

Bearing innovations must respect the circular nature of angles. If a predicted bearing is 179° and the measured bearing is -179°, the nearby error is $+2^\circ$, not -358°. Wrap the difference into a consistent interval such as $(-\pi, \pi]$, using radians in derivatives and covariances. A range of zero makes bearing undefined, so this model cannot be linearized there without a different treatment.

7.5.2 Where the Kalman gain comes from

Seek a corrected estimate of the form $\mu_t = \bar{\mu}_t + K_t r_t$. The matrix K_t translates an error in measurement coordinates into a correction in state coordinates. After correction, the state error is $e_t - K_t r_t$. At the best linear correction, this remaining error is uncorrelated with the innovation: otherwise the innovation would still contain a linearly usable prediction of the error. Thus

$$\begin{aligned} 0 &= \operatorname{Cov}(e_t - K_t r_t, r_t) \\ &= \bar{\Sigma}_t H_t^\top - K_t S_t, \end{aligned}$$

which gives

$$K_t = \bar{\Sigma}_t H_t^\top S_t^{-1}, \qquad \boxed{\mu_t = \bar{\mu}_t + K_t\big[z_t - h_t(\bar{\mu}_t)\big].}$$

For the linearized Gaussian model this is also the conditional mean. In practice, solve a linear system with S_t rather than explicitly computing its inverse.

This uncorrelated-error condition also proves that the gain is optimal among linear corrections. Use another gain $K_t + \Delta$. Its error is $(e_t - K_t r_t) - \Delta r_t$. The cross terms in its covariance vanish because $e_t - K_t r_t$ is uncorrelated with r_t. Its covariance is therefore the covariance for K_t plus $\Delta S_t \Delta^\top$, a

positive semidefinite matrix. No other linear gain can reduce the variance in any direction below this result.

The new error is $(I - K_t H_t)e_t - K_t v_t$. Taking its covariance gives

$$\Sigma_t = (I - K_t H_t)\bar{\Sigma}_t(I - K_t H_t)^\top + K_t Q_t K_t^\top.$$

This is the Joseph form. Expanding it and using $K_t S_t = \bar{\Sigma}_t H_t^\top$ gives the equivalent expressions

$$\Sigma_t = \bar{\Sigma}_t - K_t S_t K_t^\top = (I - K_t H_t)\bar{\Sigma}_t.$$

The Joseph form is often preferable in floating-point calculations because the two terms visibly have the form of covariance contributions. The EKF still makes an approximation: these identities are exact for the linearized model, not for an arbitrary nonlinear posterior.

7.5.3 Why one observation can change the whole map

A measurement of landmark j depends on the robot and that landmark. The columns of H_t for other landmarks are zero. Nevertheless, the rows of $\bar{\Sigma}_t H_t^\top$ for another landmark ℓ can be nonzero because of its cross-covariances with the robot or landmark j. Therefore the corresponding rows of K_t can be nonzero. That unobserved landmark is corrected through shared uncertainty.

This is how a loop closure can correct more than the local region in view. It also explains why keeping only the diagonal of the covariance changes the method: doing so erases the relationships that carry this correction.

Consider a sequence involving landmarks A, B, and C. The robot first observes A near its starting position. After moving, its pose is less certain, so newly located B and C inherit uncertainty from that pose. It then returns and recognizes A. Correcting the robot against A can also improve B and C even if they are not currently visible. Later reobserving B can reduce uncertainty about C because they were placed using a shared uncertain pose. The covariance blocks record the common errors that make these changes possible.

7.6 Growing the map without losing shared uncertainty

Data association determines which mapped feature, if any, generated each observation. The distinction changes the operation we perform.

- *Known feature:* compare the reading with its predicted reading and apply a measurement correction.
- *New feature:* create a new state variable using an inverse observation model, and add its uncertainty and cross-covariances to the state.

An incorrect match adds a false geometric relationship. Large pose uncertainty makes this harder because several landmarks may produce plausible readings. The filter equations assume the selected match is correct; they do not by themselves solve recognition.

7.6.1 From a local reading to a global feature

An inverse observation model g converts the pose and a new measurement into a landmark location. For example, with planar pose (p_x, p_y, θ) and a measured range r and bearing β,

$$g(x, z) = \begin{bmatrix} p_x + r\cos(\theta + \beta) \\ p_y + r\sin(\theta + \beta) \end{bmatrix}, \qquad z = \begin{bmatrix} r \\ \beta \end{bmatrix}.$$

The sensor's bearing is relative to the robot heading. Adding θ converts it to a global direction; multiplying by r gives a displacement; adding the robot position gives the landmark position. Angles and angle differences must use a consistent convention.

Let $a = \theta + \beta$. The derivatives with respect to pose and measurement are

$$G_x = \begin{bmatrix} 1 & 0 & -r\sin a \\ 0 & 1 & r\cos a \end{bmatrix}, \qquad G_z = \begin{bmatrix} \cos a & -r\sin a \\ \sin a & r\cos a \end{bmatrix}.$$

These derivatives also explain a geometric effect: heading uncertainty causes greater location uncertainty for distant landmarks because the heading column contains the range r.

7.6.2 Checking the inverse sensor geometry

Take robot pose $(p_x, p_y, \theta) = (1, 2, \pi/2)$. A new landmark is measured at range 2 and bearing 0, meaning straight ahead in the robot frame. The global direction is $\theta + \beta = \pi/2$, so

$$m_x = 1 + 2\cos(\pi/2) = 1, \qquad m_y = 2 + 2\sin(\pi/2) = 4.$$

The landmark lies at $(1, 4)$, not $(3, 2)$: ignoring heading would place it along the wrong global axis. At this configuration,

$$G_x = \begin{bmatrix} 1 & 0 & -2 \\ 0 & 1 & 0 \end{bmatrix}, \qquad G_z = \begin{bmatrix} 0 & -2 \\ 1 & 0 \end{bmatrix}.$$

A small heading or bearing error moves the landmark sideways, while range error moves it along the viewing direction. Doubling the range doubles the sideways displacement from the same small angular error. Its variance contribution therefore scales with the square of range, which is why distant features are sensitive to angular uncertainty.

7.6.3 Deriving the augmented covariance

For this step, write the existing state as $\xi \sim \mathcal{N}(\mu, \Sigma)$, after any corrections using known features. Let the new reading have independent noise covariance Q. The new mean is $\hat{m}_{\text{new}} = g(\hat{x}, z)$. Define $J = [G_x \;\; 0]$, with zero columns for the existing map. Linear uncertainty propagation gives the new feature error approximately as

$$e_{\text{new}} \approx Je + G_z\epsilon,$$

where $e = \xi - \mu$ and ϵ represents the measurement error. A possible sign change in the definition of ϵ does not affect its covariance. Independence gives

$$\begin{aligned} \text{Cov}(e, e_{\text{new}}) &= \mathbb{E}[e(Je + G_z\epsilon)^\top] = \Sigma J^\top, \\ \text{Cov}(e_{\text{new}}, e_{\text{new}}) &= J\Sigma J^\top + G_z Q G_z^\top. \end{aligned}$$

Therefore the expanded state has parameters

$$\boxed{\mu^+ = \begin{bmatrix} \mu \\ g(\hat{x}, z) \end{bmatrix}, \qquad \Sigma^+ = \begin{bmatrix} \Sigma & \Sigma J^\top \\ J\Sigma & J\Sigma J^\top + G_z Q G_z^\top \end{bmatrix}.}$$

The top-left block preserves the old belief. The bottom-right block includes both robot uncertainty and sensor uncertainty. The off-diagonal blocks record the fact that the new feature was located using the uncertain robot pose. Initializing only a landmark variance while setting these blocks to zero would discard information.

The reading used to initialize the new variable must not then be treated as a second independent observation. It has already contributed to the augmented distribution. Reusing it as new evidence would count the same information twice.

7.6.4 The order of operations

Start with a prior for the robot pose and whatever map is already available. Each iteration predicts using the new control, associates the new readings, corrects with known features, and initializes genuinely new features. If the scan contains no known features, the correction step is skipped. If it contains no new features, the state size stays unchanged.

Algorithm 7.1: One EKF SLAM iteration

Input: Previous joint mean and covariance, control u_t, observations z_t
Output: Updated joint mean and covariance, with any new landmarks appended

```
Evaluate the motion model and its Jacobian at the previous pose mean
Predict the joint mean and covariance using the static-map motion model
Associate observations with known features or classify them as new
if known-feature observations are available then
    Evaluate their predicted readings and observation Jacobian
    Compute the innovation, S_t, and Kalman gain K_t
    Correct the joint mean and covariance
if new-feature observations are available then
    Evaluate the inverse observation model and its Jacobians
    Append the new means, variances, and cross-covariances
return Updated joint mean and covariance
```

When initializing several features from the same pose, they also share pose uncertainty with one another. Apply the augmentation to the stacked new features or preserve the resulting cross-covariances in sequential initialization. Treating all new features as independent would lose the relationship between them.

7.7 A complete one-dimensional SLAM calculation

The following invented scalar example follows a feature through initialization, motion, and a later observation. It uses linear models so that the Gaussian calculations are exact. Positions are measured along one line.

7.7.1 Step 1: observe a new landmark

Initially, the robot has $x_0 \sim \mathcal{N}(2, 4)$. A sensor measures landmark position relative to robot position: $z_0 = m - x_0 + v_0$, where v_0 has variance 1. The reading is $z_0 = 3$, so the estimated landmark location is $2 + 3 = 5$. Both inverse-model derivatives are 1. Hence

$$\mu_0 = \begin{bmatrix} 2 \\ 5 \end{bmatrix}, \qquad \Sigma_0 = \begin{bmatrix} 4 & 4 \\ 4 & 5 \end{bmatrix}.$$

The landmark variance is $4 + 1 = 5$. Its covariance with the robot is 4, because both estimates share the uncertain global robot position. Their relative-position variance is $4 + 5 - 2(4) = 1$, exactly the sensor variance.

7.7.2 Step 2: move the robot

The next motion is $x_1 = x_0 + 1 + w_1$, with independent motion-noise variance 1. Since the derivative with respect to x_0 is 1, prediction gives

$$\bar{\mu}_1 = \begin{bmatrix} 3 \\ 5 \end{bmatrix}, \qquad \bar{\Sigma}_1 = \begin{bmatrix} 5 & 4 \\ 4 & 5 \end{bmatrix}.$$

Only the robot variance increases. The map stays still, and the old cross-covariance remains. The predicted relative distance is $5 - 3 = 2$; its variance is now $5 + 5 - 2(4) = 2$.

7.7.3 Step 3: observe the same landmark again

A second sensor reading is $z_1 = 3/2$, with noise variance 1. This is a known landmark, so it supplies a correction rather than another map variable. The observation Jacobian is $H = [-1 \;\; 1]$. The innovation and its variance are

$$r = \frac{3}{2} - 2 = -\frac{1}{2}, \qquad S = H\bar{\Sigma}_1 H^\top + 1 = 2 + 1 = 3.$$

The gain uses both diagonal and off-diagonal covariance entries:

$$\bar{\Sigma}_1 H^\top = \begin{bmatrix} 5 & 4 \\ 4 & 5 \end{bmatrix} \begin{bmatrix} -1 \\ 1 \end{bmatrix} = \begin{bmatrix} -1 \\ 1 \end{bmatrix}, \qquad K = \begin{bmatrix} -1/3 \\ 1/3 \end{bmatrix}.$$

Applying the innovation gives

$$\mu_1 = \begin{bmatrix} 3 \\ 5 \end{bmatrix} + \begin{bmatrix} -1/3 \\ 1/3 \end{bmatrix} \left(-\frac{1}{2} \right) = \begin{bmatrix} 19/6 \\ 29/6 \end{bmatrix}.$$

The robot moves right by $1/6$ and the landmark moves left by $1/6$ in the estimate. The estimated distance becomes $5/3$, between the predicted distance 2 and measured distance $3/2$.

For the covariance, use $\Sigma_1 = \bar{\Sigma}_1 - KSK^\top$:

$$KSK^\top = \frac{1}{3} \begin{bmatrix} 1 & -1 \\ -1 & 1 \end{bmatrix}, \qquad \Sigma_1 = \begin{bmatrix} 14/3 & 13/3 \\ 13/3 & 14/3 \end{bmatrix}.$$

The relative-distance variance falls from 2 to $14/3 + 14/3 - 2(13/3) = 2/3$. Yet the individual position variances remain large. The sensor has made the geometry between robot and landmark more certain without directly revealing where the whole configuration lies globally.

7.8 What limits EKF SLAM, and why pose graphs help

With fixed pose and feature dimensions, the joint state length grows linearly with the number of landmarks. A dense covariance, with many nonzero entries, has quadratically many entries. Even a fixed-size observation can require a quadratic covariance correction because correlations connect the observed part to the rest of the map. This is why a small sensor reading does not necessarily imply a cheap update.

There are also statistical limits. A first-order Taylor approximation can be poor when pose or heading uncertainty is large. Incorrect data association can impose a false loop closure. Keeping correlations is necessary for the intended EKF calculation, but it does not remove these modeling and recognition problems.

Large-scale systems often combine three tasks: build local metric maps, recognize previously visited places, and optimize the global arrangement when loop closures are found. A POSE GRAPH records poses as nodes and relative constraints as edges; a graph can also include explicit landmark nodes. Motion supplies constraints between consecutive poses. Reobservations supply constraints between poses and landmarks or between revisited places.

Graph optimization looks for a set of positions that jointly fits these noisy constraints. It allows a loop-closure discrepancy to be distributed across several estimates instead of attaching all of it to the current pose. The graph optimization chapter derives this calculation as a weighted least-squares problem and explains why retaining old poses can preserve useful sparse structure.

Source. Matthew R. Walter, TTIC 31170, Lecture 6.

Chapter 8: Information Filters and Marginalization

8.1 Why change the Gaussian representation?

The preceding chapter represented the joint pose-map belief by a mean vector and a covariance matrix. The covariance records how estimation errors vary together. This is useful, but correlations spread through the map, so the matrix generally becomes dense. A dense matrix has many nonzero entries; a sparse matrix has many entries equal to zero. Storing and updating all the covariance entries becomes expensive as the map grows.

This chapter represents the same Gaussian using an information matrix and examines which operations become easier. The following chapter develops the complementary approach of keeping old poses and expressing full SLAM as an optimization problem. The important question is what happens to dependencies when we add measurements or remove variables. A sparse matrix is useful only if the required operations preserve its sparsity, or if the approximation used to maintain sparsity is understood.

Throughout, ξ is a column vector containing the current robot pose and map variables. Its dimension is D. When a time index is not needed, we omit it. The robot pose is x, the complete feature map is m, and the preceding chapter's covariance representation is $\xi \sim \mathcal{N}(\mu, \Sigma)$.

8.2 Deriving Gaussian information form

Assume Σ is positive definite, so that it has an inverse. The Gaussian density is

$$p(\xi) = \frac{1}{(2\pi)^{D/2}|\Sigma|^{1/2}} \exp\left[-\frac{1}{2}(\xi - \mu)^\top \Sigma^{-1}(\xi - \mu)\right].$$

Here $|\Sigma|$ denotes the determinant. For calculations involving the unknown ξ, the coefficient outside the exponential is a constant. We can absorb such constants into a normalizer and write a proportionality.

Expand the quadratic inside the exponent:

$$\begin{aligned}(\xi - \mu)^\top \Sigma^{-1}(\xi - \mu) &= \xi^\top \Sigma^{-1}\xi - \xi^\top \Sigma^{-1}\mu - \mu^\top \Sigma^{-1}\xi + \mu^\top \Sigma^{-1}\mu \\ &= \xi^\top \Sigma^{-1}\xi - 2\xi^\top \Sigma^{-1}\mu + \mu^\top \Sigma^{-1}\mu.\end{aligned}$$

The two middle scalars are equal because Σ^{-1} is symmetric. The last term does not contain ξ, so it also becomes part of the normalizing constant. This leaves

$$p(\xi) \propto \exp\left[-\frac{1}{2}\xi^\top \Lambda \xi + \eta^\top \xi\right], \qquad \boxed{\Lambda = \Sigma^{-1}, \quad \eta = \Lambda\mu.}$$

Λ is the INFORMATION MATRIX, also called the precision matrix. η is the INFORMATION VECTOR. They are alternative parameters of the same Gaussian, not additional observations.

8.2.1 Recovering the mean

The mean is the solution of $\Lambda\mu = \eta$. One way to see this is to differentiate the log density:

$$\nabla_\xi \log p(\xi) = -\Lambda\xi + \eta.$$

Its maximum is at $\xi = \Lambda^{-1}\eta$. A Gaussian is symmetric around its mean, so this maximum is its mean. In a computation, solve the system rather than first forming the inverse. Recovering the entire covariance requires $\Sigma = \Lambda^{-1}$ and can be much more expensive than storing a sparse Λ.

Example: In one dimension, let $x \sim \mathcal{N}(2, 4)$. Its precision is $\Lambda = 1/4$ and its information vector is $\eta = (1/4) \cdot 2 = 1/2$. The equation $(1/4)\mu = 1/2$ gives back $\mu = 2$. A smaller variance gives a larger precision. In several dimensions, off-diagonal entries also describe how variables are related, so precision is more than a separate confidence number for each coordinate.

8.2.2 Precision arithmetic is a ledger of evidence

For a scalar position, covariance form stores a location and a spread; information form stores precision $\Lambda = 1/P$ and precision-weighted location $\eta = \Lambda\mu$. It is useful to think of these as a ledger: a new independent measurement adds a precision contribution and a matching precision-weighted location contribution.

Start again with prior $\mu = 2, P = 4$, so $(\Lambda, \eta) = (1/4, 1/2)$. A direct measurement $z = 4$ with variance 1 contributes $(1, 4)$. Add to obtain $(\Lambda^+, \eta^+) = (5/4, 9/2)$. Solving $(5/4)\mu^+ = 9/2$ gives $\mu^+ = 18/5$, and the variance is $1/(5/4) = 4/5$. These are the same numbers obtained by multiplying Gaussian densities in the filtering chapter. The representation changed; the posterior did not.

Updating only precision would change confidence without moving the mean in the right way. Updating only the information vector would also be wrong. The pair must describe the same evidence. In several dimensions the ledger includes cross terms, so local evidence can couple coordinates rather than merely add separate confidence scores.

8.3 What the information matrix says about a graph

For scalar coordinates, expand the exponent once more:

$$-\frac{1}{2}\xi^\top \Lambda \xi + \eta^\top \xi = \sum_i \left(-\frac{1}{2}\Lambda_{ii}\xi_i^2 + \eta_i \xi_i \right) - \sum_{i<j} \Lambda_{ij}\xi_i \xi_j .$$

Each term in the first sum involves one variable. Each term in the second involves a pair. Exponentiating turns the sum into a product of single-variable and pairwise factors. This is an undirected Gaussian graphical model: a nonzero Λ_{ij} supplies an edge between i and j.

For a nonsingular Gaussian, $\Lambda_{ij} = 0$ means that ξ_i and ξ_j are conditionally independent given every other coordinate. To understand why, hold those other coordinates fixed. The remaining exponent contains separate quadratic and linear terms in ξ_i and ξ_j. If the cross term is absent, their conditional density factors into a function of ξ_i times a function of ξ_j. For vector-valued poses or features, replace an entry by the corresponding matrix block.

Conditional independence is different from zero marginal correlation. Two landmarks may be independent once the robot pose is known, yet correlated when that pose is uncertain. The covariance matrix describes the latter relationship. Its inverse can therefore be sparse even when the covariance is dense.

8.3.1 A numerical example of the distinction

Consider zero-mean scalar variables (a_1, a_2, b) with information matrix

$$\Lambda = \begin{bmatrix} 2 & 0 & -1 \\ 0 & 2 & -1 \\ -1 & -1 & 2 \end{bmatrix}.$$

There is no direct edge between a_1 and a_2, because $\Lambda_{12} = 0$. They are conditionally independent given b. Nevertheless, the covariance is

$$\Sigma = \Lambda^{-1} = \begin{bmatrix} 3/4 & 1/4 & 1/2 \\ 1/4 & 3/4 & 1/2 \\ 1/2 & 1/2 & 1 \end{bmatrix}.$$

The covariance between a_1 and a_2 is $1/4$, not zero. If the shared variable b is uncertain, an observation of one can tell us about the other through b.



Before removing b After marginalizing b

Figure 32: Removing an uncertain shared variable creates a direct dependence between its neighbors. The dashed edge is a fill edge, not an original measurement.

8.3.2 Active and passive landmarks

In an information-filter map, a landmark is ACTIVE when its information-matrix block with the robot pose is nonzero. It is PASSIVE when that block is zero. A passive landmark can still be correlated with the robot through other variables. These terms refer to direct conditional dependence, not to whether a feature was ever observed or whether it has zero uncertainty.

The active set matters because removing a pose creates new links among its neighbors. If the pose has many active landmarks, eliminating it can affect many landmark pairs.

8.4 Deriving an information-form observation update

Let $(\bar{\eta}, \bar{\Lambda})$ describe the predicted Gaussian before a measurement. A bar again means that the observation has not yet been incorporated. Suppose

$$z = h(\xi) + v, \qquad v \sim \mathcal{N}(0, Q),$$

with independent noise. As in the EKF, linearize about a state estimate $\bar{\mu}$. Define

$$H = \left.\frac{\partial h}{\partial \xi}\right|_{\bar{\mu}}, \qquad c = h(\bar{\mu}) - H\bar{\mu}.$$

Then $h(\xi) \approx H\xi + c$. The offset c is needed because the tangent line or plane does not generally pass through the origin. Write $y = z - c = z - h(\bar{\mu}) + H\bar{\mu}$. The linearized observation is $y \approx H\xi + v$.

8.4.1 Expanding the likelihood

The likelihood is proportional to

$$\exp\left[-\frac{1}{2}(y - H\xi)^\top Q^{-1}(y - H\xi)\right].$$

Expand its quadratic:

$$(y - H\xi)^\top Q^{-1}(y - H\xi) = y^\top Q^{-1} y - 2\xi^\top H^\top Q^{-1} y + \xi^\top H^\top Q^{-1} H\xi.$$

The first term is constant in ξ. Therefore the likelihood contributes $H^\top Q^{-1} H$ to the quadratic coefficient and $H^\top Q^{-1} y$ to the linear coefficient. Bayes' rule multiplies the prior by this likelihood, so their exponents add:

$$\boxed{\Lambda^+ = \bar{\Lambda} + H^\top Q^{-1} H, \qquad \eta^+ = \bar{\eta} + H^\top Q^{-1}\big[z - h(\bar{\mu}) + H\bar{\mu}\big].}$$

The superscript + denotes the result after the observation. This is the extended information filter (EIF) correction. For a linear observation, the update is exact for the Gaussian model. For a nonlinear observation, it depends on the chosen linearization point.

8.4.2 A complete update with two scalar variables

Take $\xi = (x, m)^\top$ with independent prior variables of mean 0 and variance 1. Thus $\bar{\Lambda} = I$ and $\bar{\eta} = 0$. A sensor measures $z = m - x + v$, with noise variance 1, and reports $z = 2$. Here $H = [-1 \;\; 1]$, and there is no offset. Consequently

$$H^\top H = \begin{bmatrix} 1 & -1 \\ -1 & 1 \end{bmatrix}, \qquad H^\top z = \begin{bmatrix} -2 \\ 2 \end{bmatrix}.$$

After adding the measurement,

$$\Lambda^+ = \begin{bmatrix} 2 & -1 \\ -1 & 2 \end{bmatrix}, \qquad \eta^+ = \begin{bmatrix} -2 \\ 2 \end{bmatrix}.$$

The corrected mean solves $2\mu_x - \mu_m = -2$ and $-\mu_x + 2\mu_m = 2$. Hence $\mu_x = -2/3$ and $\mu_m = 2/3$. The posterior covariance is

$$(\Lambda^+)^{-1} = \frac{1}{3}\begin{bmatrix} 2 & 1 \\ 1 & 2 \end{bmatrix}.$$

The reading says the landmark lies to the robot's right. Both estimates move to make that possible, and their errors become correlated. The new off-diagonal precision entry is the graph edge supplied by the relative measurement.

8.4.3 What is cheap, and what still costs work?

If the reading involves the robot and one landmark, H has nonzero columns only for those variables. The added matrix touches only their blocks. With a fixed number of observed features and known local mean estimates, adding the observation can take an amount of work independent of the total map size.

There are two qualifications. A batch involving more features can touch more blocks. Also, a nonlinear observation needs an appropriate mean around which to linearize. Recovering that mean

from the information parameters requires solving a system; a cheap local addition does not imply that every part of the filter is cheap.

8.5 Conditioning and marginalization are different operations

Partition the state as $\xi = (a, b)$, where a contains variables we want to retain. Partition its parameters in the same order:

$$\Lambda = \begin{bmatrix} A & B \\ B^\top & C \end{bmatrix}, \qquad \eta = \begin{bmatrix} u \\ v \end{bmatrix}.$$

Here A acts on a, C acts on b, and B links the two groups. The letters in this subsection are block names, not motion or observation Jacobians.

- *Conditioning on* $b = b_0$ means the value of b has been learned exactly. Substitute that known value into the density.
- *Marginalizing* b means its value remains unknown, but we no longer store it. Integrate over all possible values, weighted by their probabilities.

These operations answer different questions and usually give different uncertainty.

8.5.1 Conditioning in information form

With $b = b_0$, the terms involving a in the exponent are

$$-\frac{1}{2}a^\top A a - a^\top B b_0 + u^\top a = -\frac{1}{2}a^\top A a + (u - B b_0)^\top a.$$

Thus the conditional information matrix and vector are A and $u - Bb_0$. No block inverse is required to obtain these parameters. The conditional mean still requires solving $A\mu_{a|b_0} = u - Bb_0$.

8.5.2 Knowing a variable and leaving it unknown are different

Suppose two locations share an uncertain offset B. If an external survey reveals B exactly, the shared uncertainty disappears. This is conditioning. If we merely stop storing B while remaining unsure of its value, both locations still share its error. This is marginalization. Removing a column from a data structure is not itself either probabilistic operation.

The numerical three-variable example above makes this distinction visible. After marginalizing b, the covariance of (a_1, a_2) must be the upper-left block of the full covariance:

$$\Sigma_a = \begin{bmatrix} 3/4 & 1/4 \\ 1/4 & 3/4 \end{bmatrix}.$$

Its positive off-diagonal entry records the shared uncertainty. If instead b is known, the conditional covariance is $(1/2)I$. The following square-completion derivation explains how information parameters preserve this distinction without first inverting the full matrix.

8.5.3 Marginalization by completing the square

For marginalization, retain every term involving b:

$$p(a,b) \propto \exp\left[-\frac{1}{2}a^\top A a + u^\top a - \frac{1}{2}b^\top C b + b^\top(v - B^\top a)\right].$$

Assume C is positive definite. Define $d(a) = v - B^\top a$. The terms in b become

$$-\frac{1}{2}b^\top C b + b^\top d = -\frac{1}{2}(b - C^{-1}d)^\top C(b - C^{-1}d) + \frac{1}{2}d^\top C^{-1}d.$$

The first term is a centered Gaussian quadratic in b. Its integral is a constant independent of a: shifting its center does not change the integral. The second term remains in the exponent of the marginal. Expand it:

$$\frac{1}{2}(v - B^\top a)^\top C^{-1}(v - B^\top a) = \frac{1}{2}v^\top C^{-1}v - a^\top B C^{-1}v + \frac{1}{2}a^\top B C^{-1}B^\top a.$$

Discarding the constant and collecting the terms in a gives

$$p(a) \propto \exp\left[-\frac{1}{2}a^\top(A - BC^{-1}B^\top)a + (u - BC^{-1}v)^\top a\right].$$

Therefore

$$\boxed{\Lambda_{\text{marg}} = A - BC^{-1}B^\top, \qquad \eta_{\text{marg}} = u - BC^{-1}v.}$$

The matrix $A - BC^{-1}B^\top$ is the Schur complement. It accounts for uncertainty in the removed variables. Merely deleting their rows and columns would keep A, which corresponds to conditioning at the level of precision rather than marginalizing.

8.5.4 Calculating a fill edge

Return to the three-variable matrix for (a_1, a_2, b). To retain $a = (a_1, a_2)$ and remove b, take

$$A = \begin{bmatrix} 2 & 0 \\ 0 & 2 \end{bmatrix}, \qquad B = \begin{bmatrix} -1 \\ -1 \end{bmatrix}, \qquad C = [2].$$

Since $C^{-1} = 1/2$,

$$BC^{-1}B^\top = \frac{1}{2}\begin{bmatrix} 1 & 1 \\ 1 & 1 \end{bmatrix}, \qquad \Lambda_{\text{marg}} = \begin{bmatrix} 3/2 & -1/2 \\ -1/2 & 3/2 \end{bmatrix}.$$

The new off-diagonal entry $-1/2$ is the FILL-IN. It replaces the indirect dependence through the removed variable by a direct dependence among retained variables. Inverting gives the same retained covariance as before, $\begin{bmatrix} 3/4 & 1/4 \\ 1/4 & 3/4 \end{bmatrix}$, as marginalization must.

By comparison, conditioning on $b = 0$ gives precision A and covariance $\frac{1}{2}I$. Knowing b exactly removes shared uncertainty and makes a_1 and a_2 independent. Forgetting its coordinate while remaining uncertain about it does not.

8.6 Deriving time prediction in the information filter

An information-form measurement update is additive. Prediction is harder because online filtering removes the previous pose. It is useful to separate that operation into two stages.

1. *Augment:* introduce the next pose and multiply by the motion likelihood, retaining both poses temporarily.
2. *Roll up:* marginalize the previous pose, leaving the new pose and map.

8.6.1 Stage 1: add the next pose

Let the old pose be x and the next pose be y. Linearize the motion model as $y \approx Fx + c + w$, with $w \sim \mathcal{N}(0, R)$ and $c = f(\hat{x}, u) - F\hat{x}$. Partition the old information parameters over (x, m) as

$$\Lambda = \begin{bmatrix} \Lambda_{xx} & \Lambda_{xm} \\ \Lambda_{mx} & \Lambda_{mm} \end{bmatrix}, \qquad \eta = \begin{bmatrix} \eta_x \\ \eta_m \end{bmatrix}.$$

The motion factor has residual $y - Fx - c$. Equivalently, $c = [-F \;\; I \;\; 0](x, y, m)^\top + w'$ with the same noise covariance. Applying the additive linear-factor rule produces, in order (x, y, m),

$$\widetilde{\Lambda} = \begin{bmatrix} \Lambda_{xx} + F^\top R^{-1} F & -F^\top R^{-1} & \Lambda_{xm} \\ -R^{-1}F & R^{-1} & 0 \\ \Lambda_{mx} & 0 & \Lambda_{mm} \end{bmatrix}, \qquad \widetilde{\eta} = \begin{bmatrix} \eta_x - F^\top R^{-1} c \\ R^{-1} c \\ \eta_m \end{bmatrix}.$$

The zeros between y and the map have a meaning: given x, the new motion does not directly depend on landmarks. At this stage only the old pose links the new pose to the map.

8.6.2 Stage 2: remove the old pose

Regroup the augmented variables as retained $a = (y, m)$ and eliminated $b = x$. Apply the Schur-complement formulas just derived. The matrix block for the removed pose has fixed pose dimension, but it can link to many landmarks. The correction $BC^{-1}B^\top$ then touches pairs of those neighbors, creating landmark-landmark links as well as new-pose-to-landmark links.

If there are q active landmarks of fixed dimension, the correction may affect q^2 landmark-block pairs. Thus roll-up can cost quadratically in the active map size and can make that map densely connected. This is the computational cost hidden by the simple instruction to marginalize the old pose.

8.6.3 A scalar prediction worked through

Suppose the old belief over (x, m) has

$$\Lambda = \begin{bmatrix} 2 & -1 \\ -1 & 1 \end{bmatrix}, \qquad \eta = \begin{bmatrix} 0 \\ 1 \end{bmatrix}.$$

Its mean is $(1, 2)^\top$ and covariance is $\begin{bmatrix} 1 & 1 \\ 1 & 2 \end{bmatrix}$. The robot moves according to $y = x + 1 + w$, with motion variance 1. Hence $F = 1$, $c = 1$, and $R = 1$. Augmentation gives

$$\widetilde{\Lambda} = \begin{bmatrix} 3 & -1 & -1 \\ -1 & 1 & 0 \\ -1 & 0 & 1 \end{bmatrix}, \qquad \widetilde{\eta} = \begin{bmatrix} -1 \\ 1 \\ 1 \end{bmatrix}.$$

Retain (y, m) and eliminate x. The retained block is $A = I$, the linking block is $B = (-1, -1)^\top$, the removed block is $C = 3$, and the information-vector blocks are $u = (1, 1)^\top$, $v = -1$. Therefore

$$\Lambda' = \begin{bmatrix} 2/3 & -1/3 \\ -1/3 & 2/3 \end{bmatrix}, \qquad \eta' = \begin{bmatrix} 2/3 \\ 2/3 \end{bmatrix}.$$

Solving gives mean $(2, 2)^\top$ and covariance $\begin{bmatrix} 2 & 1 \\ 1 & 2 \end{bmatrix}$. The robot mean advances by 1, its variance increases by 1, and the map remains unchanged. This is exactly the covariance-form prediction, obtained by augmentation and roll-up.

8.6.4 Why eliminating one pose can connect many landmarks

Picture the old pose at the center of a star, joined to several landmarks. The new pose is also joined to it by a motion factor. Once the center is removed while remaining uncertain, its neighbors must retain the effects of that common uncertainty. The Schur complement creates pairwise blocks among those neighbors. With q landmark neighbors, there can be on the order of q^2 such pairs.

This is the same phenomenon as discrete variable elimination: a message over all surviving neighbors replaces the eliminated variable. In a Gaussian model, the message is quadratic and represented by matrix blocks. Exact zeros in the initial factor graph do not guarantee exact zeros after elimination. Choosing an elimination order is therefore a computational decision with consequences for intermediate storage and work, even though exact arithmetic would give the same final marginal.

8.7 Sparsity, approximation, and uncertainty

Repeated observations add active landmarks. Repeated roll-up connects their neighbors. Some information entries may become small, but a small nonzero entry is still a dependency. Exact information-form filtering is therefore not automatically sparse simply because the robot sees only nearby features.

One strategy bounds the number of landmarks directly connected to the robot. It partitions the map into landmarks that remain active, landmarks to be made passive, and those already passive. Making a landmark passive means enforcing conditional independence from the robot given the retained active variables. This changes a relationship in the posterior; it is not just a storage operation.

Sparse extended information filters (SEIFs) approximate the posterior to limit these dependencies. Sparsification must account for how changing one relationship affects the others. Removing selected off-diagonal entries alone is not a general guarantee of a sensible covariance: it can alter positive definiteness, uncertainty in particular directions, and the mean unless the information vector is treated consistently. The approximation studied here can lead to overconfident pose and feature estimates.

Exactly sparse extended information filtering (ESEIF) instead controls the dependencies created in the represented posterior. The term "exactly sparse" describes exact zeros in that representation; it does not mean that nonlinear SLAM has become an exact Gaussian inference problem or that all information is retained. The central aim is to obtain a sparse, statistically consistent estimate rather than reporting unjustifiably small uncertainty.

An accuracy plot alone cannot establish consistency. If an estimator reports covariance P for an error vector e, the normalized estimation error squared is $e^\top P^{-1} e$. A large value can result from a large error or from a covariance that is too small. Under a correctly calibrated Gaussian model with

a d-dimensional error, this statistic has a chi-square distribution with d degrees of freedom. That is the distribution of the sum of squares of d independent standard normal variables: scaling the error by its covariance reduces the calculation to this form. Repeatedly exceeding an appropriate statistical bound is evidence that reported uncertainty is too optimistic.

8.7.1 Interpreting a reported covariance with a scalar example

Suppose a position estimator makes an error of 2 meters. If it reports variance $P = 1$ square meter, the normalized squared error is $2^2/1 = 4$. If it reports $P = 4$, the same physical error gives $2^2/4 = 1$. The second estimate describes that error as one standard deviation; the first describes it as two. Error size and confidence must therefore be assessed together.

A single large normalized error does not establish inconsistency: even a correct Gaussian model sometimes produces large errors. Under a calibrated, zero-mean scalar Gaussian error model, $E[e^2/P] = 1$. For a d-dimensional Gaussian error with covariance P, $E[e^\top P^{-1} e] = d$. Repeated trials can compare the distribution with the corresponding chi-square law. Bias, wrong noise assumptions, and data-association errors can all disrupt that calibration; sparsity of the stored matrix alone says nothing about it.

Source. Matthew R. Walter, TTIC 31170, Lecture 7.

Chapter 9: Graph Based SLAM

The filtering chapters removed old poses as the robot moved. Here we retain them and ask for a trajectory and map that jointly explain the measurements. Local factors turn that question into a sparse optimization problem.

9.1 Keeping old poses: full SLAM as an optimization problem

Marginalization creates fill-in. An alternative is to keep old poses and their local factors. The state becomes larger, but each measurement still touches only a few variables. The distinction is between the number of unknowns and the number of direct relationships; fewer variables do not always imply a cheaper problem.

Let the unknown trajectory be $x_0, \dots, x_T$ and let the map be m. Suppose there are K feature observations. Observation z_k was made at time t_k and corresponds to landmark m_{j_k}. The index k counts observations, while t_k identifies the associated pose. Several observations may share the same time.

Assume an initial-pose prior, a static-map prior $p(m)$, independent motion noises, and independent measurement noises. With controls treated as given inputs and associations treated as known, the posterior factors as

$$p(x_{0:T}, m \mid u_{1:T}, z_{1:K}) \propto p(x_0)p(m) \prod_{i=1}^{T} p(x_i \mid x_{i-1}, u_i) \prod_{k=1}^{K} p(z_k \mid x_{t_k}, m_{j_k}).$$

We now use a locally flat map prior, so $p(m)$ contributes no additional penalty in the region under consideration. If informative map priors are available, their factors must be included instead. The pose prior fixes the coordinate frame. Without an anchor, translating or rotating the whole configuration may preserve all relative measurements, making the system non-unique.

9.1.1 Reading the factor graph as a list of local agreements

A factor graph separates unknown quantities from pieces of evidence. A variable node is a pose or landmark. A factor node represents a prior, a motion measurement, or a feature measurement. Connect a factor only to the variables needed to predict that evidence. Unlike an arrow in a Bayesian network, this line describes participation in a factor, not a direction of physical influence.

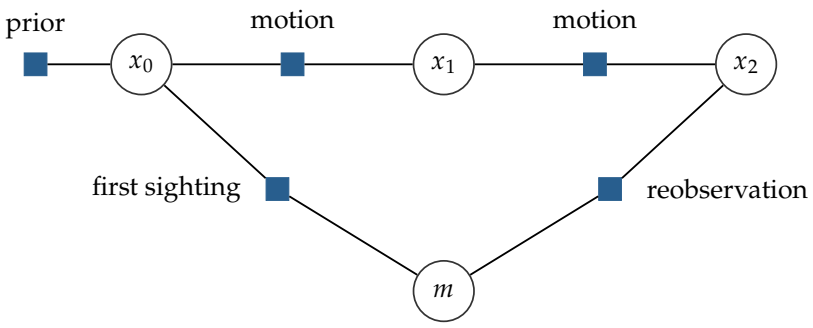


Figure 33: Circles are unknowns and squares are evidence factors. Two sightings of the same landmark connect the early and late parts of the trajectory. The prior anchors the coordinate frame.

A helpful physical analogy is a collection of points joined by springs whose preferred separations come from measurements. A precise measurement behaves like a stiff spring: disagreement is expensive. A noisy measurement behaves like a soft spring. The optimizer moves connected points to reduce the total weighted disagreement. This is an analogy for a squared-error objective, not a statement that real sensor noise is produced by physical springs.

Keeping old poses stores more points, but avoids immediately replacing all paths through an uncertain pose by dense relationships among its neighbors. A pose-only graph uses constraints between poses; a graph with explicit landmarks has the bipartite observation connections drawn here. Both can express loop closure, but their variables and factors are not identical.

9.1.2 From Gaussian factors to squared errors

Specify the noise models:

$$\begin{aligned} x_0 &\sim \mathcal{N}(\hat{x}_0, P_0), \\ x_i &= f_i(x_{i-1}, u_i) + w_i, & w_i &\sim \mathcal{N}(0, R_i), \\ z_k &= h_k(x_{t_k}, m_{j_k}) + v_k, & v_k &\sim \mathcal{N}(0, Q_k). \end{aligned}$$

Here R_i remains motion covariance and Q_k remains observation covariance, matching the earlier chapters. Define the residuals, the discrepancies between a proposed configuration and the data:

$$e_0 = x_0 - \hat{x}_0, \qquad e_i^{\text{mot}} = x_i - f_i(x_{i-1}, u_i), \qquad e_k^{\text{obs}} = z_k - h_k(x_{t_k}, m_{j_k}).$$

For a Gaussian factor with residual e and fixed covariance C,

$$-\log p = \frac{1}{2} e^\top C^{-1} e + \frac{1}{2} \log |C| + \text{constant}.$$

The determinant term is constant with respect to the state when the covariance is fixed. The logarithm of the product of factors is the sum of their logarithms. Maximizing the posterior is

therefore equivalent to minimizing

$$\boxed{J = e_0^\top P_0^{-1} e_0 + \sum_{i=1}^{T} (e_i^{\text{mot}})^\top R_i^{-1} e_i^{\text{mot}} + \sum_{k=1}^{K} (e_k^{\text{obs}})^\top Q_k^{-1} e_k^{\text{obs}}.}$$

The common factor $1/2$ is omitted because it does not change the minimizer. This is a MAXIMUM A POSTERIORI (MAP) estimate. It finds one most probable configuration; it does not by itself return the entire posterior distribution.

The notation $\|e\|_W^2 = e^\top W e$ is called a weighted squared norm. In one dimension, a variance σ^2 contributes e^2/σ^2. An error of two units counts more for a precise sensor than for a noisy one. In multiple dimensions, the covariance also accounts for different scales and correlated noise directions.

9.2 Turning nonlinear residuals into a linear least-squares step

Stack every pose and landmark coordinate into a parameter vector θ. Start from a current guess θ_0 and look for a correction δ, so the next guess will be $\theta_0 + \delta$. For one residual $e(\theta)$,

$$e(\theta_0 + \delta) \approx e(\theta_0) + J_e \delta,$$

where J_e is the residual Jacobian at θ_0. This J_e is a matrix; it is unrelated to the scalar objective J above.

To remove the covariance weight, choose a matrix L with $L^\top L = C^{-1}$. For example, such a factor follows from Cholesky factorization. Then $e^\top C^{-1} e = \|Le\|^2$. Applying this to each residual and stacking the results gives

$$\min_\delta \|A\delta - b\|^2, \qquad A = \begin{bmatrix} L_1 J_{e_1} \\ L_2 J_{e_2} \\ \vdots \end{bmatrix}, \qquad b = -\begin{bmatrix} L_1 e_1(\theta_0) \\ L_2 e_2(\theta_0) \\ \vdots \end{bmatrix}.$$

Here the subscripts enumerate all residual blocks, including the prior. Each row block of A has nonzero columns only for the variables used by its factor. An odometry factor touches two consecutive poses. A landmark factor touches one pose and one landmark. This is the source of sparsity.

After solving for δ, update the guess and recompute residuals and Jacobians. For linear models one solve suffices. For nonlinear models this is a local iterative method, and its success depends on the starting guess and the quality of the linear approximation. A small correction is meaningful only if the resulting configuration actually improves the nonlinear fit.

9.2.1 One nonlinear least-squares step with every number visible

As a small illustration of linearization, estimate a scalar x from a prior $x \sim \mathcal{N}(1, 1)$ and an independent observation $z = x^2 + w = 4$, with $w \sim \mathcal{N}(0, 1)$. The negative-log posterior, up to constants and a common factor, is

$$J(x) = (x - 1)^2 + (x^2 - 4)^2.$$

At guess $x^{(0)} = 1$, the residual vector is $(0, -3)^\top$ and its Jacobian is $(1, 2)^\top$. Since both noise variances are one, no additional scaling is needed. The linearized correction minimizes

$$\left\| \begin{bmatrix} 1 \\ 2 \end{bmatrix} \delta - \begin{bmatrix} 0 \\ 3 \end{bmatrix} \right\|^2 .$$

The normal equation is $(1^2 + 2^2)\delta = 1(0) + 2(3)$, giving $\delta = 6/5$ and a new guess $11/5 = 2.2$.

Check the *original* objective: at $x = 1$, $J = 9$. At $x = 11/5$,

$$J = \left(\frac{6}{5}\right)^2 + \left(\frac{21}{25}\right)^2 = \frac{1341}{625} \approx 2.146.$$

This step improved the fit, but it is not the final solution. The tangent has changed, so another iteration recomputes residuals and derivatives. In a larger nonlinear problem a full proposed step may increase the objective; step control or damping is then needed. The linearized solution alone is not a certificate of a global optimum.

9.2.2 Whitening is measuring error in noise units

If a scalar range observation has standard deviation 2 meters and its residual is 3 meters, the normalized residual is $3/2$ and its squared contribution is $9/4$. With standard deviation $1/2$ meter, the same residual has normalized value 6 and contribution 36. The second report is much harder to dismiss because the discrepancy is many of its expected noise units. For independent coordinates, divide each residual by its standard deviation. For correlated coordinates, a matrix whitening factor also accounts for which combined directions are noisy; dividing by diagonal entries alone is not equivalent.

9.2.3 Deriving the normal equations

Expand the squared norm:

$$\|A\delta - b\|^2 = \delta^\top A^\top A\delta - 2\delta^\top A^\top b + b^\top b.$$

Differentiating with respect to δ and setting the gradient to zero yields

$$A^\top A\delta = A^\top b.$$

These are the normal equations. If A has full column rank, meaning its columns are linearly independent, the minimizer is unique. A missing global anchor or an unobserved degree of freedom can cause rank deficiency. Writing an inverse does not resolve such missing information.

9.2.4 Why use QR factorization?

For a tall, full-column-rank matrix, write $A = Q\begin{bmatrix}\mathcal{R}\\0\end{bmatrix}$, where Q is orthogonal and $\mathcal{R}$ is upper triangular. Orthogonal transformations preserve Euclidean length, so

$$\|A\delta - b\|^2 = \left\|\begin{bmatrix}\mathcal{R}\\0\end{bmatrix}\delta - Q^\top b\right\|^2.$$

Partition $Q^\top b = (c_1, c_2)^\top$ to match the row blocks. The objective becomes $\|\mathcal{R}\delta - c_1\|^2 + \|c_2\|^2$. The second term does not depend on δ. Solve the triangular system $\mathcal{R}\delta = c_1$ by back substitution.

QR avoids explicitly forming $A^\top A$, which squares the condition number for a full-rank system. The condition number measures sensitivity to small perturbations; a large value signals that small data or rounding changes can cause large solution changes. This can matter when different constraints produce poorly scaled or nearly dependent directions. The symbols Q and $\mathcal{R}$ here denote factorization matrices, not sensor or motion covariances. Sparse ordering still matters: eliminating a variable can create fill even when the original factors are local.

9.3 Worked loop closure: how a discrepancy is distributed

Consider a scalar robot trajectory with the starting position fixed at $x_0 = 0$. Two odometry measurements each report a displacement of 1. A loop-closure constraint reports that the endpoint displacement from the start is $9/5 = 1.8$. The odometry path alone would end at 2, so the three constraints cannot all be satisfied exactly. The following invented example makes the optimization explicit.

9.3.1 Equal measurement variances

With all three noise variances equal to 1, minimize

$$J(x_1, x_2) = (x_1 - 1)^2 + (x_2 - x_1 - 1)^2 + (x_2 - 9/5)^2.$$

This already is a linear least-squares problem. With $\theta = (x_1, x_2)^\top$,

$$A = \begin{bmatrix} 1 & 0 \\ -1 & 1 \\ 0 & 1 \end{bmatrix}, \qquad b = \begin{bmatrix} 1 \\ 1 \\ 9/5 \end{bmatrix}, \qquad A\theta - b = \begin{bmatrix} x_1 - 1 \\ x_2 - x_1 - 1 \\ x_2 - 9/5 \end{bmatrix}.$$

Multiplication gives

$$A^\top A = \begin{bmatrix} 2 & -1 \\ -1 & 2 \end{bmatrix}, \qquad A^\top b = \begin{bmatrix} 0 \\ 14/5 \end{bmatrix}.$$

Thus $2x_1 - x_2 = 0$ and $-x_1 + 2x_2 = 14/5$. The first gives $x_2 = 2x_1$. Substitution into the second gives $3x_1 = 14/5$, so

$$x_1 = \frac{14}{15}, \qquad x_2 = \frac{28}{15}.$$

The residuals are $-1/15, -1/15, +1/15$. Each odometry displacement is shortened slightly, while the endpoint remains slightly beyond the loop-closure reading. The optimization balances all three constraints according to their equal precision.

9.3.2 A more precise loop-closure measurement

Now give the loop closure variance $1/4$, keeping both odometry variances equal to 1. Its precision is 4, so its squared residual receives weight 4:

$$J = (x_1 - 1)^2 + (x_2 - x_1 - 1)^2 + 4(x_2 - 9/5)^2.$$

The first normal equation remains $2x_1 - x_2 = 0$. The second becomes $-x_1 + 5x_2 = 41/5$. Therefore $x_1 = 41/45$ and $x_2 = 82/45$. The endpoint is now closer to 1.8. The residuals are $-4/45, -4/45, +1/45$: the more precise measurement is fitted more closely, while the less precise motion constraints absorb more of the discrepancy.

This is the role of uncertainty in graph optimization. A loop closure is not automatically enforced exactly, and old poses are not frozen. The solution adjusts all connected unknowns to fit the available constraints with their stated precision.

9.3.3 A point estimate and uncertainty answer different questions

In the equal-variance linear example, the information matrix of the posterior for (x_1, x_2) is $A^\top A = \left[\begin{smallmatrix} 2 & -1 \\ -1 & 2 \end{smallmatrix}\right]$. With the start fixed and the stated independent Gaussian errors, the full posterior is Gaussian, and its covariance is

$$(A^\top A)^{-1} = \frac{1}{3}\begin{bmatrix} 2 & 1 \\ 1 & 2 \end{bmatrix}.$$

Both estimates have variance $2/3$, and they are correlated. The variance of the displacement $x_2 - x_1$ is $2/3 + 2/3 - 2(1/3) = 2/3$.

With the more precise loop closure, the information matrix is $\left[\begin{smallmatrix} 2 & -1 \\ -1 & 5 \end{smallmatrix}\right]$, giving

$$\Sigma = \frac{1}{9}\begin{bmatrix} 5 & 1 \\ 1 & 2 \end{bmatrix}.$$

The endpoint variance falls to $2/9$, while the first pose also benefits indirectly. The optimized means alone do not reveal this change in confidence. For nonlinear SLAM, an inverse local information matrix is a local Gaussian approximation under the chosen linearization and anchoring assumptions, rather than the exact covariance of an arbitrary posterior.

A correct correspondence is equally important. If two different doors are mistaken for the same door, their factor asserts a false relationship. An optimizer can distribute that error smoothly and still produce a wrong map. The mathematics of fitting constraints does not establish that the constraints were correctly recognized. This is why motion, sensing, association, and uncertainty modeling all remain part of the same estimation problem.

Source. Matthew R. Walter, TTIC 31170, Lecture 7.